合肥大學
HEFEI UNIVERSITY




# An Introduction to Optimization

Thomas Weise (汤卫思)
tweise@hfuu.edu.cn · https://thomasweise.github.io

School of Artificial Intelligence and Big Data
Hefei University
Hefei, Anhui, China

人工智能与大数据学院
合肥大学
中国安徽省合肥市

Version: 2026-11-10

# Outline




1. Introduction
2. Problems that we can solve with equations
3. Problems that algorithms can solve fast and efficiently
4. Problems that algorithms cannot solve both efficiently *and* exactly
5. Randomly Guessing Solutions
6. Local Search: Using Information
7. Summary
8. Advertisement




# Introduction



# What is Optimization?

- There are three ways to approach this topic.

# What is Optimization?

- There are three ways to approach this topic
  1. Optimization means searching for superlatives.

# What is Optimization?

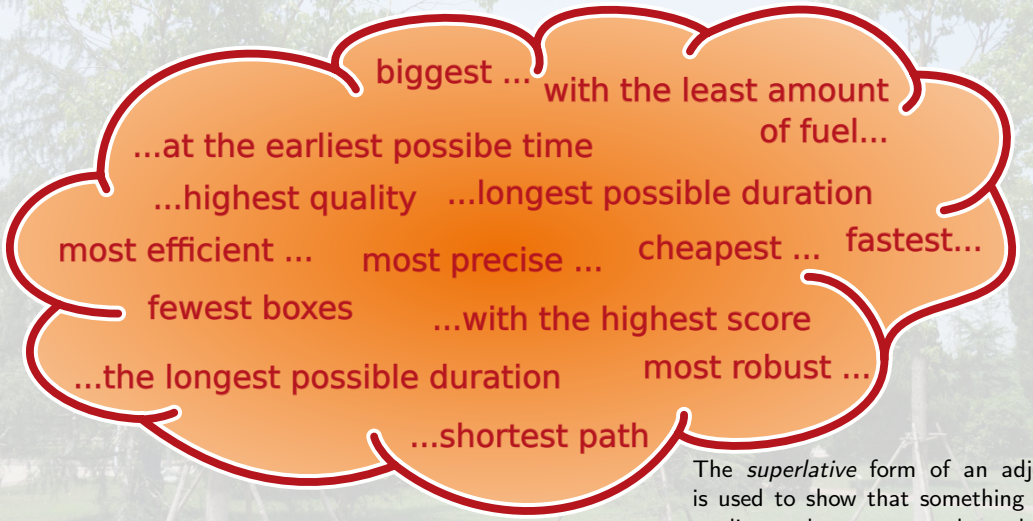
- There are three ways to approach this topic
  1. Optimization means searching for superlatives.
  2. Optimization is the art of solving hard problems.

# What is Optimization?

- There are three ways to approach this topic
  1. Optimization means searching for superlatives.
  2. Optimization is the art of solving hard problems.
  3. Optimization allows us to find acceptable solutions for problems where our tools from high school maths *and* traditional algorithms from computer science are no longer enough, problems where exact solutions cannot always be reached.

# Optimization is the search for superlatives

- Optimization means finding superlatives.




The *superlative* form of an adjective is used to show that something has a quality to the greatest or least degree.

# Optimization is the search for superlatives

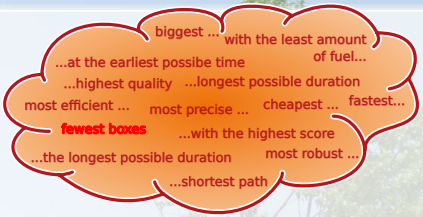
- Optimization means finding superlatives.
- Pick up and deliver packages from different places to customers using the least amount of fuel[174,215–217].

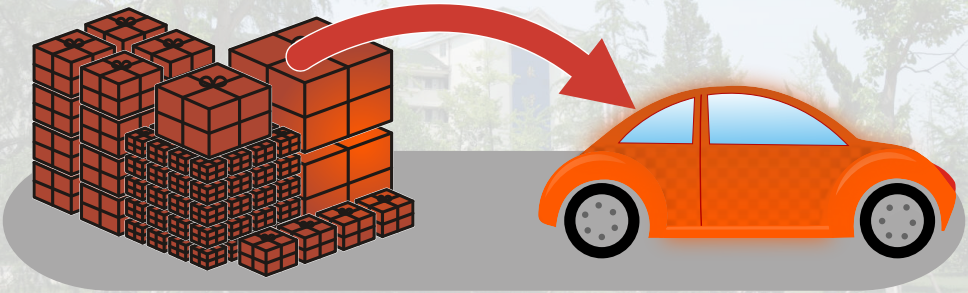





# Optimization is the search for superlatives

- Optimization means finding superlatives.
- Pick up and deliver packages from different places to customers using the least amount of fuel[174,215–217].
- Pack a set of things into the fewest boxes[70,184,233,234].

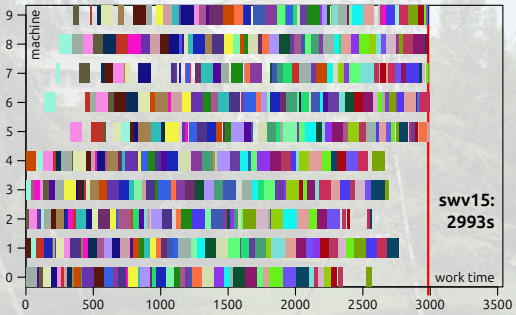


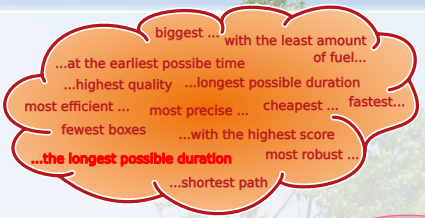
# Optimization is the search for superlatives




- Optimization means finding superlatives.
- Pick up and deliver packages from different places to customers using the least amount of fuel[174,215–217].
- Pack a set of things into the fewest boxes[70,184,233,234].
- Assign tasks to machines such that we can finish our work the at earliest possible time.



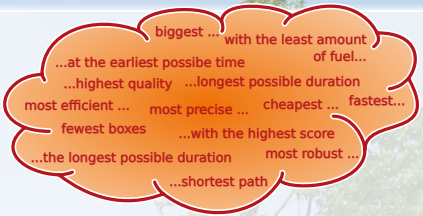
# Optimization is the search for superlatives




- Optimization means finding superlatives.
- Pick up and deliver packages from different places to customers using the least amount of fuel[174,215–217].
- Pack a set of things into the fewest boxes[70,184,233,234].
- Assign tasks to machines such that we can finish our work the at earliest possible time.
- Find a strategy to manage the power of the nodes in this sensor network so that full coverage is guaranteed for the longest possible duration.



# Optimization is the search for superlatives



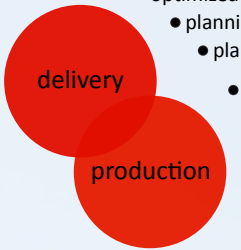

- Optimization means finding superlatives.
- Pick up and deliver packages from different places to customers using the least amount of fuel[174,215–217].
- Pack a set of things into the fewest boxes[70,184,233,234].
- Assign tasks to machines such that we can finish our work the at earliest possible time.
- Find a strategy to manage the power of the nodes in this sensor network so that full coverage is guaranteed for the longest possible duration.
- And so on.

# Optimization in Business

- optimized logistics (business-to-customer)
  - planning and scheduling of maintenance visits
    - planning and scheduling of supply visits



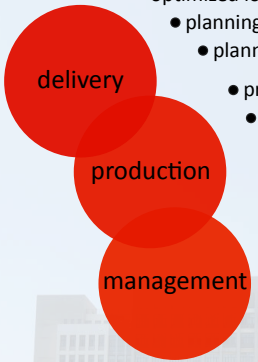
# Optimization in Business




- optimized logistics (business-to-customer)
- planning and scheduling of maintenance visits
- planning and scheduling of supply visits

- production planning and scheduling
- optimized assignment of jobs/orders to machines
- optimization of production processes
- optimization of stock-keeping
- optimization of intra-enterprise logistics
- optimization of supply chains
- optimization of factory layouts and intra-factory logistics

# Optimization in Business




- optimized logistics (business-to-customer)
- planning and scheduling of maintenance visits
- planning and scheduling of supply visits

- production planning and scheduling
- optimized assignment of jobs/orders to machines
- optimization of production processes
- optimization of stock-keeping
- optimization of intra-enterprise logistics
- optimization of supply chains
- optimization of factory layouts and intra-factory logistics

- scheduling of employee work
- optimal assignment of employees to tasks or customers
- optimized locations for new branch offices
  (based current or predicted future customers)

# Optimization in Business




- optimized logistics (business-to-customer)
- planning and scheduling of maintenance visits
- planning and scheduling of supply visits

- production planning and scheduling
- optimized assignment of jobs/orders to machines
- optimization of production processes
- optimization of stock-keeping
- optimization of intra-enterprise logistics
- optimization of supply chains
- optimization of factory layouts and intra-factory logistics

- scheduling of employee work
- optimal assignment of employees to tasks or customers
- optimized locations for new branch offices
  (based current or predicted future customers)

- optimization of product design
- optimization of product feature configuration
- optimization of service offers
- improved tailoring of products/services to customers

# Optimization in Business




- optimized logistics (business-to-customer)
- planning and scheduling of maintenance visits
- planning and scheduling of supply visits

- production planning and scheduling
- optimized assignment of jobs/orders to machines
- optimization of production processes
- optimization of stock-keeping
- optimization of intra-enterprise logistics
- optimization of supply chains
- optimization of factory layouts and intra-factory logistics

- scheduling of employee work
- optimal assignment of employees to tasks or customers
- optimized locations for new branch offices
  (based current or predicted future customers)

- optimization of product design
- optimization of product feature configuration
- optimization of service offers
- improved tailoring of products/services to customers

- optimization of pricing and offers
- mining of customer data for targeted offers

# Optimization in Business




optimization
operations research
artificial intelligence (AI)
computational intelligence
machine learning
data mining

**delivery**

- optimized logistics (business-to-customer)
- planning and scheduling of maintenance visits
- planning and scheduling of supply visits

**production**

- production planning and scheduling
- optimized assignment of jobs/orders to machines
- optimization of production processes
- optimization of stock-keeping
- optimization of intra-enterprise logistics
- optimization of supply chains
- optimization of factory layouts and intra-factory logistics

**management**

- scheduling of employee work
- optimal assignment of employees to tasks or customers
- optimized locations for new branch offices (based current or predicted future customers)

**products/ services**

- optimization of product design
- optimization of product feature configuration
- optimization of service offers
- improved tailoring of products/services to customers

**sales**

- optimization of pricing and offers
- mining of customer data for targeted offers

# Optimization in Business




**optimization**
**operations research**
artificial intelligence (AI)
**computational intelligence**
machine learning
data mining

delivery
- optimized logistics (business-to-customer)
- planning and scheduling of maintenance visits
- planning and scheduling of supply visits

production
- production planning and scheduling
- optimized assignment of jobs/orders to machines
- optimization of production processes
- optimization of stock-keeping
- optimization of intra-enterprise logistics
- optimization of supply chains
- optimization of factory layouts and intra-factory logistics

management
- scheduling of employee work
- optimal assignment of employees to tasks or customers
- optimized locations for new branch offices (based current or predicted future customers)

products/ services
- optimization of product design
- optimization of product feature configuration
- optimization of service offers
- improved tailoring of products/services to customers

sales
- optimization of pricing and offers
- mining of customer data for targeted offers

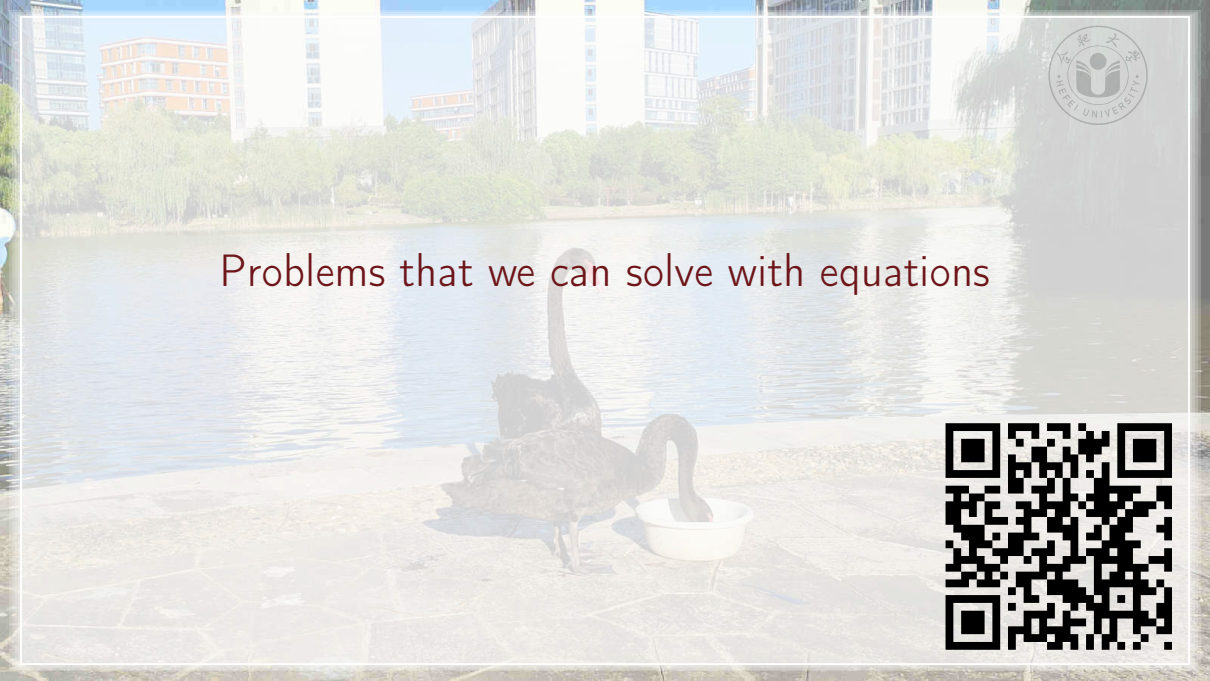
# Optimization

- There are incredibly many problems from a very wide area where we can use optimization and Operations Research (OR).

# Optimization

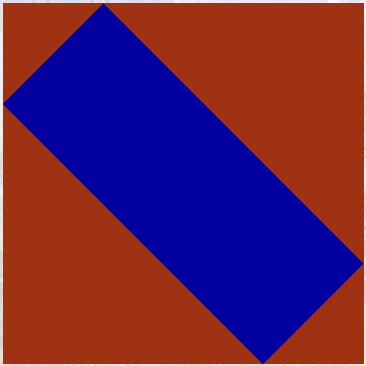
- There are incredibly many problems from a very wide area where we can use optimization and Operations Research (OR).
- How is this related to what you already learned?

# Problems that we can solve with equations



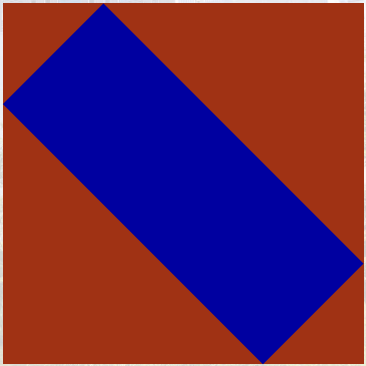
# A Problem that we can solve with (high school Maths) equations

- A rectangle should be placed inside a square with side length 13cm as sketched.



# A Problem that we can solve with (high school Maths) equations

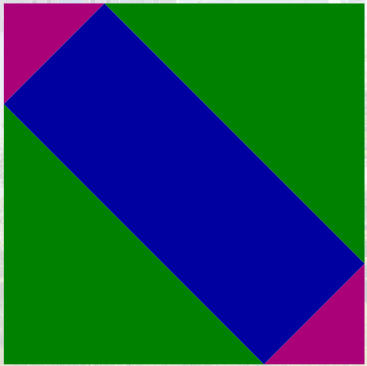
- A rectangle should be placed inside a square with side length 13cm as sketched.
- What area has the *largest* such rectangle?



# A Problem that we can solve with (high school Maths) equations

- A rectangle should be placed inside a square with side length 13cm as sketched.
- What area has the *largest* such rectangle?

$$A_{\blacksquare} = A_{\blacksquare} - 2A_{\blacktriangledown} - 2A_{\blacktriangle}$$

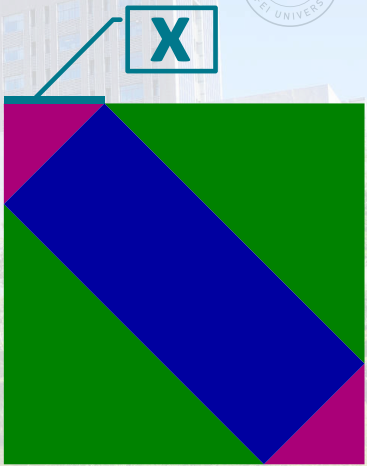


# A Problem that we can solve with (high school Maths) equations

- A rectangle should be placed inside a square with side length 13cm as sketched.
- What area has the *largest* such rectangle?

$$A_{\blacksquare} = A_{\blacksquare} - 2A_{\blacktriangledown} - 2A_{\blacktriangle}$$

$$A_{\blacksquare}(x) = (13\text{cm}^2) - 2\left(\tfrac{1}{2}x^2\right) - 2\left(\tfrac{1}{2}(13\text{cm} - x)^2\right)$$



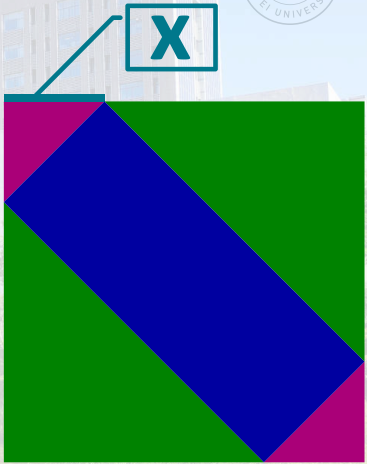
# A Problem that we can solve with (high school Maths) equations

- A rectangle should be placed inside a square with side length 13cm as sketched.
- What area has the *largest* such rectangle?

$$A_{\blacksquare} = A_{\blacksquare} - 2A_{\blacktriangledown} - 2A_{\blacktriangle}$$

$$A_{\blacksquare}(x) = (13\text{cm}^2) - 2\left(\tfrac{1}{2}x^2\right) - 2\left(\tfrac{1}{2}(13\text{cm} - x)^2\right)$$

$$A_{\blacksquare}(x) = 169\text{cm}^2 - x^2 - (13\text{cm} - x)^2$$



# A Problem that we can solve with (high school Maths) equations

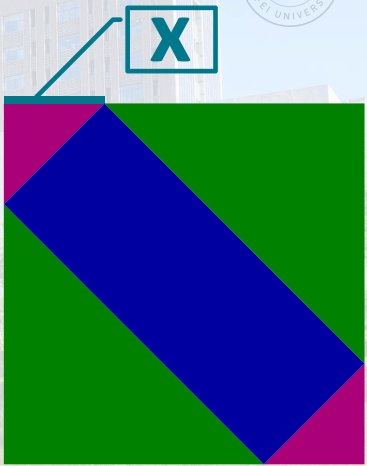
- A rectangle should be placed inside a square with side length 13cm as sketched.
- What area has the *largest* such rectangle?

$$A_{\blacksquare} = A_{\blacksquare} - 2A_{\blacktriangledown} - 2A_{\blacktriangle}$$

$$A_{\blacksquare}(x) = (13\text{cm}^2) - 2\left(\tfrac{1}{2}x^2\right) - 2\left(\tfrac{1}{2}(13\text{cm} - x)^2\right)$$

$$A_{\blacksquare}(x) = 169\text{cm}^2 - x^2 - (13\text{cm} - x)^2$$

$$A_{\blacksquare}(x) = 169\text{cm}^2 - x^2 - 169\text{cm}^2 + 26\text{cm} * x - x^2$$



# A Problem that we can solve with (high school Maths) equations

- A rectangle should be placed inside a square with side length 13cm as sketched.
- What area has the *largest* such rectangle?

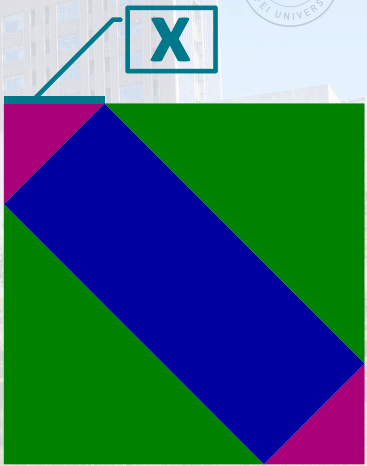
$$A_{\blacksquare} = A_{\blacksquare} - 2A_{\blacktriangledown} - 2A_{\blacktriangle}$$

$$A_{\blacksquare}(x) = (13\text{cm}^2) - 2\left(\tfrac{1}{2}x^2\right) - 2\left(\tfrac{1}{2}(13\text{cm} - x)^2\right)$$

$$A_{\blacksquare}(x) = 169\text{cm}^2 - x^2 - (13\text{cm} - x)^2$$

$$A_{\blacksquare}(x) = 169\text{cm}^2 - x^2 - 169\text{cm}^2 + 26\text{cm} * x - x^2$$

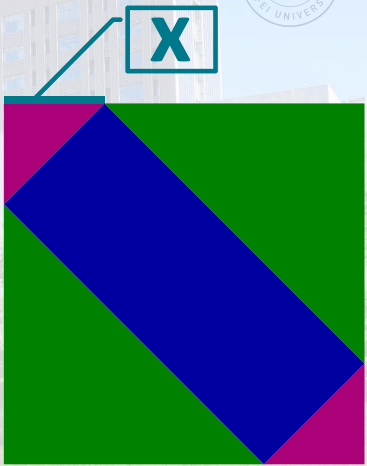
$$A_{\blacksquare}(x) = -2x^2 + 26\text{cm} * x$$



# A Problem that we can solve with (high school Maths) equations

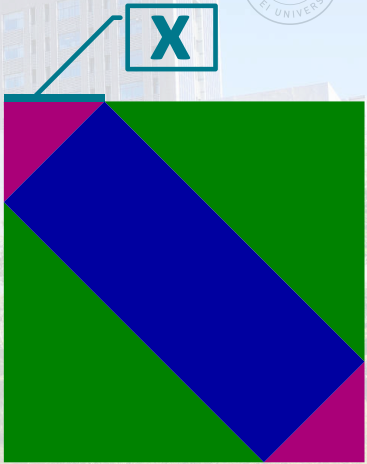
- A rectangle should be placed inside a square with side length 13cm as sketched.
- What area has the *largest* such rectangle?

$$A_{\blacksquare} = A_{\blacksquare} - 2A_{\blacktriangledown} - 2A_{\blacktriangle}$$
$$A_{\blacksquare}(x) = (13\text{cm}^2) - 2\left(\tfrac{1}{2}x^2\right) - 2\left(\tfrac{1}{2}(13\text{cm} - x)^2\right)$$
$$A_{\blacksquare}(x) = 169\text{cm}^2 - x^2 - (13\text{cm} - x)^2$$
$$A_{\blacksquare}(x) = 169\text{cm}^2 - x^2 - 169\text{cm}^2 + 26\text{cm} * x - x^2$$
$$A_{\blacksquare}(x) = -2x^2 + 26\text{cm} * x$$
$$A'_{\blacksquare}(x) = -4x + 26\text{cm}$$



# A Problem that we can solve with (high school Maths) equations

- A rectangle should be placed inside a square with side length 13cm as sketched.
- What area has the *largest* such rectangle?




$$A_{\blacksquare} = A_{\blacksquare} - 2A_{\blacktriangledown} - 2A_{\blacktriangle}$$

$$A_{\blacksquare}(x) = (13\text{cm}^2) - 2\left(\tfrac{1}{2}x^2\right) - 2\left(\tfrac{1}{2}(13\text{cm} - x)^2\right)$$

$$A_{\blacksquare}(x) = 169\text{cm}^2 - x^2 - (13\text{cm} - x)^2$$

$$A_{\blacksquare}(x) = 169\text{cm}^2 - x^2 - 169\text{cm}^2 + 26\text{cm} * x - x^2$$

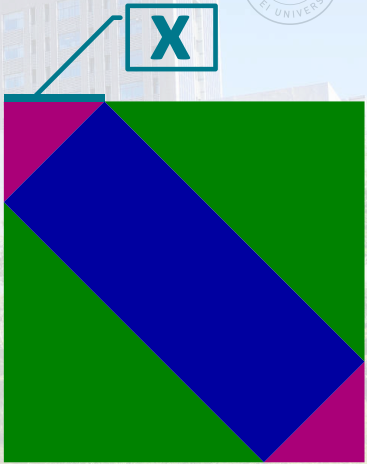
$$A_{\blacksquare}(x) = -2x^2 + 26\text{cm} * x$$

$$A'_{\blacksquare}(x) = -4x + 26\text{cm}$$

$$0 = -4\hat{x} + 26\text{cm}$$

# A Problem that we can solve with (high school Maths) equations

- A rectangle should be placed inside a square with side length 13cm as sketched.
- What area has the *largest* such rectangle?




$$A_{▮} = A_{■} - 2A_{▼} - 2A_{▲}$$

$$A_{▮}(x) = (13\text{cm}^2) - 2\left(\tfrac{1}{2}x^2\right) - 2\left(\tfrac{1}{2}(13\text{cm} - x)^2\right)$$

$$A_{▮}(x) = 169\text{cm}^2 - x^2 - (13\text{cm} - x)^2$$

$$A_{▮}(x) = 169\text{cm}^2 - x^2 - 169\text{cm}^2 + 26\text{cm} * x - x^2$$

$$A_{▮}(x) = -2x^2 + 26\text{cm} * x$$

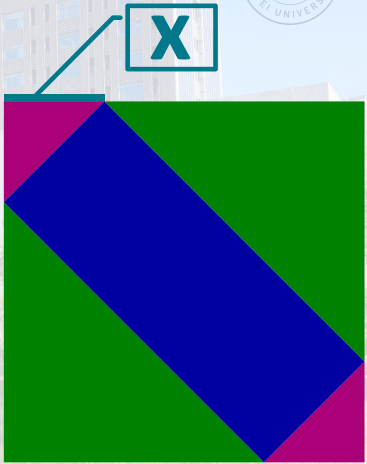
$$A'_{▮}(x) = -4x + 26\text{cm}$$

$$0 = -4\hat{x} + 26\text{cm}$$

$$4\hat{x} = 26\text{cm}$$

# A Problem that we can solve with (high school Maths) equations

- A rectangle should be placed inside a square with side length 13cm as sketched.
- What area has the *largest* such rectangle?



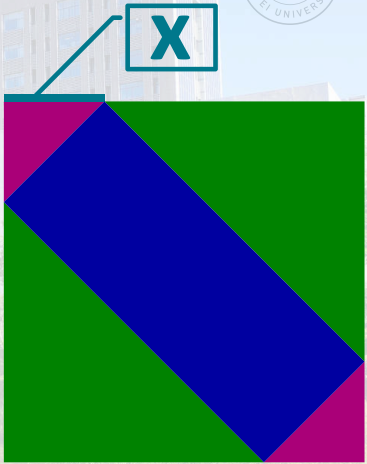

$$
\begin{aligned}
A_{\blacksquare} &= A_{\blacksquare} - 2A_{\blacktriangledown} - 2A_{\blacktriangle} \\
A_{\blacksquare}(x) &= (13\text{cm}^2) - 2\left(\tfrac{1}{2}x^2\right) - 2\left(\tfrac{1}{2}(13\text{cm} - x)^2\right) \\
A_{\blacksquare}(x) &= 169\text{cm}^2 - x^2 - (13\text{cm} - x)^2 \\
A_{\blacksquare}(x) &= 169\text{cm}^2 - x^2 - 169\text{cm}^2 + 26\text{cm} * x - x^2 \\
A_{\blacksquare}(x) &= -2x^2 + 26\text{cm} * x \\
A'_{\blacksquare}(x) &= -4x + 26\text{cm} \\
0 &= -4\hat{x} + 26\text{cm} \\
4\hat{x} &= 26\text{cm} \\
\hat{x} &= 6.5\text{cm}
\end{aligned}
$$

# A Problem that we can solve with (high school Maths) equations

- A rectangle should be placed inside a square with side length 13cm as sketched.
- What area has the *largest* such rectangle?

$$A_{\blacksquare}(x) = -2x^2 + 26\text{cm} * x$$

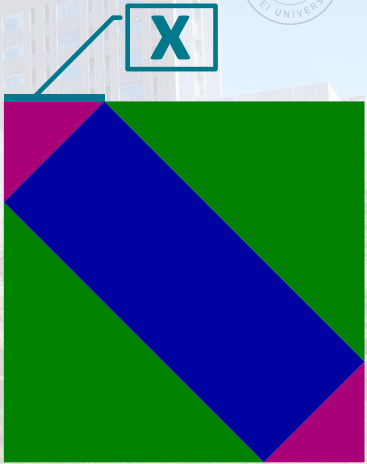
$$\hat{x} = 6.5\text{cm}$$



# A Problem that we can solve with (high school Maths) equations

- A rectangle should be placed inside a square with side length 13cm as sketched.
- What area has the *largest* such rectangle?

$$A_{\blacksquare}(x) = -2x^2 + 26\text{cm} * x$$
$$\hat{x} = 6.5\text{cm}$$
$$\widehat{A_{\blacksquare}} = A_{\blacksquare}(\hat{x}) = A_{\blacksquare}(6.5\text{cm})$$
$$\widehat{A_{\blacksquare}} = -2(6.5\text{cm})^2 + 26\text{cm} * 6.5\text{cm}$$



# A Problem that we can solve with (high school Maths) equations

- A rectangle should be placed inside a square with side length 13cm as sketched.
- What area has the *largest* such rectangle?

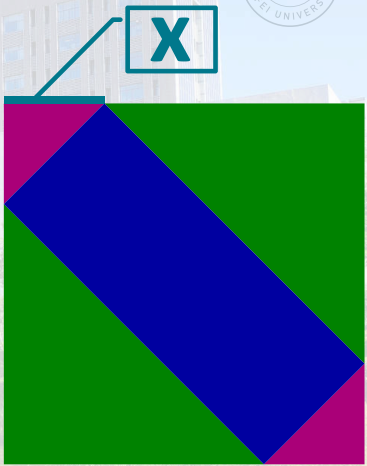
$$A_{\blacksquare}(x) = -2x^2 + 26\text{cm} * x$$

$$\hat{x} = 6.5\text{cm}$$

$$\widehat{A_{\blacksquare}} = A_{\blacksquare}(\hat{x}) = A_{\blacksquare}(6.5\text{cm})$$

$$\widehat{A_{\blacksquare}} = -2(6.5\text{cm})^2 + 26\text{cm} * 6.5\text{cm}$$

$$\widehat{A_{\blacksquare}} = 84.5\text{cm}^2$$



## A Problem that we can solve with (high school Maths) equations

- A rectangle should be placed inside a square with side length 13cm as sketched.
- What area has the *largest* such rectangle?

$$A_{\blacksquare}(x) = -2x^2 + 26\text{cm} * x$$

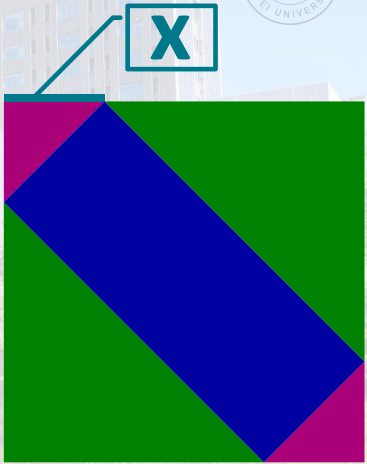
$$\hat{x} = 6.5\text{cm}$$

$$\widehat{A_{\blacksquare}} = A_{\blacksquare}(\hat{x}) = A_{\blacksquare}(6.5\text{cm})$$

$$\widehat{A_{\blacksquare}} = -2(6.5\text{cm})^2 + 26\text{cm} * 6.5\text{cm}$$

$$\widehat{A_{\blacksquare}} = 84.5\text{cm}^2$$

**Solved.**



## Problems that we can solve with equations

- We can actually solve a lot of problems.

## Problems that we can solve with equations

- We can actually solve a lot of problems.
- We just used an equation.

## Problems that we can solve with equations

- We can actually solve a lot of problems.
- We just used an equation.
- We did computations in multiple steps.

## Problems that we can solve with equations

- We can actually solve a lot of problems.
- We just used an equation.
- We did computations in multiple steps.
- Regardless how the previous problem would be parameterized (say, 16cm instead of 13cm), we could perform the exactly same steps.

## Problems that we can solve with equations

- We can actually solve a lot of problems.
- We just used an equation.
- We did computations in multiple steps.
- Regardless how the previous problem would be parameterized (say, 16cm instead of 13cm), we could perform the exactly same steps.
- Can we always do that?

Problems that algorithms can solve fast and efficiently



# Find the Shortest Path from Start to Goal



# Find the Shortest Path from Start to Goal




- I am at the starting point 合肥大学南二区北大门.

# Find the Shortest Path from Start to Goal




- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.

# Find the Shortest Path from Start to Goal




- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- How do I get there the fastest?

# Find the Shortest Path from Start to Goal



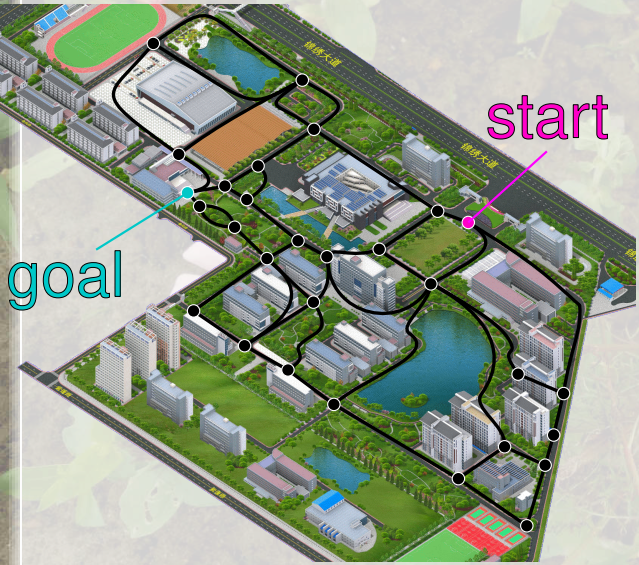

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- How do I get there the fastest?
- We know the campus map.

# Find the Shortest Path from Start to Goal



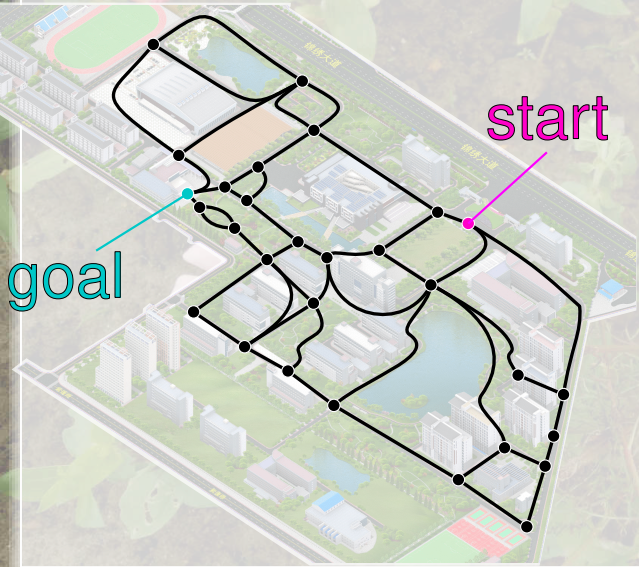

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- How do I get there the fastest?
- We know the campus map.
- We want to compute the shortest path (before actually walking it).

# Find the Shortest Path from Start to Goal



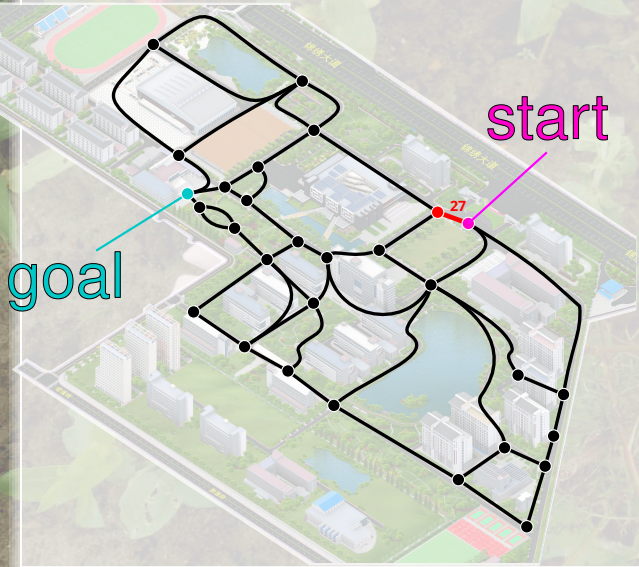

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- How do I get there the fastest?
- We know the campus map.
- We want to compute the shortest path (before actually walking it).
- We know all the intersections where I could make turns.

# Find the Shortest Path from Start to Goal



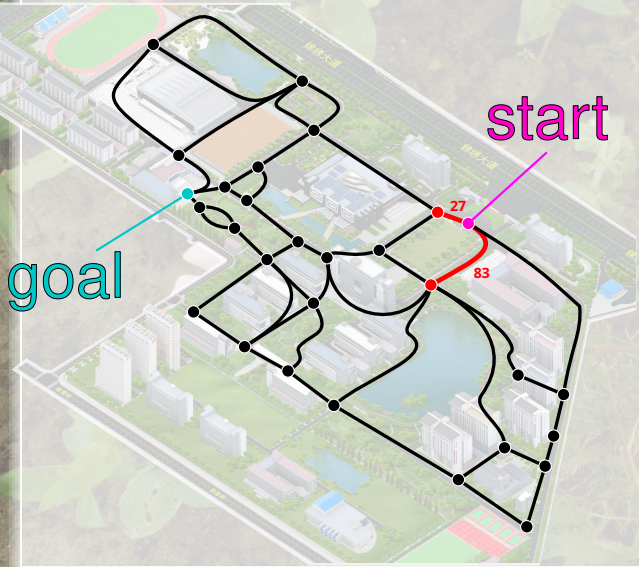

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- How do I get there the fastest?
- We know the campus map.
- We want to compute the shortest path (before actually walking it).
- We know all the intersections where I could make turns.

# Find the Shortest Path from Start to Goal



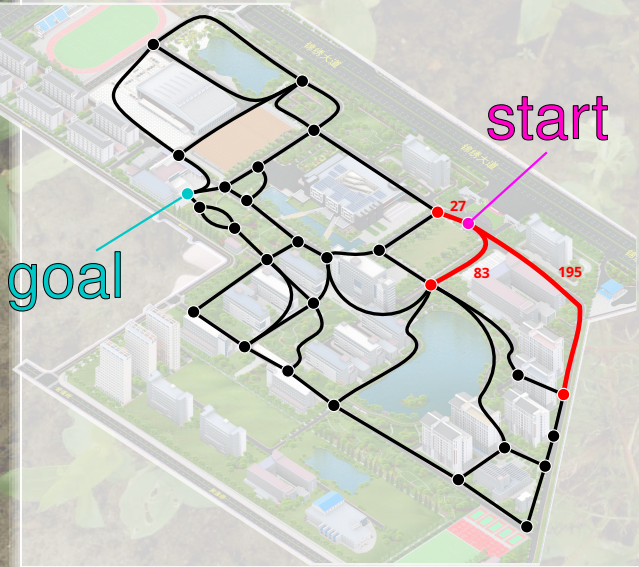

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- For example, I could walk for 27$s$ to this intersection.

# Find the Shortest Path from Start to Goal



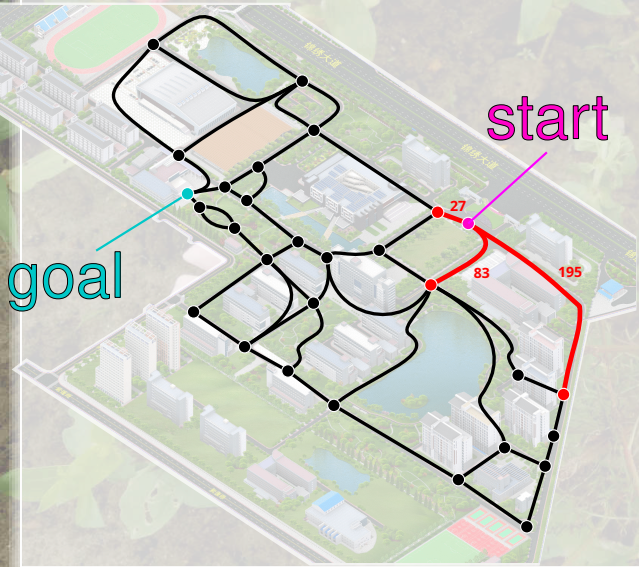

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- For example, I could walk for 27$s$ to this intersection.
- Or for 83$s$ to that one.

# Find the Shortest Path from Start to Goal



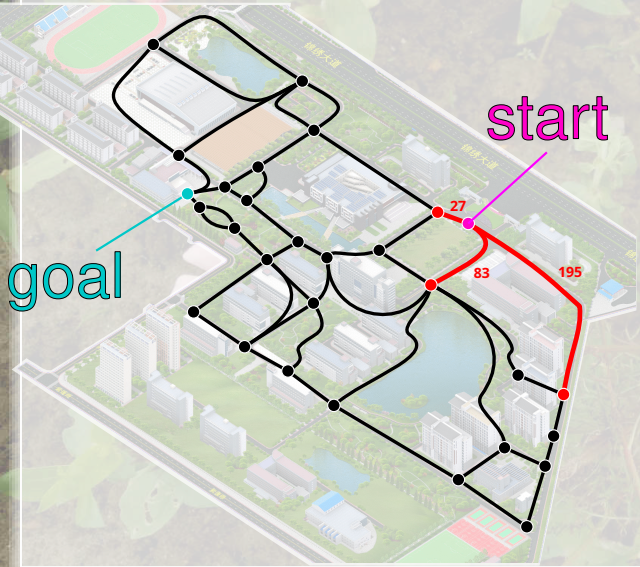

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- For example, I could walk for $27s$ to this intersection.
- Or for $83s$ to that one.
- Or for $195s$ to that one.

# Find the Shortest Path from Start to Goal



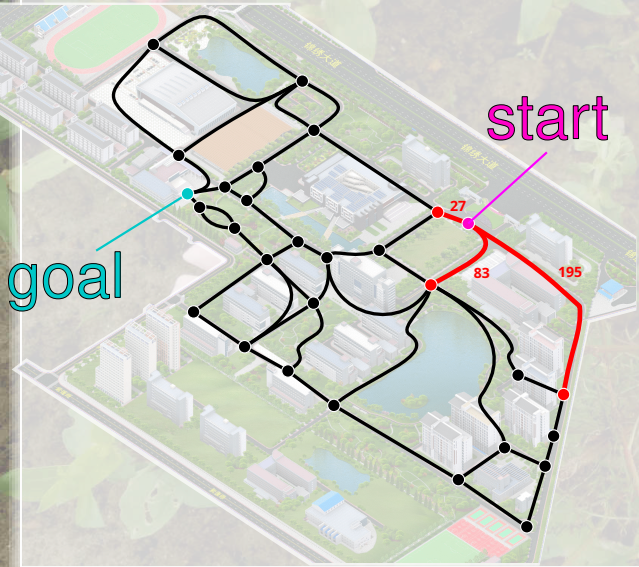

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- For example, I could walk for $27s$ to this intersection.
- Or for $83s$ to that one.
- Or for $195s$ to that one.
- Which one should we pick?

# Find the Shortest Path from Start to Goal



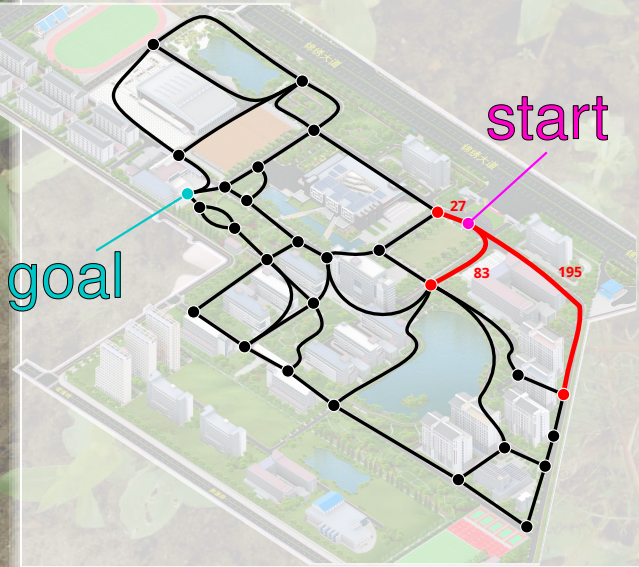

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We have 3 choices: (a) $27s$, (b) $83s$, (c) $195s$.

# Find the Shortest Path from Start to Goal



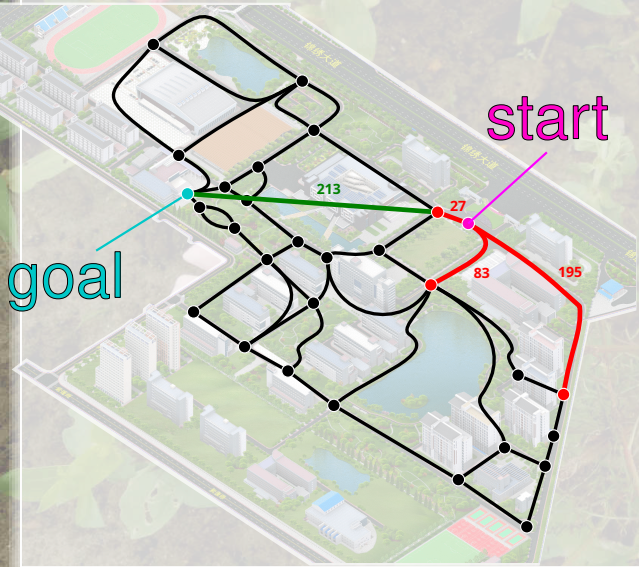

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We have 3 choices: (a) $27s$, (b) $83s$, (c) $195s$.
- For each intersection, we can compute the airline distance (as the crow flies) to the 食堂.

# Find the Shortest Path from Start to Goal



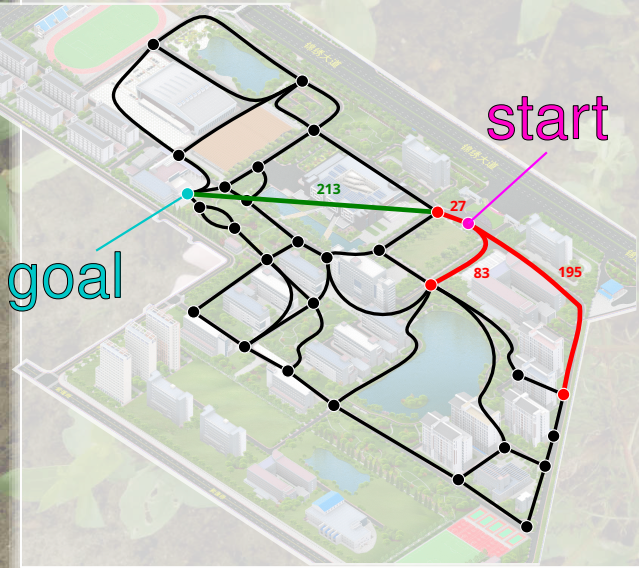

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We have 3 choices: (a) $27s$, (b) $83s$, (c) $195s$.
- For each intersection, we can compute the airline distance (as the crow flies) to the 食堂.
- The actual walking distance can never be shorter than that.

# Find the Shortest Path from Start to Goal



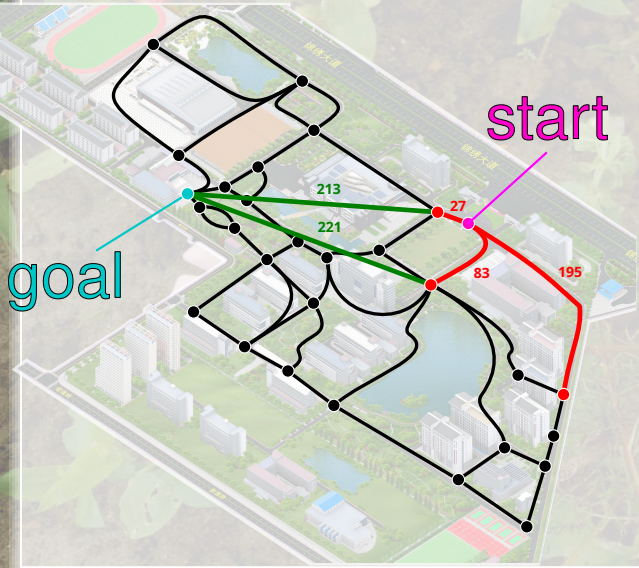

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We have 3 choices: (a) $27s$, (b) $83s$, (c) $195s$.
- For each intersection, we can compute the airline distance (as the crow flies) to the 食堂.
- The actual walking distance can never be shorter than that.

# Find the Shortest Path from Start to Goal



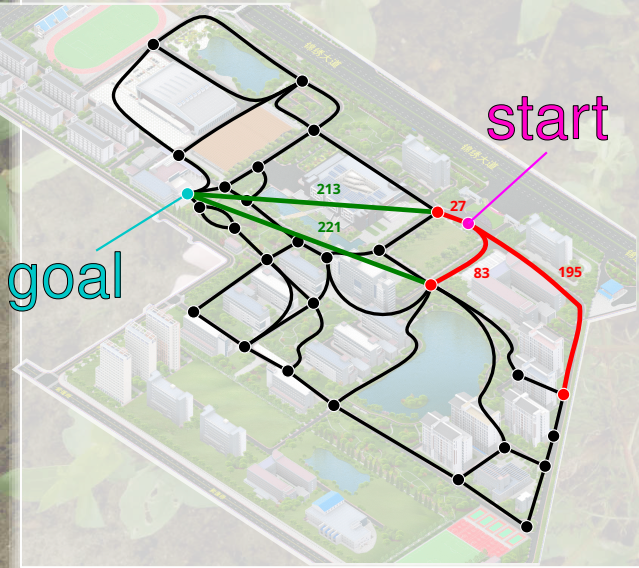

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We have 3 choices: (a) $27s + 213s$, (b) $83s$, (c) $195s$.
- For each intersection, we can compute the airline distance (as the crow flies) to the 食堂.
- The actual walking distance can never be shorter than that.

# Find the Shortest Path from Start to Goal



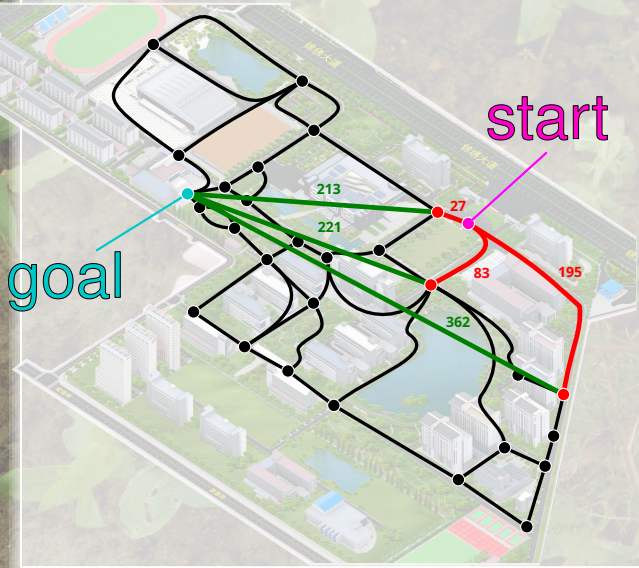

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We have 3 choices: (a) $27s + 213s$, (b) $83s$, (c) $195s$.
- For each intersection, we can compute the airline distance (as the crow flies) to the 食堂.
- The actual walking distance can never be shorter than that.

# Find the Shortest Path from Start to Goal



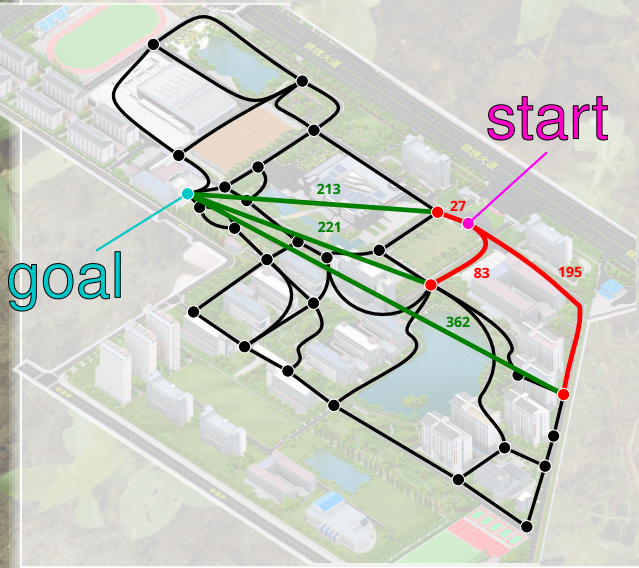

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We have 3 choices: (a) $27s + 213s$, (b) $83s + 221s$, (c) $195s$.
- For each intersection, we can compute the airline distance (as the crow flies) to the 食堂.
- The actual walking distance can never be shorter than that.

# Find the Shortest Path from Start to Goal



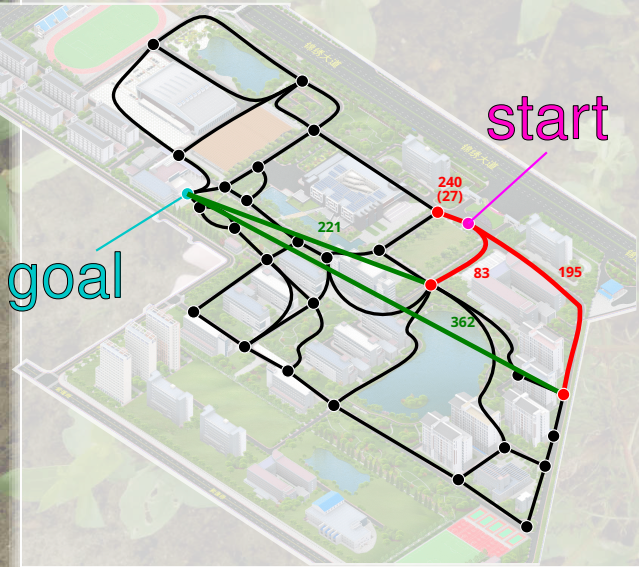

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We have 3 choices: (a) $27s + 213s$, (b) $83s + 221s$, (c) $195s$.
- For each intersection, we can compute the airline distance (as the crow flies) to the 食堂.
- The actual walking distance can never be shorter than that.

# Find the Shortest Path from Start to Goal



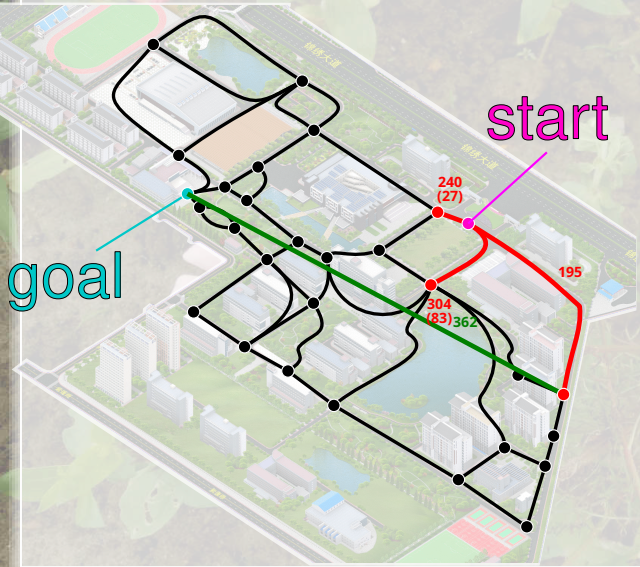

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We have 3 choices: (a) $27s + 213s$, (b) $83s + 221s$, (c) $195s + 362s$.
- For each intersection, we can compute the airline distance (as the crow flies) to the 食堂.
- The actual walking distance can never be shorter than that.

# Find the Shortest Path from Start to Goal



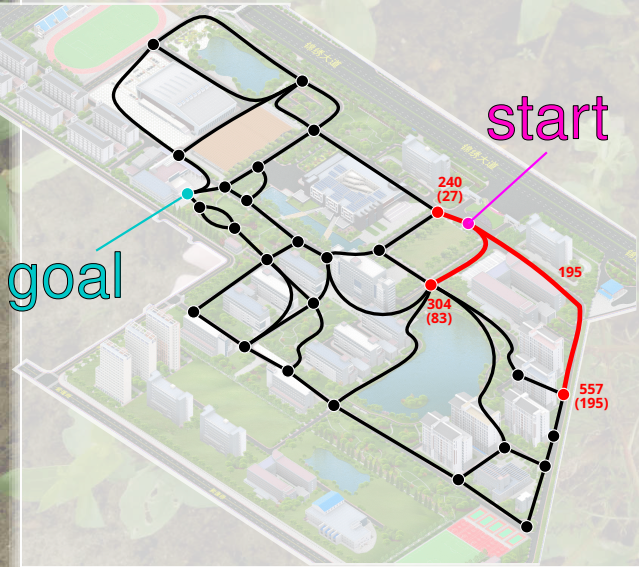

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We have 3 choices:
  (a) $240s = 27s + 213s$, (b) $83s + 221s$,
  (c) $195s + 362s$.

# Find the Shortest Path from Start to Goal



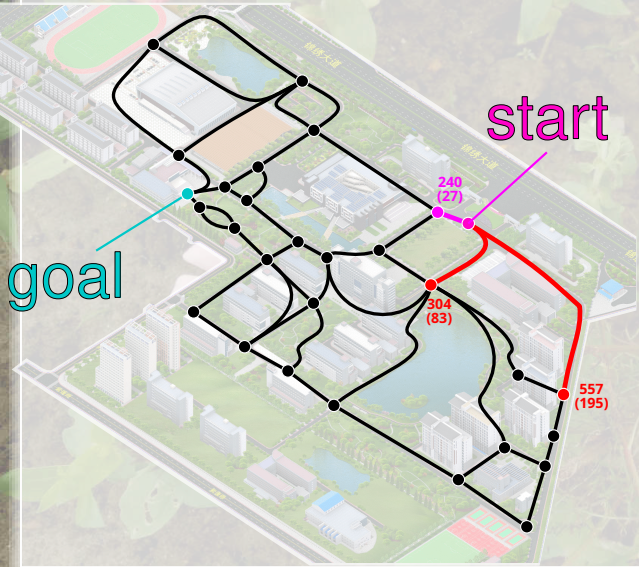

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We have 3 choices:
  (a) $240s = 27s + 213s$,
  (b) $304s = 83s + 221s$, (c) $195s + 362s$.

# Find the Shortest Path from Start to Goal



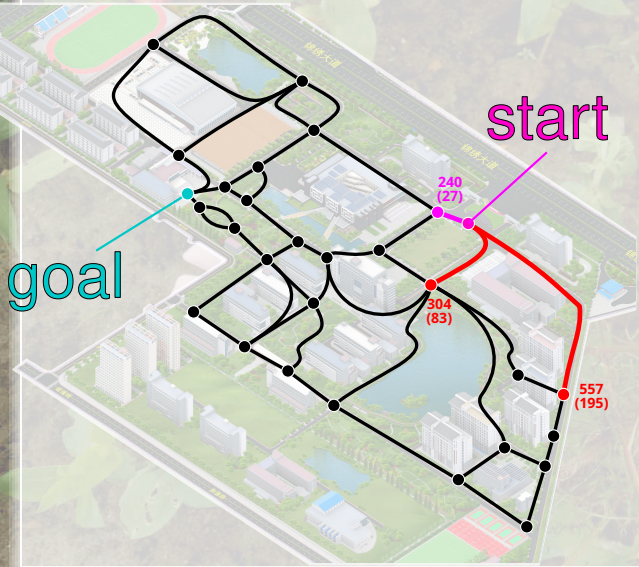

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We have 3 choices:
  (a) $240s = 27s + 213s$,
  (b) $304s = 83s + 221s$,
  (c) $557s = 195s + 362s$.

# Find the Shortest Path from Start to Goal



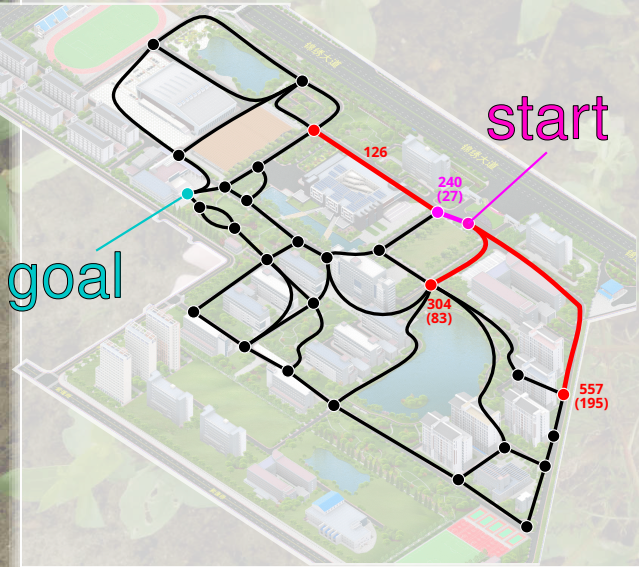

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We have 3 choices:
  (a) $240s = 27s + 213s$,
  (b) $304s = 83s + 221s$,
  (c) $557s = 195s + 362s$.
- The most interesting candidate for the first step is clearly (a).

# Find the Shortest Path from Start to Goal



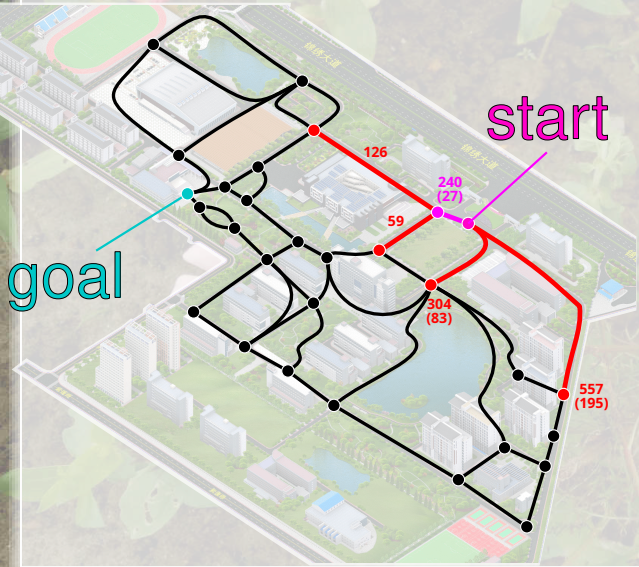

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- The most interesting candidate for the first step is clearly (a).
- We have 2 remaining unexplored choices:
  (b) $304s = 83s + 221s$,
  (c) $557s = 195s + 362s$.

# Find the Shortest Path from Start to Goal



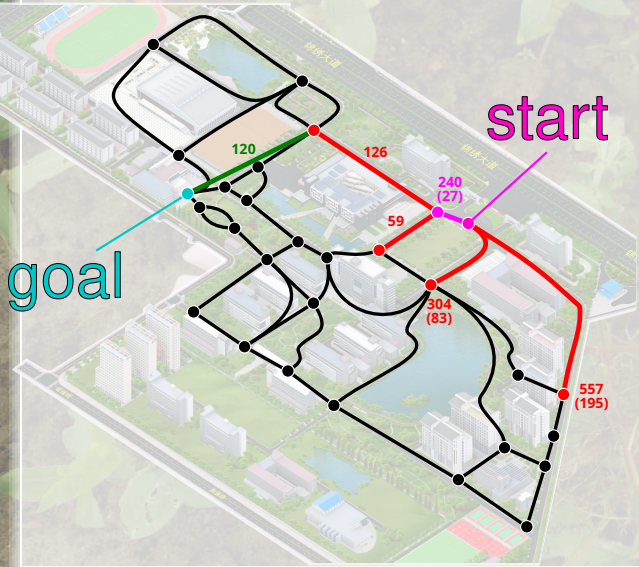

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- The most interesting candidate for the first step is clearly (a).
- We have 2 remaining unexplored choices:
  (b) $304s = 83s + 221s$,
  (c) $557s = 195s + 362s$.
- From (a), we can go to (d) by walking for $126s$.

# Find the Shortest Path from Start to Goal



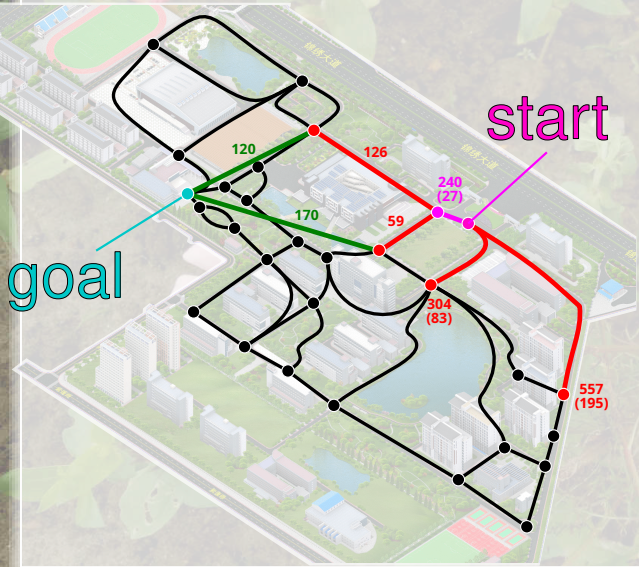

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- The most interesting candidate for the first step is clearly (a).
- We have 2 remaining unexplored choices:
  (b) $304s = 83s + 221s$,
  (c) $557s = 195s + 362s$.
- From (a), we can go to (d) by walking for $126s$ or to (e) by walking for $59s$.

# Find the Shortest Path from Start to Goal



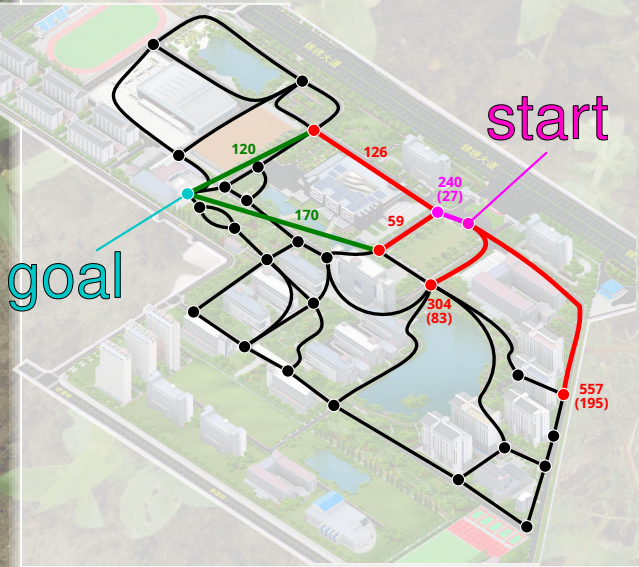

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- The most interesting candidate for the first step is clearly (a).
- We have 2 remaining unexplored choices:
  (b) $304s = 83s + 221s$,
  (c) $557s = 195s + 362s$.
- From (a), we can go to (d) by walking for $126s$ or to (e) by walking for $59s$.
- For both we compute the distance as the crow flies.

# Find the Shortest Path from Start to Goal



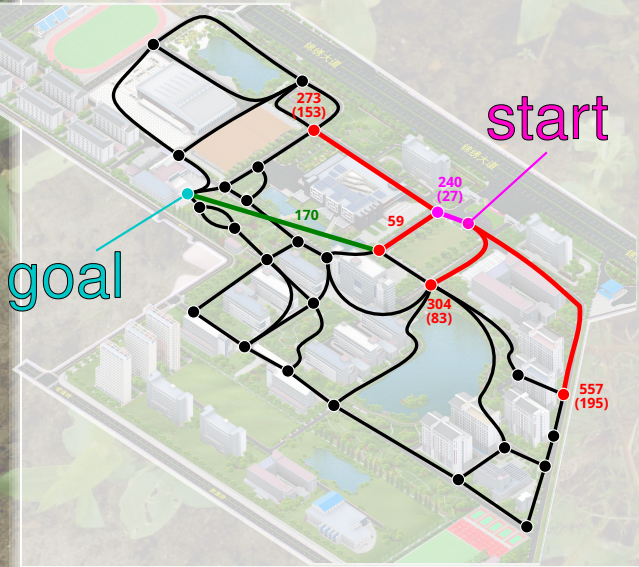

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- The most interesting candidate for the first step is clearly (a).
- From (a), we can go to (d) by walking for $126s$ or to (e) by walking for $59s$.
- For both we compute the distance as the crow flies.

# Find the Shortest Path from Start to Goal



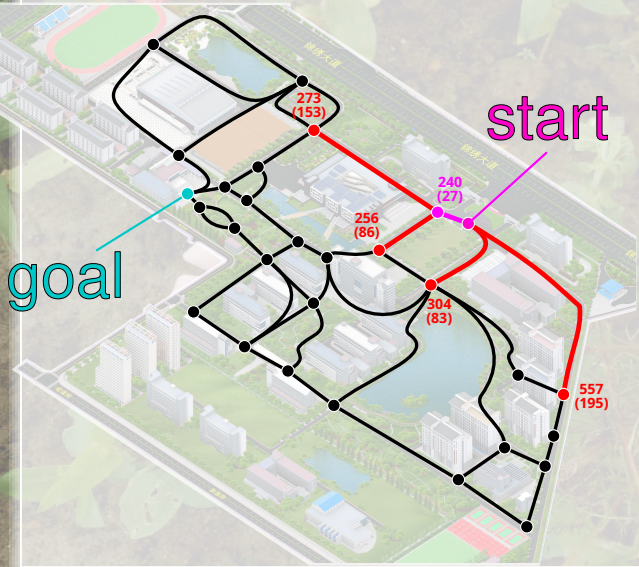

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- The most interesting candidate for the first step is clearly (a).
- From (a), we can go to (d) by walking for $126s$ or to (e) by walking for $59s$.
- For both we compute the distance as the crow flies.
- We had 2 remaining unexplored choices:
  (b) $304s = 83s + 221s$,
  (c) $557s = 195s + 362s$.

# Find the Shortest Path from Start to Goal



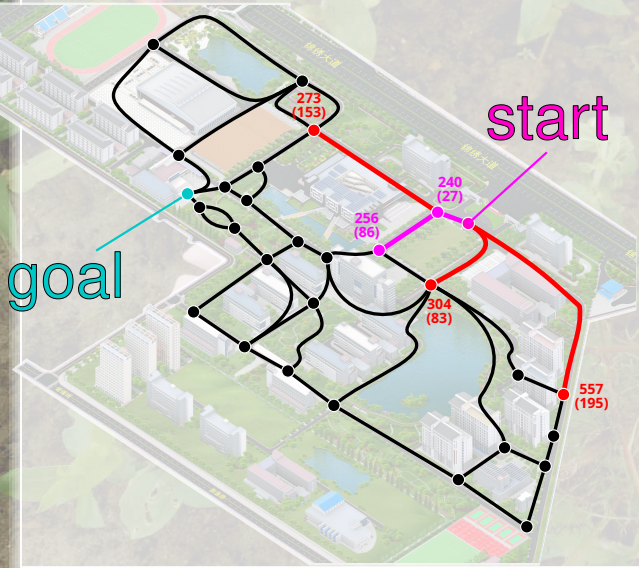

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- The most interesting candidate for the first step is clearly (a).
- From (a), we can go to (d) by walking for $126s$ or to (e) by walking for $59s$.
- For both we compute the distance as the crow flies.
- We now have 3 unexplored choices:
  (b) $304s = 83s + 221s$,
  (c) $557s = 195s + 362s$,
  (d) $273s = 153s + 120s$.

# Find the Shortest Path from Start to Goal



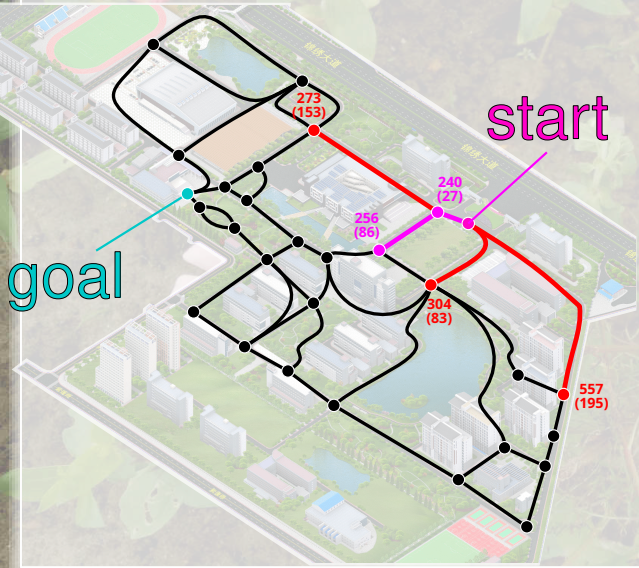

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- The most interesting candidate for the first step is clearly (a).
- From (a), we can go to (d) by walking for $126s$ or to (e) by walking for $59s$.
- For both we compute the distance as the crow flies.
- We now have 4 unexplored choices:
  (b) $304s = 83s + 221s$,
  (c) $557s = 195s + 362s$,
  (d) $273s = 153s + 120s$,
  (e) $256s = 86s + 170s$.

# Find the Shortest Path from Start to Goal



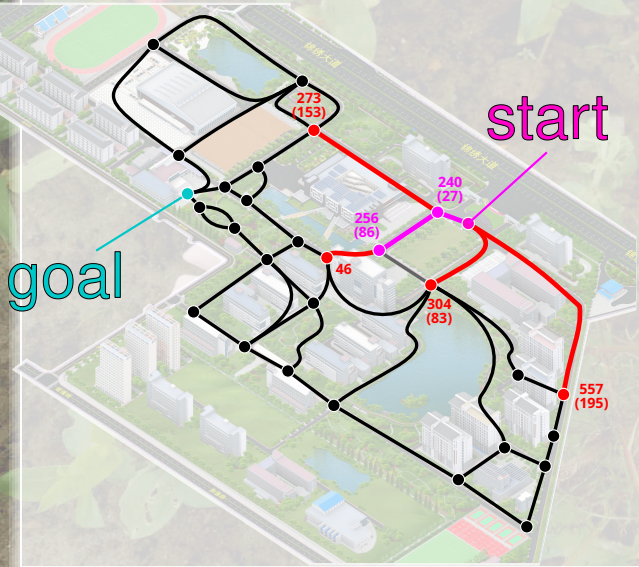

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We now have 4 unexplored choices:
  (b) $304s = 83s + 221s$,
  (c) $557s = 195s + 362s$,
  (d) $273s = 153s + 120s$,
  (e) $256s = 86s + 170s$.
- (e) it is...

# Find the Shortest Path from Start to Goal



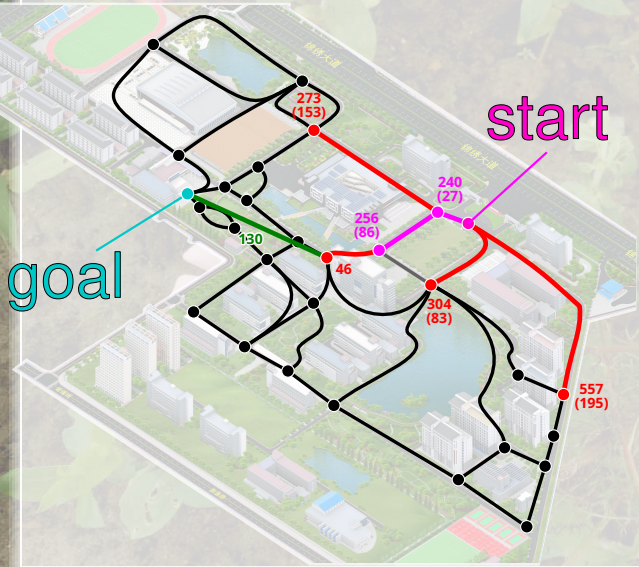

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- (e) it is...
- We have 3 remaining unexplored choices:
  (b) $304s = 83s + 221s$,
  (c) $557s = 195s + 362s$,
  (d) $273s = 153s + 120s$.

# Find the Shortest Path from Start to Goal



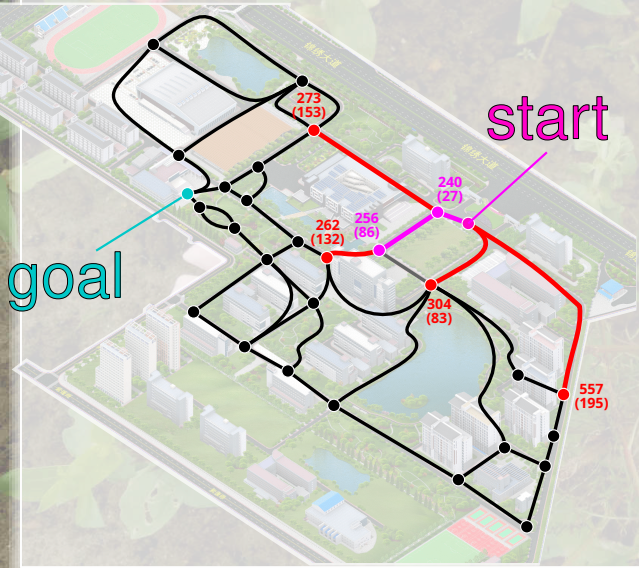

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We have 3 remaining unexplored choices:
  (b) $304s = 83s + 221s$,
  (c) $557s = 195s + 362s$,
  (d) $273s = 153s + 120s$.
- From this point (e), we only have one reasonable choice where to go next.

# Find the Shortest Path from Start to Goal



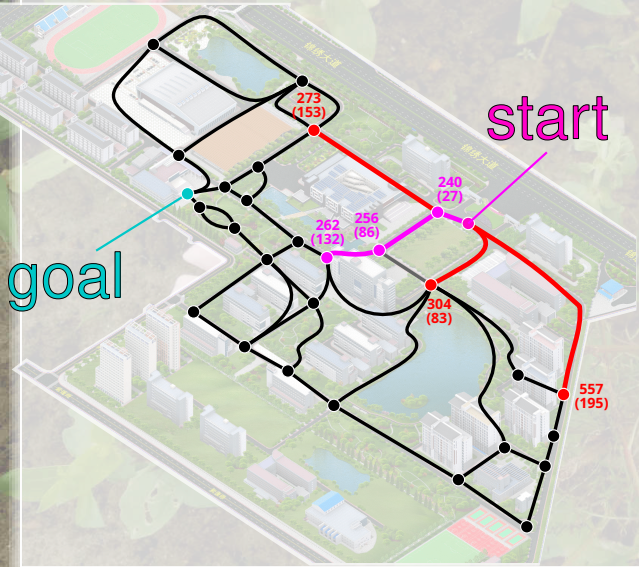

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We have 3 remaining unexplored choices:
  (b) $304s = 83s + 221s$,
  (c) $557s = 195s + 362s$,
  (d) $273s = 153s + 120s$.
- From this point (e), we only have one reasonable choice where to go next.
- And we again compute the airline distance to the goal.

# Find the Shortest Path from Start to Goal



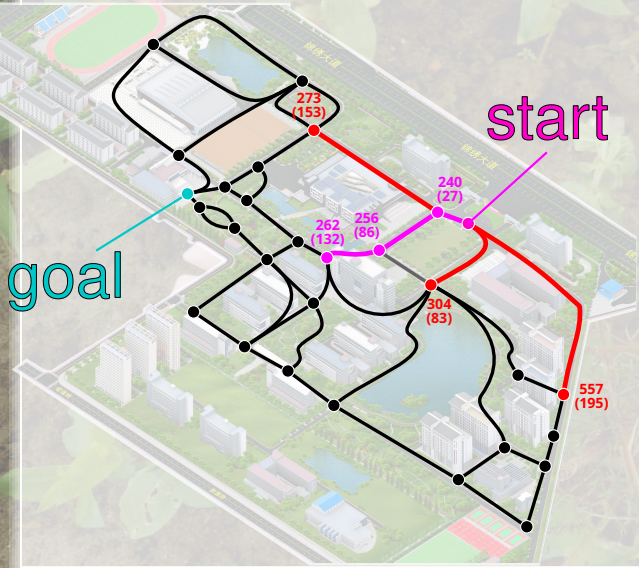

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- From this point (e), we only have one reasonable choice where to go next.
- And we again compute the airline distance to the goal.
- We now have 4 choices:
  (b) $304s = 83s + 221s$,
  (c) $557s = 195s + 362s$,
  (d) $273s = 153s + 120s$,
  (f) $262s = 132s + 130s$.

# Find the Shortest Path from Start to Goal



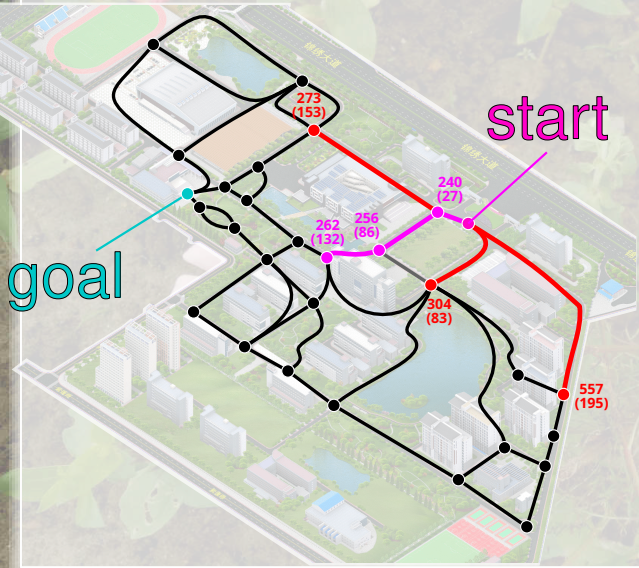

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- And we again compute the airline distance to the goal.
- We now have 4 choices:
  (b) $304s = 83s + 221s$,
  (c) $557s = 195s + 362s$,
  (d) $273s = 153s + 120s$,
  (f) $262s = 132s + 130s$.
- (f) it is. . .

# Find the Shortest Path from Start to Goal



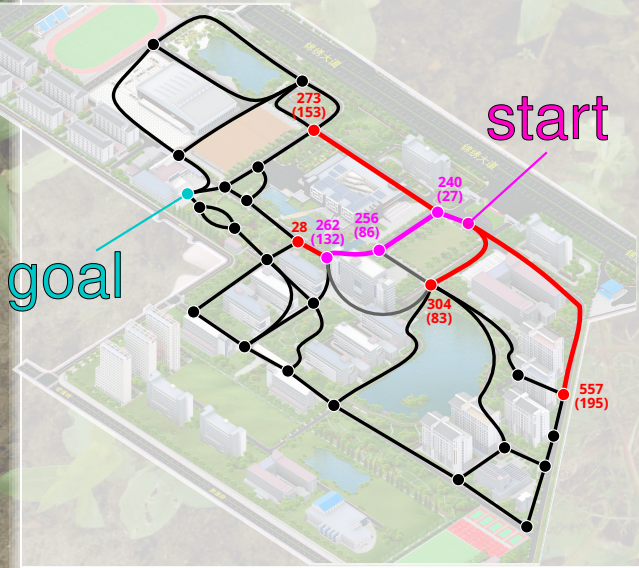

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- And we again compute the airline distance to the goal.
- (f) it is...
- We have 3 remaining unexplored choices:
  (b) $304s = 83s + 221s$,
  (c) $557s = 195s + 362s$,
  (d) $273s = 153s + 120s$.

# Find the Shortest Path from Start to Goal



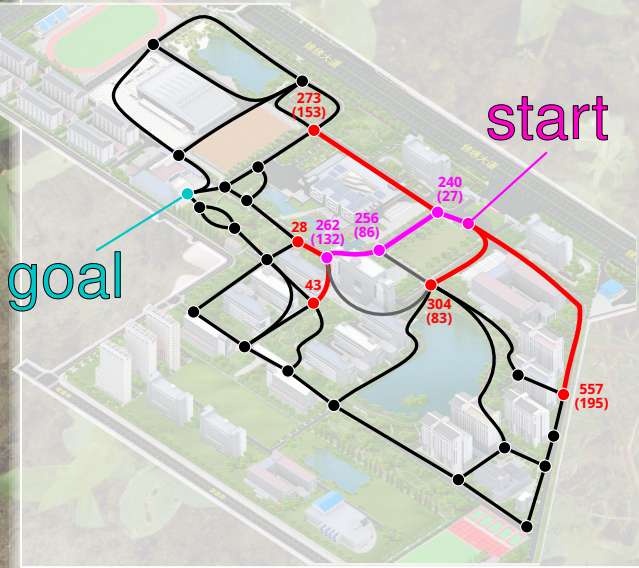

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- And we again compute the airline distance to the goal.
- (f) it is...
- We have 3 remaining unexplored choices:
  (b) $304s = 83s + 221s$,
  (c) $557s = 195s + 362s$,
  (d) $273s = 153s + 120s$.
- From (f), we have two possible choices to continue.

# Find the Shortest Path from Start to Goal



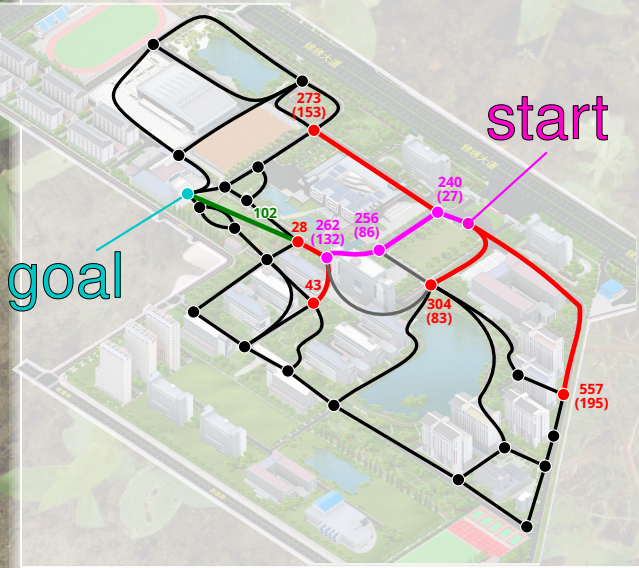

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- And we again compute the airline distance to the goal.
- (f) it is...
- We have 3 remaining unexplored choices:
  (b) $304s = 83s + 221s$,
  (c) $557s = 195s + 362s$,
  (d) $273s = 153s + 120s$.
- From (f), we have two possible choices to continue.

# Find the Shortest Path from Start to Goal



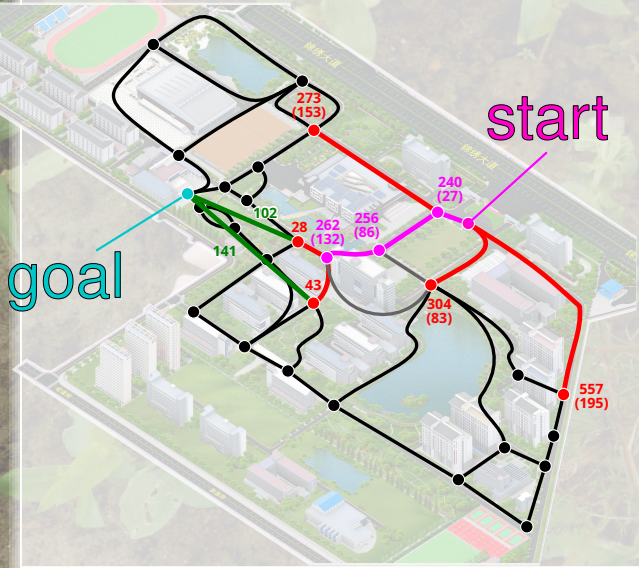

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- And we again compute the airline distance to the goal.
- (f) it is...
- We have 3 remaining unexplored choices:
  (b) $304s = 83s + 221s$,
  (c) $557s = 195s + 362s$,
  (d) $273s = 153s + 120s$.
- From (f), we have two possible choices to continue.

# Find the Shortest Path from Start to Goal



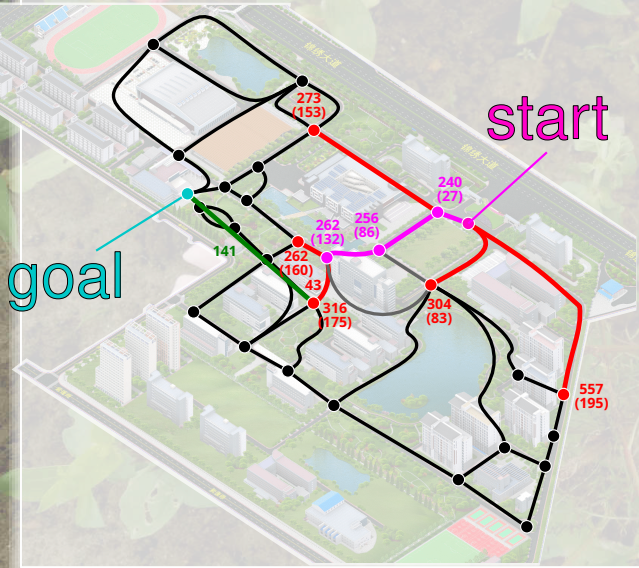

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- And we again compute the airline distance to the goal.
- (f) it is...
- We have 3 remaining unexplored choices:
  (b) $304s = 83s + 221s$,
  (c) $557s = 195s + 362s$,
  (d) $273s = 153s + 120s$.
- From (f), we have two possible choices to continue.

# Find the Shortest Path from Start to Goal



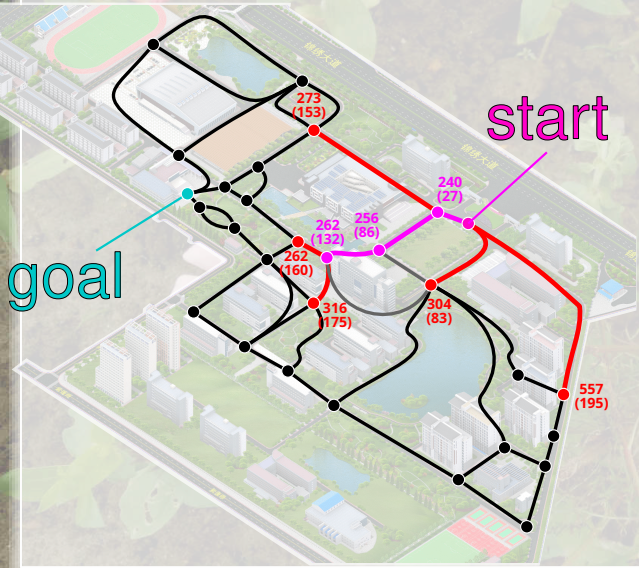

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- And we again compute the airline distance to the goal.
- (f) it is...
- We have 3 remaining unexplored choices:
  (b) $304s = 83s + 221s$,
  (c) $557s = 195s + 362s$,
  (d) $273s = 153s + 120s$.
- From (f), we have two possible choices to continue.

# Find the Shortest Path from Start to Goal



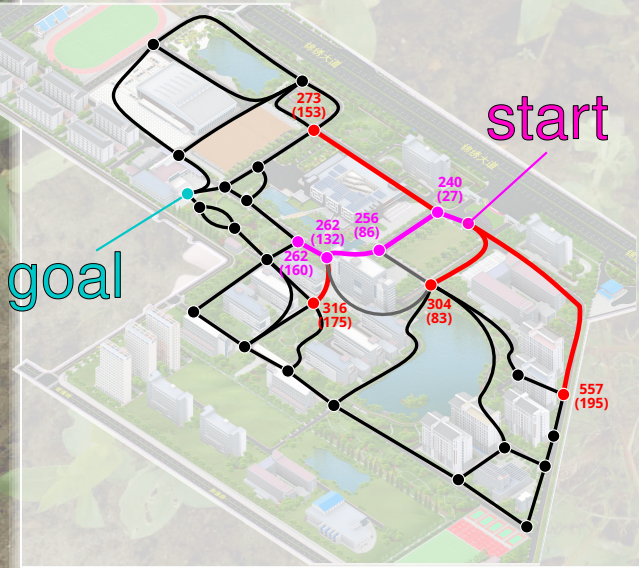

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- And we again compute the airline distance to the goal.
- (f) it is...
- From (f), we have two possible choices to continue.
- We now have 4 unexplored choices:
  (b) $304s = 83s + 221s$,
  (c) $557s = 195s + 362s$,
  (d) $273s = 153s + 120s$,
  (g) $262s = 160s + 102s$.

# Find the Shortest Path from Start to Goal



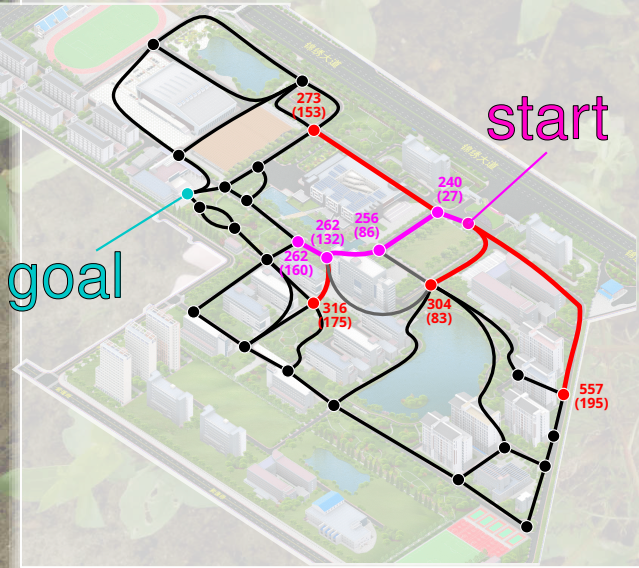

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- And we again compute the airline distance to the goal.
- (f) it is...
- From (f), we have two possible choices to continue.
- We now have 5 unexplored choices:
  (b) $304s = 83s + 221s$,
  (c) $557s = 195s + 362s$,
  (d) $273s = 153s + 120s$,
  (g) $262s = 160s + 102s$,
  (h) $316s = 175s + 141s$.

# Find the Shortest Path from Start to Goal



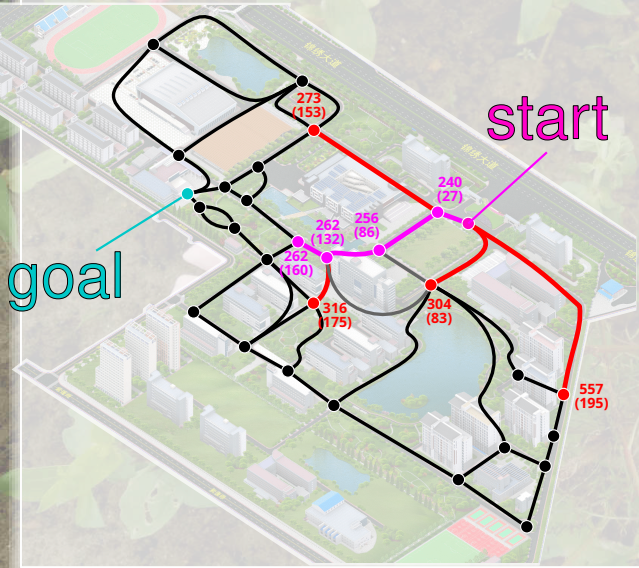

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- And we again compute the airline distance to the goal.
- From (f), we have two possible choices to continue.
- We now have 5 unexplored choices:
  (b) $304s = 83s + 221s$,
  (c) $557s = 195s + 362s$,
  (d) $273s = 153s + 120s$,
  (g) $262s = 160s + 102s$,
  (h) $316s = 175s + 141s$.
- (g) it is...

# Find the Shortest Path from Start to Goal



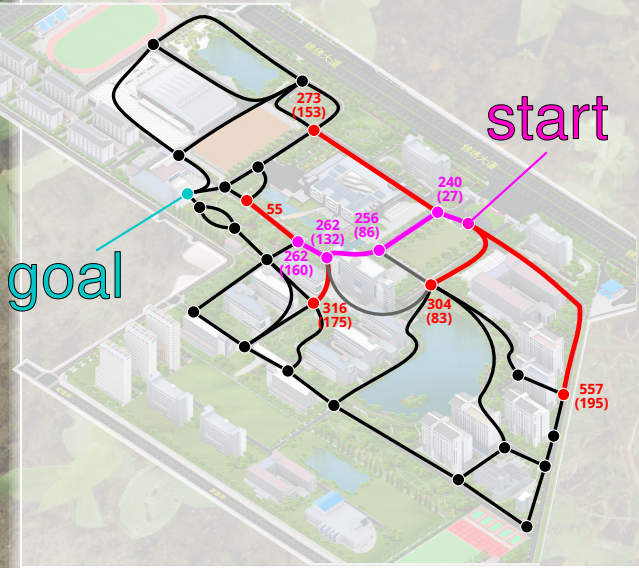

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- (g) it is...
- We now have 4 unexplored choices:
  - (b) $304s = 83s + 221s$,
  - (c) $557s = 195s + 362s$,
  - (d) $273s = 153s + 120s$,
  - (h) $316s = 175s + 141s$.

# Find the Shortest Path from Start to Goal



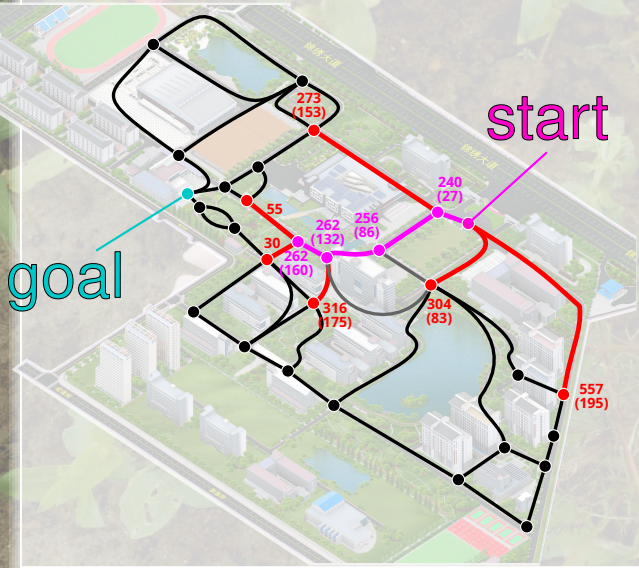

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- (g) it is...
- We now have 4 unexplored choices:
  - (b) $304s = 83s + 221s$,
  - (c) $557s = 195s + 362s$,
  - (d) $273s = 153s + 120s$,
  - (h) $316s = 175s + 141s$.
- And we get two new choices.

# Find the Shortest Path from Start to Goal



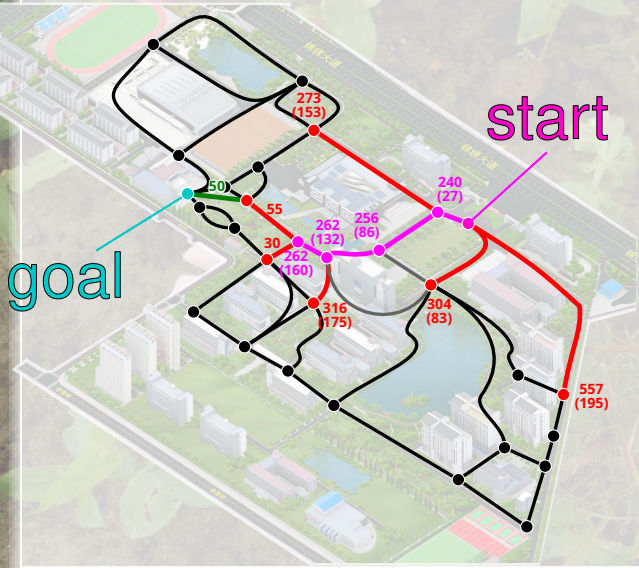

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- (g) it is...
- We now have 4 unexplored choices:
  - (b) $304s = 83s + 221s$,
  - (c) $557s = 195s + 362s$,
  - (d) $273s = 153s + 120s$,
  - (h) $316s = 175s + 141s$.
- And we get two new choices.

# Find the Shortest Path from Start to Goal



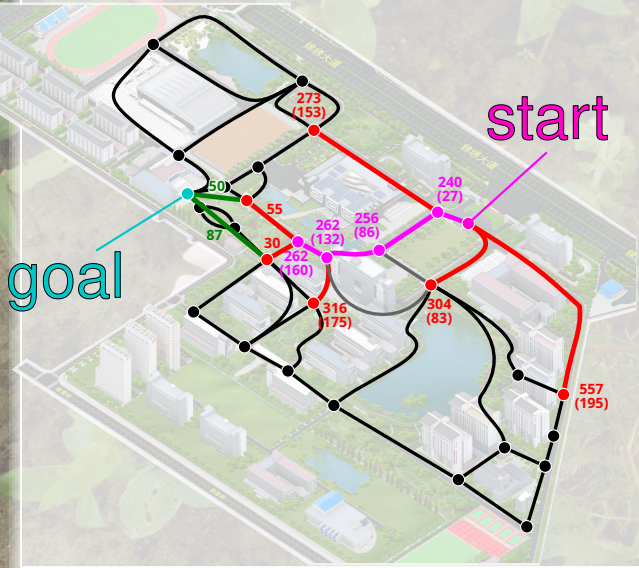

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- (g) it is...
- We now have 4 unexplored choices:
  - (b) $304s = 83s + 221s$,
  - (c) $557s = 195s + 362s$,
  - (d) $273s = 153s + 120s$,
  - (h) $316s = 175s + 141s$.
- And we get two new choices.

# Find the Shortest Path from Start to Goal



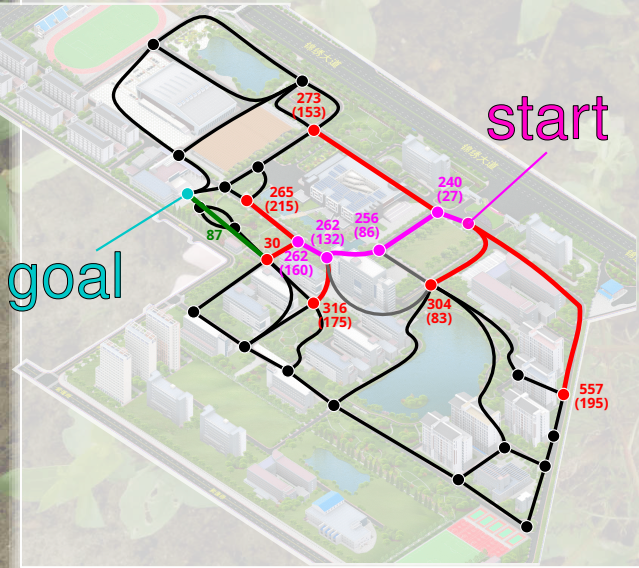

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- (g) it is...
- We now have 4 unexplored choices:
  - (b) $304s = 83s + 221s$,
  - (c) $557s = 195s + 362s$,
  - (d) $273s = 153s + 120s$,
  - (h) $316s = 175s + 141s$.
- And we get two new choices.

# Find the Shortest Path from Start to Goal



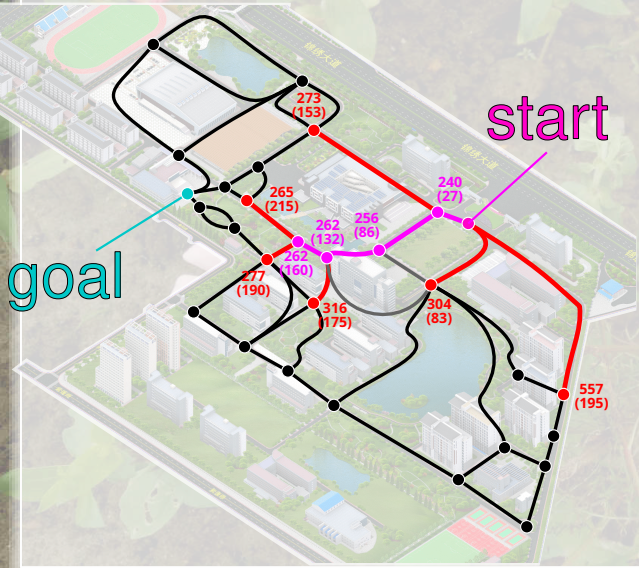

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- (g) it is...
- We now have 4 unexplored choices:
  - (b) $304s = 83s + 221s$,
  - (c) $557s = 195s + 362s$,
  - (d) $273s = 153s + 120s$,
  - (h) $316s = 175s + 141s$.
- And we get two new choices.

# Find the Shortest Path from Start to Goal



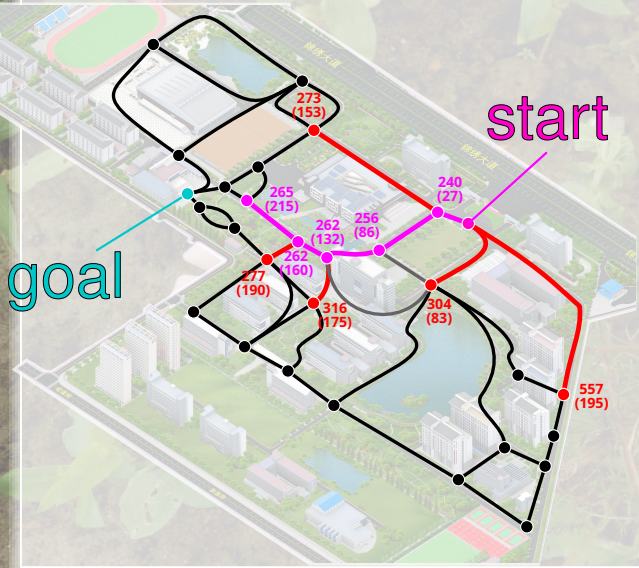

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- And we get two new choices.
- We now have 5 unexplored choices:
  - (b) $304s = 83s + 221s$,
  - (c) $557s = 195s + 362s$,
  - (d) $273s = 153s + 120s$,
  - (h) $316s = 175s + 141s$,
  - (i) $265s = 215s + 50s$.

# Find the Shortest Path from Start to Goal



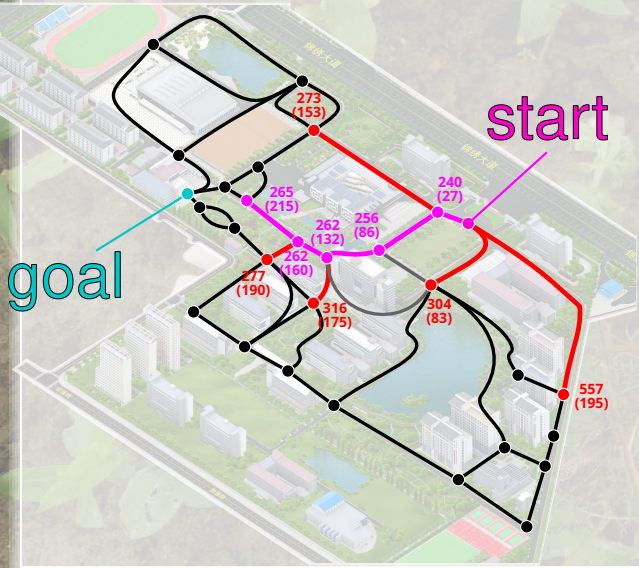

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- And we get two new choices.
- We now have 6 unexplored choices:
  - (b) $304s = 83s + 221s$,
  - (c) $557s = 195s + 362s$,
  - (d) $273s = 153s + 120s$,
  - (h) $316s = 175s + 141s$,
  - (i) $265s = 215s + 50s$,
  - (j) $277s = 190s + 87s$.

# Find the Shortest Path from Start to Goal



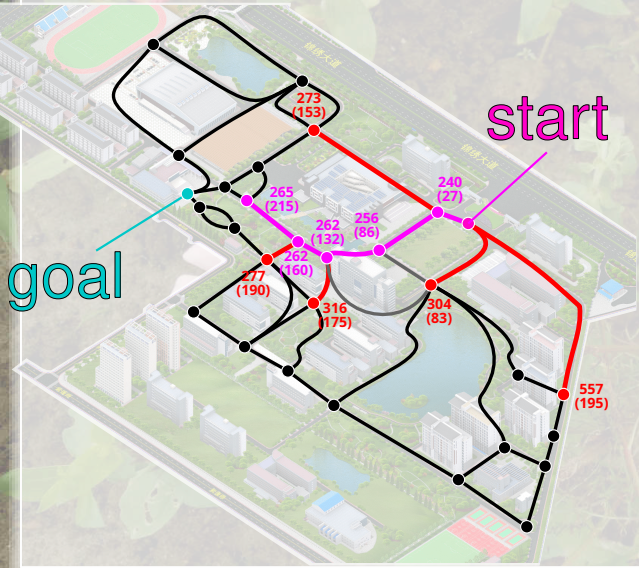

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- And we get two new choices.
- We now have 6 unexplored choices:
  - (b) $304s = 83s + 221s$,
  - (c) $557s = 195s + 362s$,
  - (d) $273s = 153s + 120s$,
  - (h) $316s = 175s + 141s$,
  - (i) $265s = 215s + 50s$,
  - (j) $277s = 190s + 87s$.
- (i) it is. . .

# Find the Shortest Path from Start to Goal



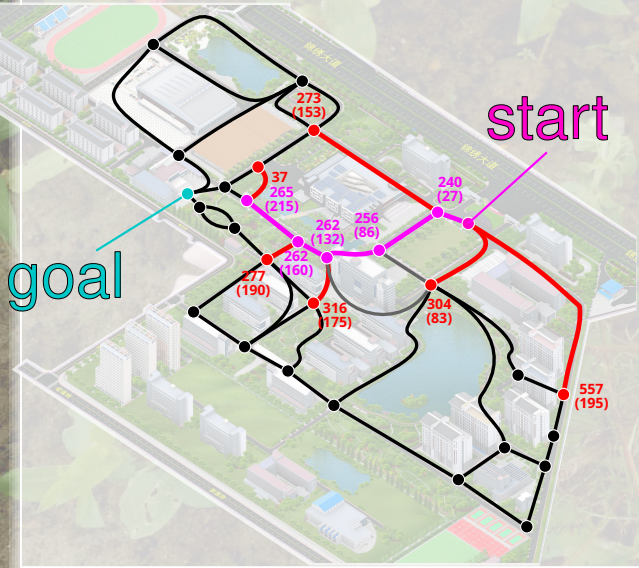

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We now have 5 unexplored choices:
  (b) $304s = 83s + 221s$,
  (c) $557s = 195s + 362s$,
  (d) $273s = 153s + 120s$,
  (h) $316s = 175s + 141s$,
  (j) $277s = 190s + 87s$.

# Find the Shortest Path from Start to Goal



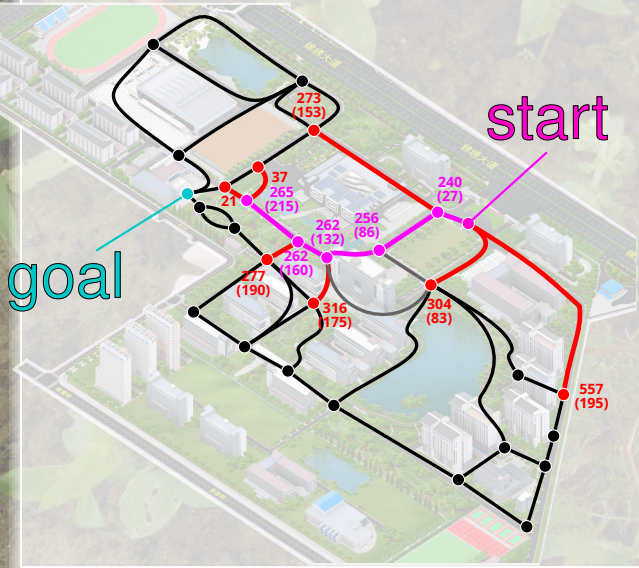

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We now have 5 unexplored choices:
  (b) $304s = 83s + 221s$,
  (c) $557s = 195s + 362s$,
  (d) $273s = 153s + 120s$,
  (h) $316s = 175s + 141s$,
  (j) $277s = 190s + 87s$.
- From (i), we again got two new choices.

# Find the Shortest Path from Start to Goal



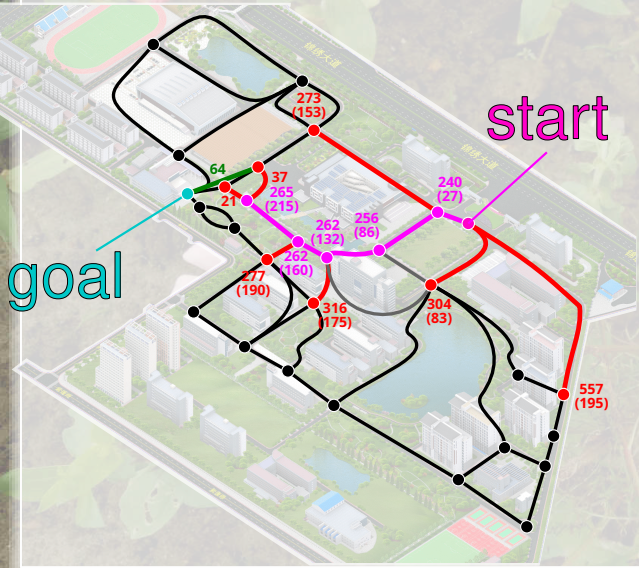

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We now have 5 unexplored choices:
  (b) $304s = 83s + 221s$,
  (c) $557s = 195s + 362s$,
  (d) $273s = 153s + 120s$,
  (h) $316s = 175s + 141s$,
  (j) $277s = 190s + 87s$.
- From (i), we again got two new choices.

# Find the Shortest Path from Start to Goal



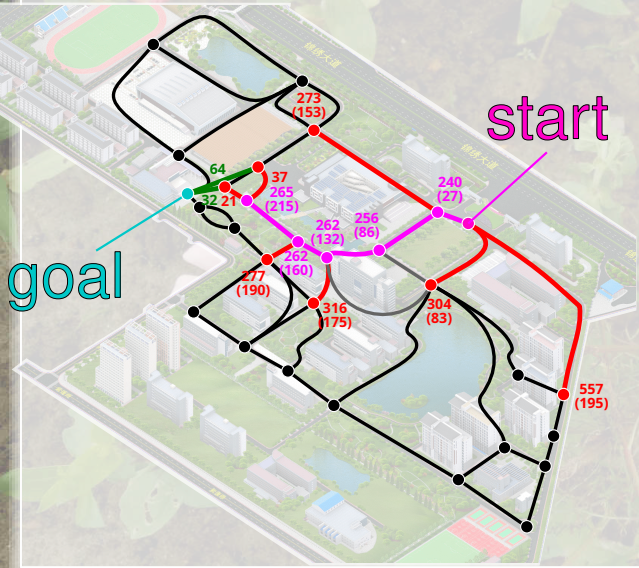

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We now have 5 unexplored choices:
  (b) $304s = 83s + 221s$,
  (c) $557s = 195s + 362s$,
  (d) $273s = 153s + 120s$,
  (h) $316s = 175s + 141s$,
  (j) $277s = 190s + 87s$.
- From (i), we again got two new choices.

# Find the Shortest Path from Start to Goal



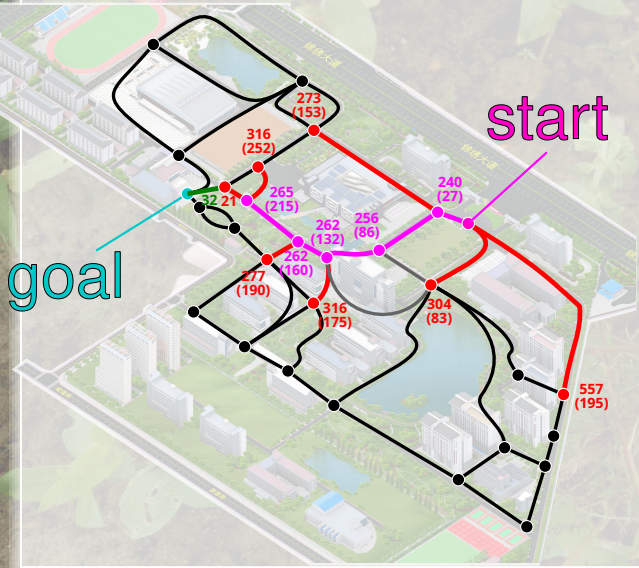

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We now have 5 unexplored choices:
  (b) $304s = 83s + 221s$,
  (c) $557s = 195s + 362s$,
  (d) $273s = 153s + 120s$,
  (h) $316s = 175s + 141s$,
  (j) $277s = 190s + 87s$.
- From (i), we again got two new choices.

# Find the Shortest Path from Start to Goal



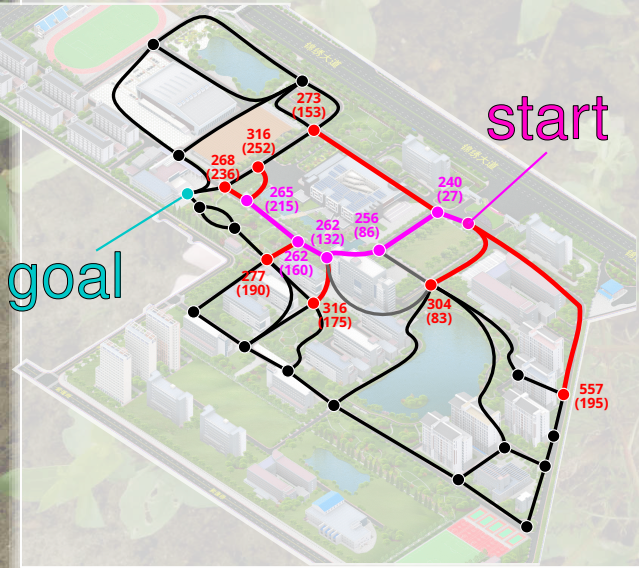

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We now have 5 unexplored choices:
  (b) $304s = 83s + 221s$,
  (c) $557s = 195s + 362s$,
  (d) $273s = 153s + 120s$,
  (h) $316s = 175s + 141s$,
  (j) $277s = 190s + 87s$.
- From (i), we again got two new choices.

# Find the Shortest Path from Start to Goal



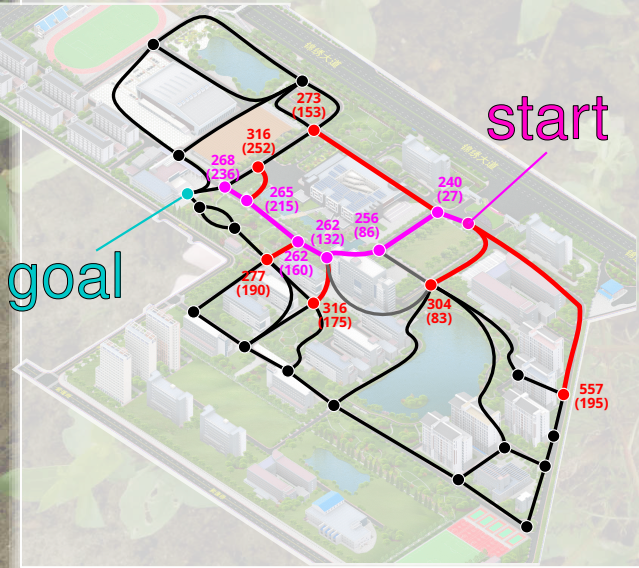

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- From (i), we again got two new choices.
- We now have 6 unexplored choices:
  - (b) $304s = 83s + 221s$,
  - (c) $557s = 195s + 362s$,
  - (d) $273s = 153s + 120s$,
  - (h) $316s = 175s + 141s$,
  - (j) $277s = 190s + 87s$,
  - (k) $316s = 252s + 64s$.

# Find the Shortest Path from Start to Goal



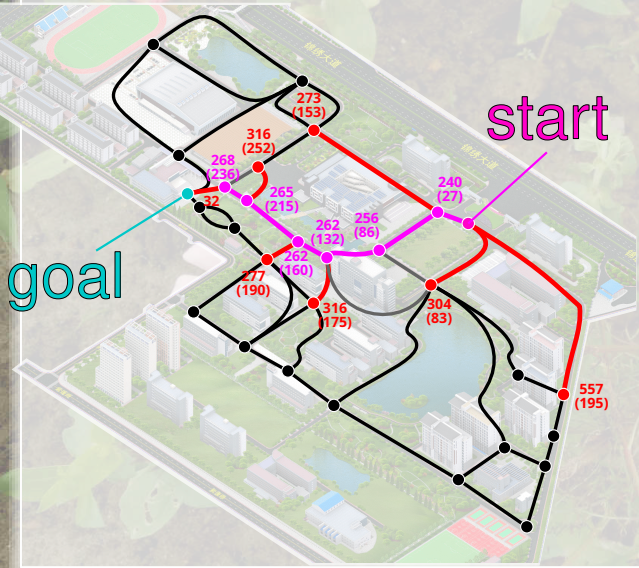

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- From (i), we again got two new choices.
- We now have 7 unexplored choices:
  - (b) $304s = 83s + 221s$,
  - (c) $557s = 195s + 362s$,
  - (d) $273s = 153s + 120s$,
  - (h) $316s = 175s + 141s$,
  - (j) $277s = 190s + 87s$,
  - (k) $316s = 252s + 64s$,
  - (l) $268s = 236s + 32s$.

# Find the Shortest Path from Start to Goal



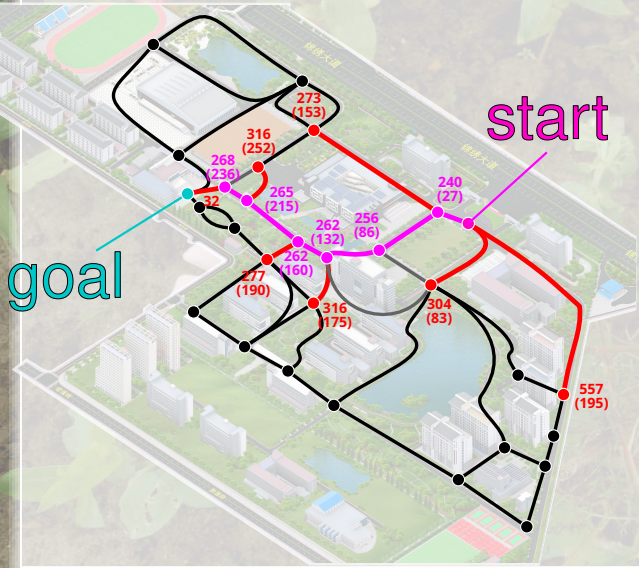

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- From (i), we again got two new choices.
- We now have 7 unexplored choices:
  - (b) $304s = 83s + 221s$,
  - (c) $557s = 195s + 362s$,
  - (d) $273s = 153s + 120s$,
  - (h) $316s = 175s + 141s$,
  - (j) $277s = 190s + 87s$,
  - (k) $316s = 252s + 64s$,
  - (l) $268s = 236s + 32s$.
- (l) it is...

# Find the Shortest Path from Start to Goal



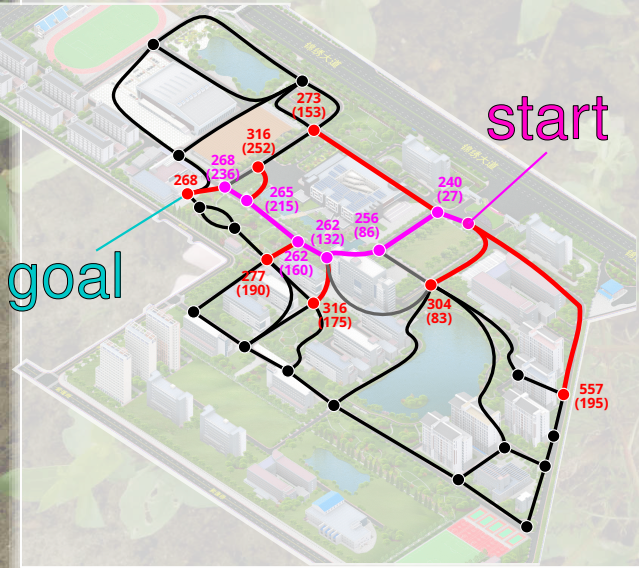

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We now have 6 unexplored choices:
  (b) $304s = 83s + 221s$,
  (c) $557s = 195s + 362s$,
  (d) $273s = 153s + 120s$,
  (h) $316s = 175s + 141s$,
  (j) $277s = 190s + 87s$,
  (k) $316s = 252s + 64s$.
- (l) it is...
- There is only one choice to continue: go to the goal!

# Find the Shortest Path from Start to Goal



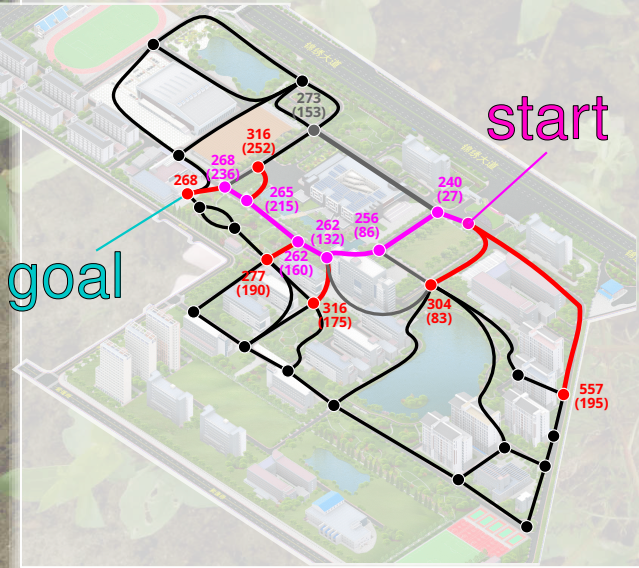

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We now have 6 unexplored choices:
  (b) $304s = 83s + 221s$,
  (c) $557s = 195s + 362s$,
  (d) $273s = 153s + 120s$,
  (h) $316s = 175s + 141s$,
  (j) $277s = 190s + 87s$,
  (k) $316s = 252s + 64s$.
- (l) it is...
- There is only one choice to continue: go to the goal!
- We found a first complete path from start to goal.

# Find the Shortest Path from Start to Goal



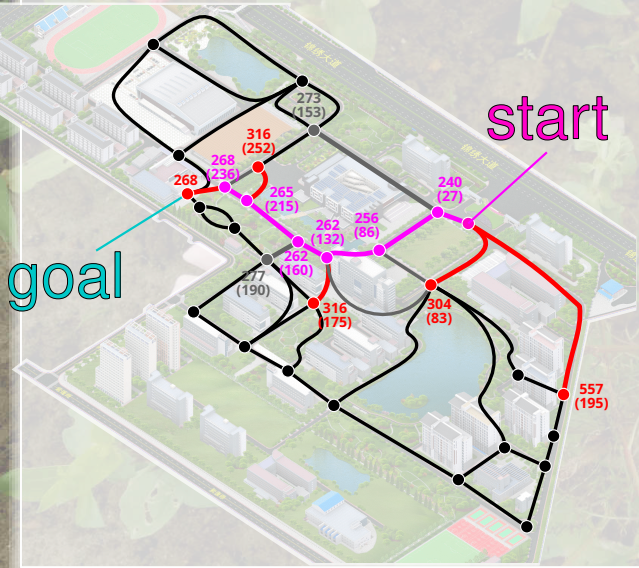

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We now have 6 unexplored choices:
  (b) $304s = 83s + 221s$,
  (c) $557s = 195s + 362s$,
  (d) $273s = 153s + 120s$,
  (h) $316s = 175s + 141s$,
  (j) $277s = 190s + 87s$,
  (k) $316s = 252s + 64s$.
- There is only one choice to continue: go to the goal!
- We found a first complete path from start to goal.
- It has the total length $268s$.

# Find the Shortest Path from Start to Goal



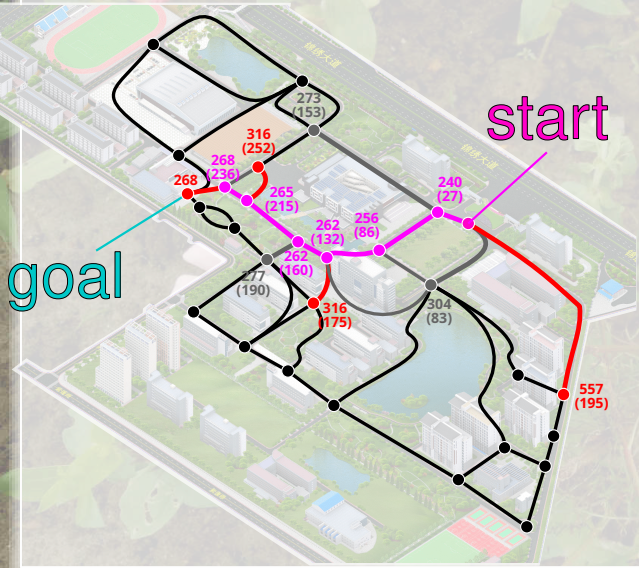

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We now have 56 unexplored choices:
  - (b) $304s = 83s + 221s$,
  - (c) $557s = 195s + 362s$,
  - (d) $273s = 153s + 120s$,
  - (h) $316s = 175s + 141s$,
  - (j) $277s = 190s + 87s$,
  - (k) $316s = 252s + 64s$.
- (d) definitely will need more than $268s$.

# Find the Shortest Path from Start to Goal



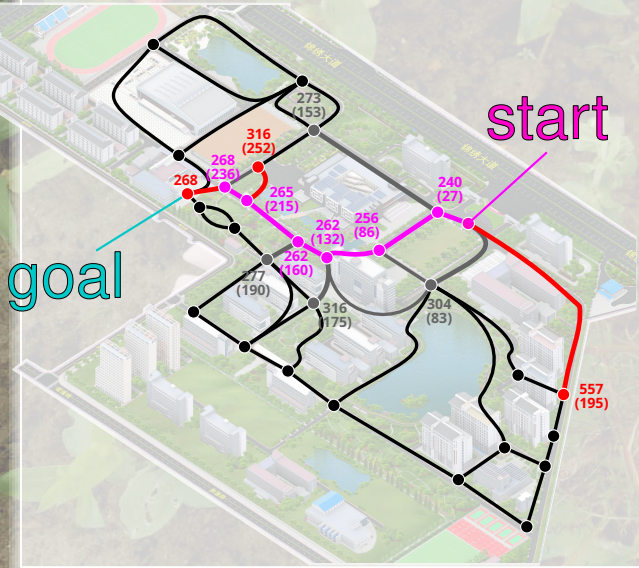

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We now have 4 unexplored choices:
  - (b) $304s = 83s + 221s$,
  - (c) $557s = 195s + 362s$,
  - (d) $273s = 153s + 120s$,
  - (h) $316s = 175s + 141s$,
  - (j) $277s = 190s + 87s$,
  - (k) $316s = 252s + 64s$.
- (j) definitely will need more than $268s$.

# Find the Shortest Path from Start to Goal



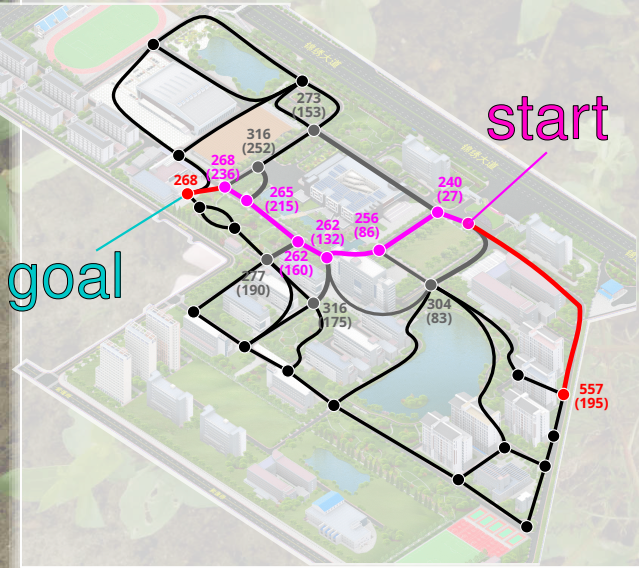

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We now have 3 unexplored choices:
  - (b) $304s = 83s + 221s$,
  - (c) $557s = 195s + 362s$,
  - (d) $273s = 153s + 120s$,
  - (h) $316s = 175s + 141s$,
  - (j) $277s = 190s + 87s$,
  - (k) $316s = 252s + 64s$.
- (b) definitely will need more than $268s$.

# Find the Shortest Path from Start to Goal



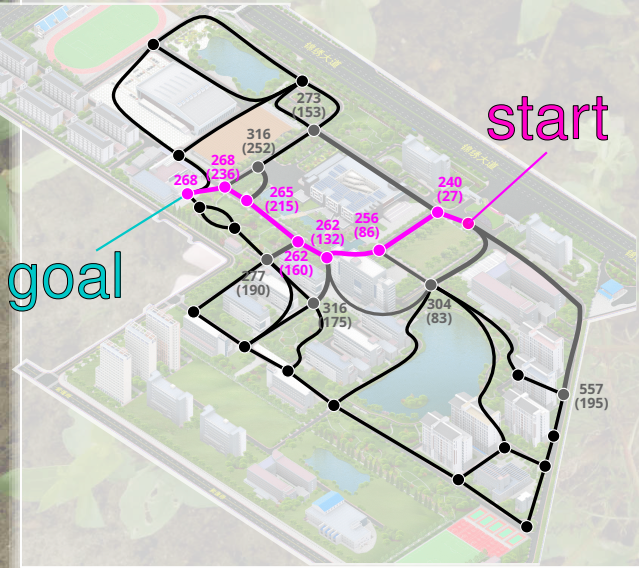

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We now have 2 unexplored choices:
  - (b) $304s = 83s + 221s$,
  - (c) $557s = 195s + 362s$,
  - (d) $273s = 153s + 120s$,
  - (h) $316s = 175s + 141s$,
  - (j) $277s = 190s + 87s$,
  - (k) $316s = 252s + 64s$.
- (h) definitely will need more than $268s$.

# Find the Shortest Path from Start to Goal



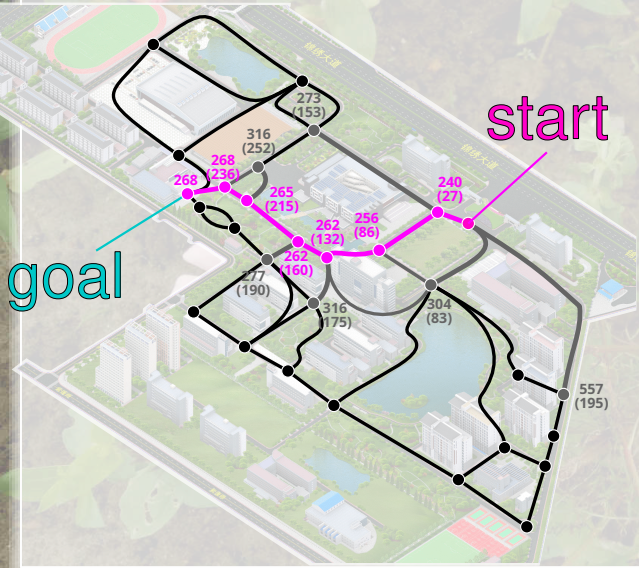

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We now have 1 unexplored choices:
  - (b) $304s = 83s + 221s$,
  - (c) $557s = 195s + 362s$,
  - (d) $273s = 153s + 120s$,
  - (h) $316s = 175s + 141s$,
  - (j) $277s = 190s + 87s$,
  - (k) $316s = 252s + 64s$.
- (k) definitely will need more than $268s$.

# Find the Shortest Path from Start to Goal



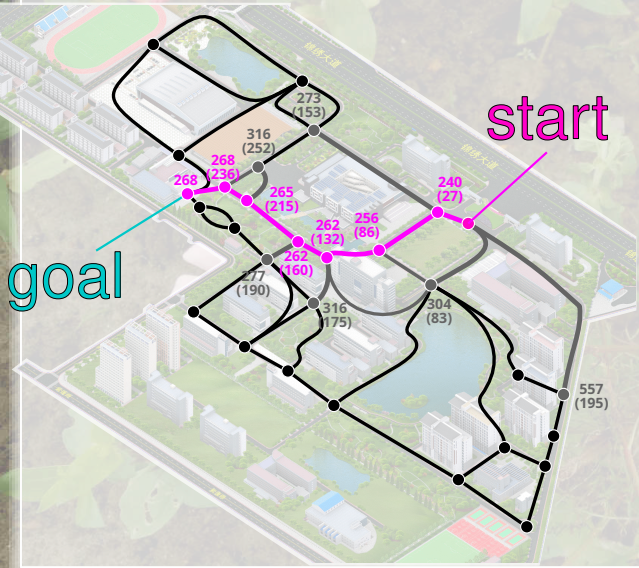

- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We now have 0 unexplored choices:
  - (b) $304s = 83s + 221s$,
  - (c) $557s = 195s + 362s$,
  - (d) $273s = 153s + 120s$,
  - (h) $316s = 175s + 141s$,
  - (j) $277s = 190s + 87s$,
  - (k) $316s = 252s + 64s$.
- (c) definitely will need more than $268s$.

# Find the Shortest Path from Start to Goal




- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We now have 0 unexplored choices:
  - (b) $304s = 83s + 221s$,
  - (c) $557s = 195s + 362s$,
  - (d) $273s = 153s + 120s$,
  - (h) $316s = 175s + 141s$,
  - (j) $277s = 190s + 87s$,
  - (k) $316s = 252s + 64s$.
- We found the shortest possible path and it takes $268s$.

# Find the Shortest Path from Start to Goal




- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We now have 0 unexplored choices:
  - (b) $304s = 83s + 221s$,
  - (c) $557s = 195s + 362s$,
  - (d) $273s = 153s + 120s$,
  - (h) $316s = 175s + 141s$,
  - (j) $277s = 190s + 87s$,
  - (k) $316s = 252s + 64s$.
- We found the shortest possible path and it takes $268s$.
- **Solved.**

# The A$^\star$ Algorithmfor Finding Shortest Paths



- We just applied the A$^\star$ Algorithm[99,169].

# The A$^\star$ Algorithmfor Finding Shortest Paths



- We just applied the A$^\star$ Algorithm[99,169].
- The A$^\star$ Algorithm iteratively constructs a solution.

# The A* Algorithmfor Finding Shortest Paths

- We just applied the A* Algorithm[99,169].
- The A* Algorithm iteratively constructs a solution.
- It decides next step to test based on a combination of heuristic and cost.

# The A* Algorithmfor Finding Shortest Paths



- We just applied the A* Algorithm[99,169].
- The A* Algorithm iteratively constructs a solution.
- It decides next step to test based on a combination of heuristic and cost.
- In the worst case, it needs to "look" at all intersections (nodes) and streets (edges) once.

# The A* Algorithmfor Finding Shortest Paths

- We just applied the A* Algorithm[99,169].
- The A* Algorithm iteratively constructs a solution.
- It decides next step to test based on a combination of heuristic and cost.
- In the worst case, it needs to "look" at all intersections (nodes) and streets (edges) once.
- Usually, it is quite efficient to find shortest paths on maps.

# The A* Algorithmfor Finding Shortest Paths



- We just applied the A* Algorithm[99,169].
- The A* Algorithm iteratively constructs a solution.
- It decides next step to test based on a combination of heuristic and cost.
- In the worst case, it needs to "look" at all intersections (nodes) and streets (edges) once.
- Usually, it is quite efficient to find shortest paths on maps.
- (If we look for shortest paths that do not visit any node twice, where the edge distances are not negative, where the number of choices per node is limited, and where the graph and path are not too big – then this algorithm is very efficient.)

Problems that algorithms cannot solve both efficiently *and* exactly



# Hard Problems (1)




- There exists a group of problems that are *hard*.

# Hard Problems (1)




- There exists a group of problems that are *hard*.
- Actually, most of the problems I mention at the beginning of this talk belong to this group of hard problems.

# Hard Problems (1)




- There exists a group of problems that are *hard*.
- Actually, most of the problems I mention at the beginning of this talk belong to this group of hard problems.
- A hard problem cannot be solved both exactly and efficiently.

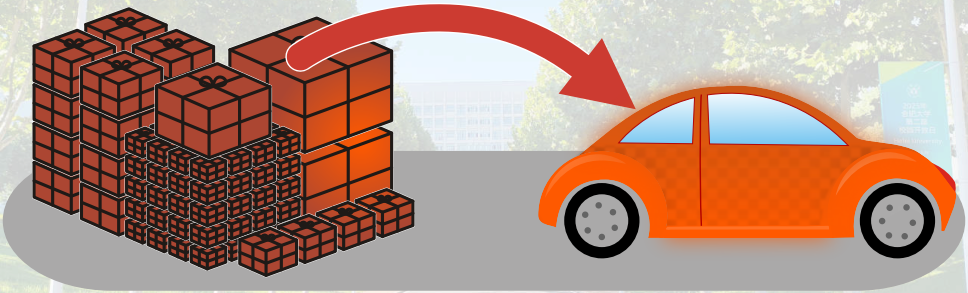
# Hard Problems (1)




- There exists a group of problems that are *hard*.
- Actually, most of the problems I mention at the beginning of this talk belong to this group of hard problems.
- A hard problem cannot be solved both exactly and efficiently.
- Let's look at two quick examples.

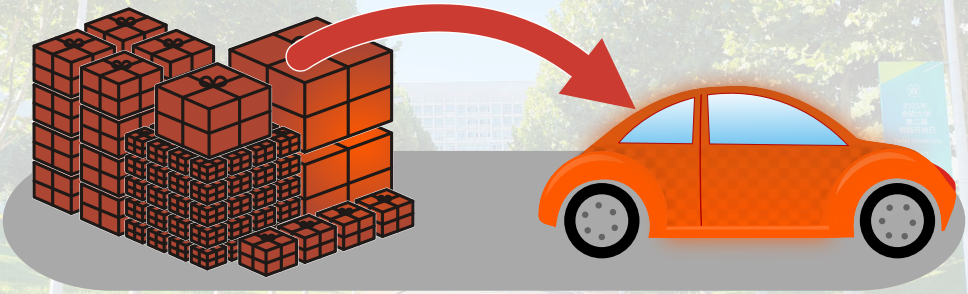
# Bin Packing Problem

- My car can carry $T$ kg of weight.



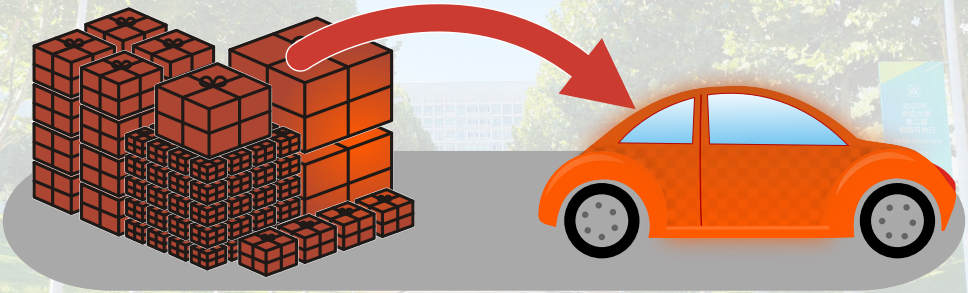
# Bin Packing Problem

- My car can carry $T$ kg of weight.
- I have $n$ objects, each with weight $w_i$ with $i$ in 1 to $n$.



# Bin Packing Problem

- My car can carry $T$ kg of weight.
- I have $n$ objects, each with weight $w_i$ with $i$ in 1 to $n$.
- How can I pack my car so that I can carry them from $A$ to $B$ with the fewest possible hauls?[233,234]



# Traveling Salesperson Problem

- Find the shortest path that visits $n$ locations and returns back to its origin.[4,130]



## Hard Problems (2)

- These are just two examples from a huge family of problems called $\mathcal{NP}$-hard.

## Hard Problems (2)



- These are just two examples from a huge family of problems called $\mathcal{NP}$-hard.
- What does that mean?

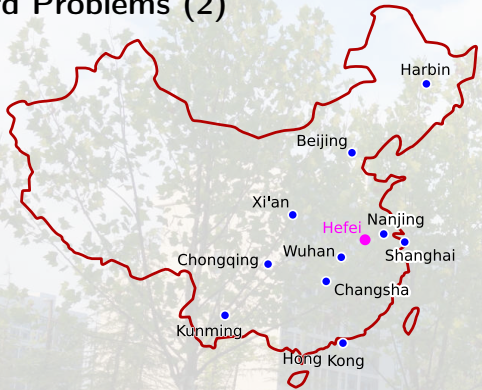
## Hard Problems (2)



- These are just two examples from a huge family of problems called $\mathcal{NP}$-hard.
- What does that mean?

If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$-hard problem, then its worst-case runtime will grow exponential with the input size $n$.

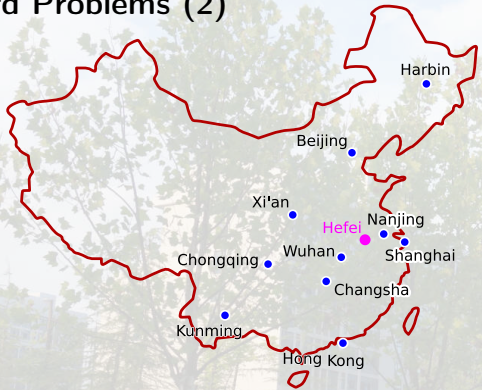
# Hard Problems (2)






If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$-hard problem, then its worst-case runtime will grow exponential with the input size $n$.

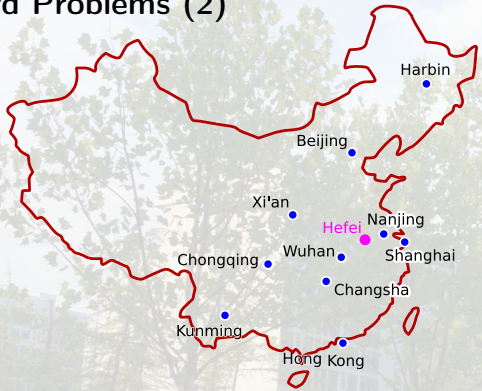
# Hard Problems (2)






If an algorithm guarantees to always find the **best-possible** solution for an $\mathcal{NP}$-hard problem, then its worst-case runtime will grow exponential with the input size $n$.

# Hard Problems (2)



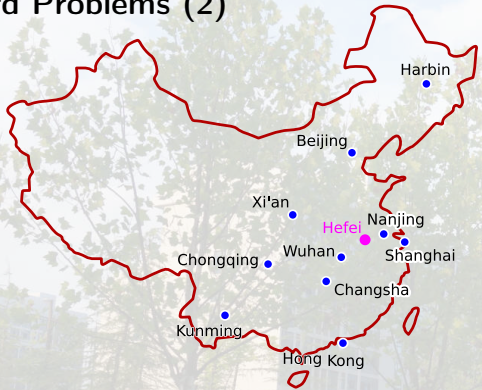

shortest round-trip tour



packing with fewest hauls

If an algorithm guarantees to always find the **best-possible** solution for an $\mathcal{NP}$-hard problem, then its worst-case runtime will grow exponential with the input size $n$.

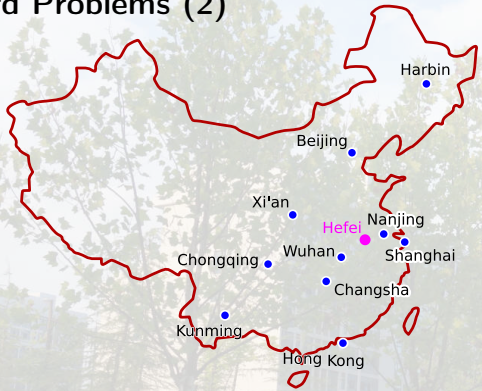
# Hard Problems (2)






If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$-hard problem, then its worst-case runtime will grow exponential with the **input size** $n$.

# Hard Problems (2)



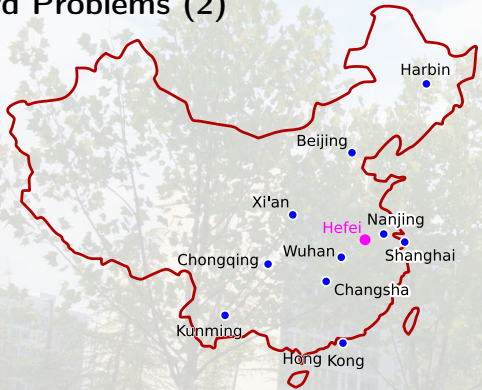

number $n$ of cities to visit



number $n$ of items to pack

If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$-hard problem, then its worst-case runtime will grow exponential with the **input size** $n$.

# Hard Problems (2)

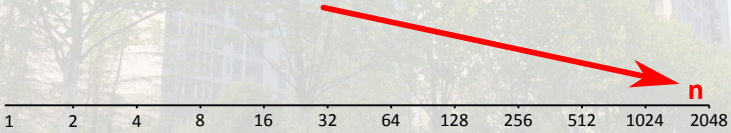





If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$-hard problem, then its worst-case runtime will **grow exponential** with the input size $n$.

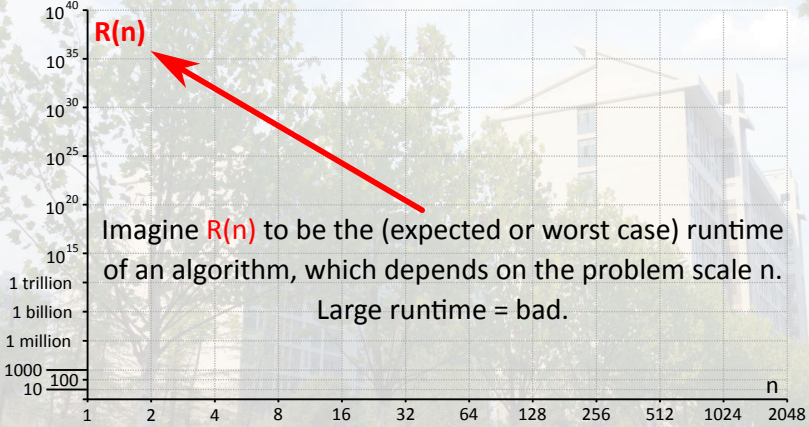
# Hard Problems (2)

The problem scale n usually corresponding to the number of decision variables.



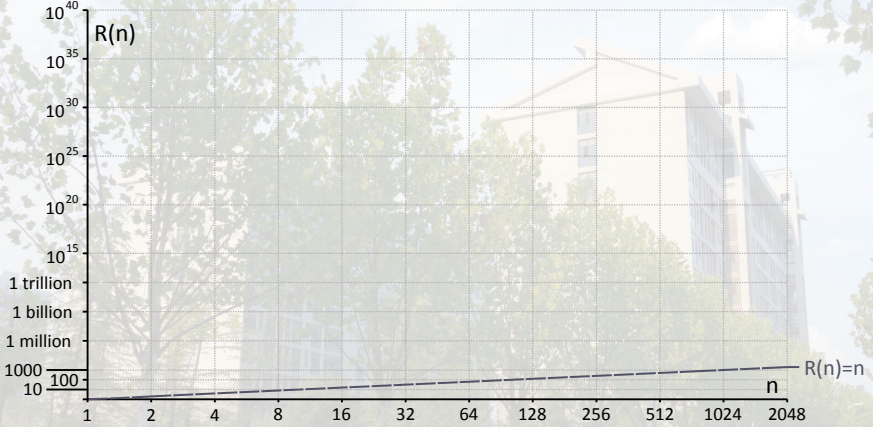

If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$-hard problem, then its worst-case runtime will **grow exponential** with the input size $n$.

# Hard Problems (2)



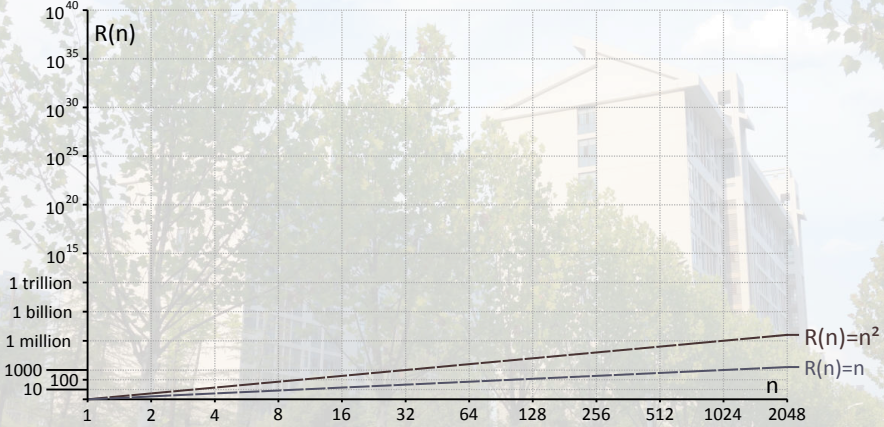

If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$-hard problem, then its worst-case runtime will **grow exponential** with the input size $n$.

# Hard Problems (2)



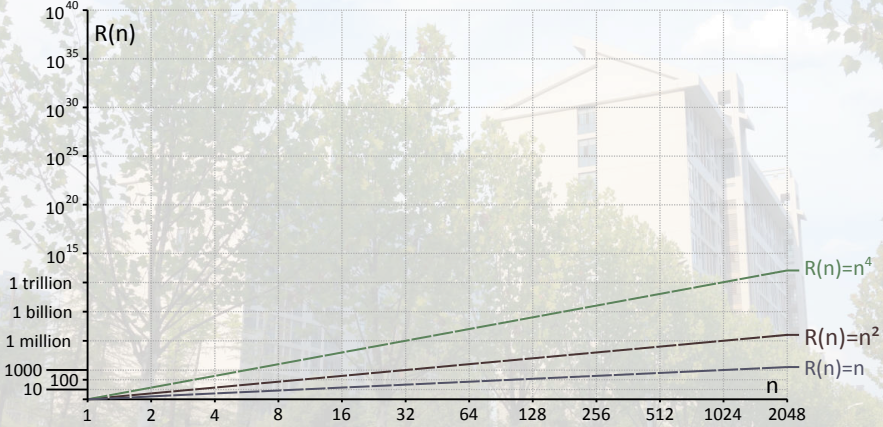

If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$-hard problem, then its worst-case runtime will **grow exponential** with the input size $n$.

# Hard Problems (2)



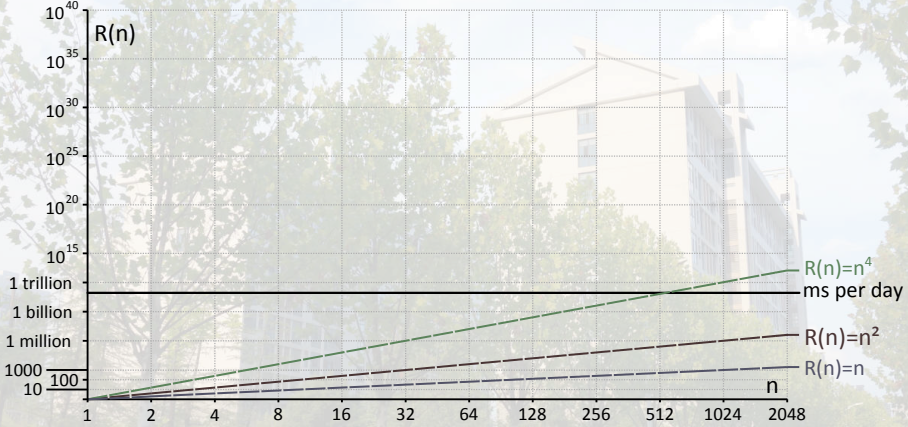

If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$-hard problem, then its worst-case runtime will **grow exponential** with the input size $n$.

# Hard Problems (2)



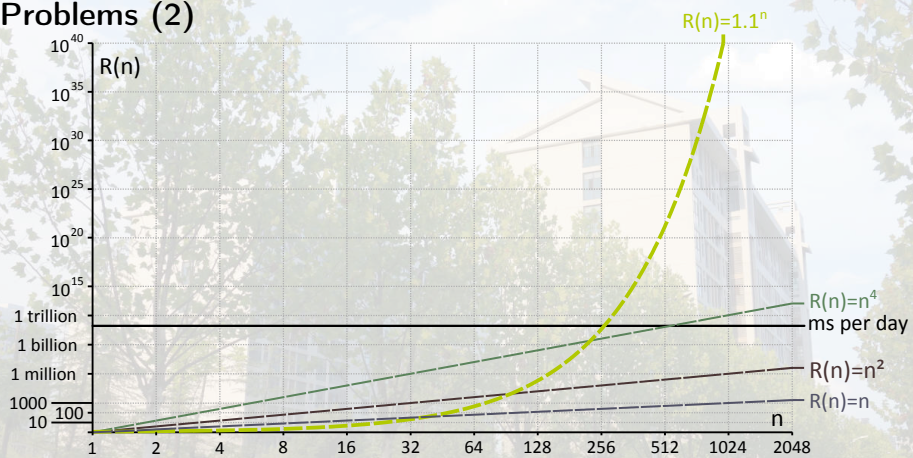

If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$-hard problem, then its worst-case runtime will **grow exponential** with the input size $n$.

# Hard Problems (2)



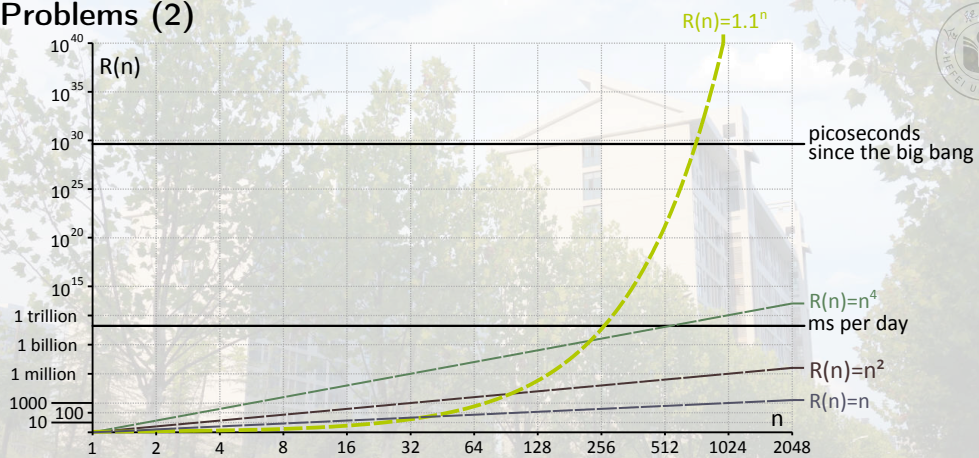

If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$-hard problem, then its worst-case runtime will **grow exponential** with the input size $n$.

# Hard Problems (2)



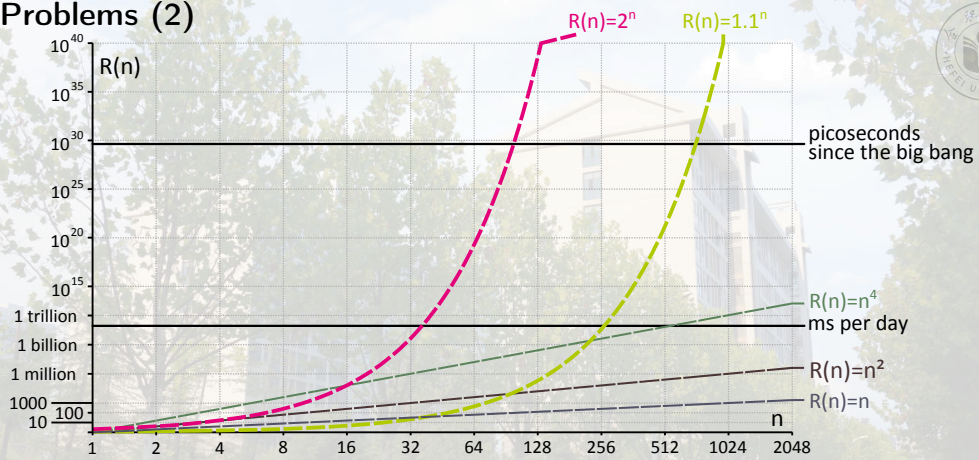

If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$-hard problem, then its worst-case runtime will **grow exponential** with the input size $n$.

# Hard Problems (2)



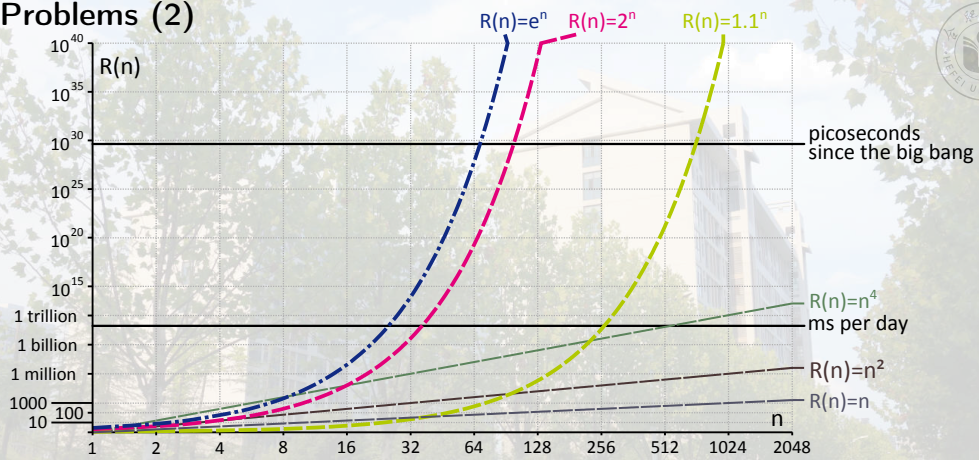

If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$-hard problem, then its worst-case runtime will **grow exponential** with the input size $n$.

# Hard Problems (2)



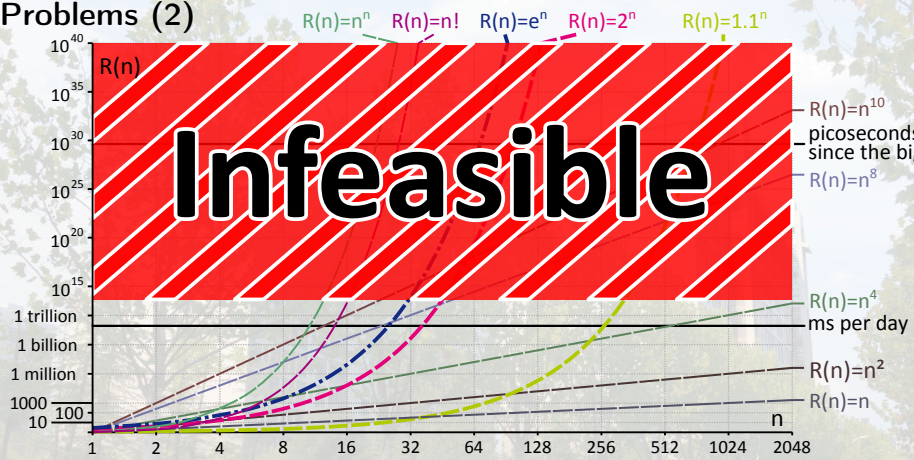

If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$-hard problem, then its worst-case runtime will **grow exponential** with the input size $n$.

# Hard Problems (2)




If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$-hard problem, then its worst-case runtime will **grow exponential** with the input size $n$.

# Hard Problems (2)




If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$-hard problem, then its worst-case runtime will **grow exponential** with the input size $n$.

## Hard Problems (2)



- These are just two examples from a huge family of problems called $\mathcal{NP}$-hard.
- What does that mean?

If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$-hard problem, then its worst-case runtime will grow exponential with the input size $n$.

## Hard Problems (2)



- These are just two examples from a huge family of problems called $\mathcal{NP}$-hard.
- What does that mean?

**If we want to get the optimal solution, it will take too long.**

If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$-hard problem, then its worst-case runtime will grow exponential with the input size $n$.

## Hard Problems (2)

- These are just two examples from a huge family of problems called $\mathcal{NP}$-hard.
- What does that mean?

**If we want to get the optimal solution,**
**it will sometimes/often/always take too long.**

If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$-hard problem, then its **worst-case** runtime will grow exponential with the input size $n$.

## Traveling Salesperson Problem



- In an Traveling Salesperson Problem (TSP), we want to find the shortest path through $n$ locations and back to the start[4,97,130,134,212,222].

# Traveling Salesperson Problem

- In an Traveling Salesperson Problem (TSP), we want to find the shortest path through $n$ locations and back to the start[4,97,130,134,212,222].



# Traveling Salesperson Problem

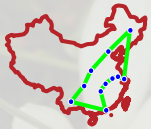
- In an Traveling Salesperson Problem (TSP), we want to find the shortest path through $n$ locations and back to the start[4,97,130,134,212,222].



# Traveling Salesperson Problem

- In an Traveling Salesperson Problem (TSP), we want to find the shortest path through $n$ locations and back to the start[4,97,130,134,212,222].

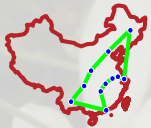
getting the optimal solution
for a TSP



# Traveling Salesperson Problem

- In an Traveling Salesperson Problem (TSP), we want to find the shortest path through $n$ locations and back to the start[4,97,130,134,212,222].

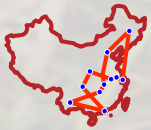
getting the optimal solution
for a TSP may take too long



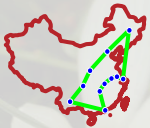
consumed runtime: very much / too (?) long

# Traveling Salesperson Problem

- In an Traveling Salesperson Problem (TSP), we want to find the shortest path through $n$ locations and back to the start[4,97,130,134,212,222].



getting the optimal solution
for a TSP may take too long



very little / fast — consumed runtime — very much / too (?) long

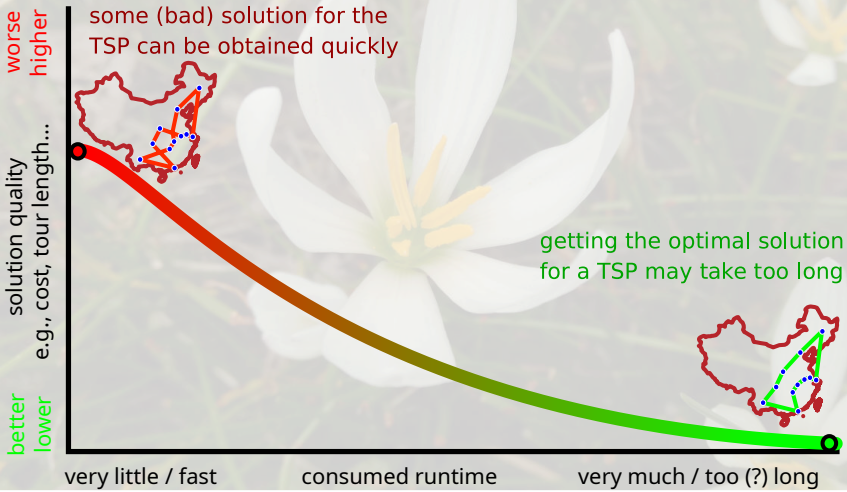
# Traveling Salesperson Problem

- In an Traveling Salesperson Problem (TSP), we want to find the shortest path through $n$ locations and back to the start[4,97,130,134,212,222].



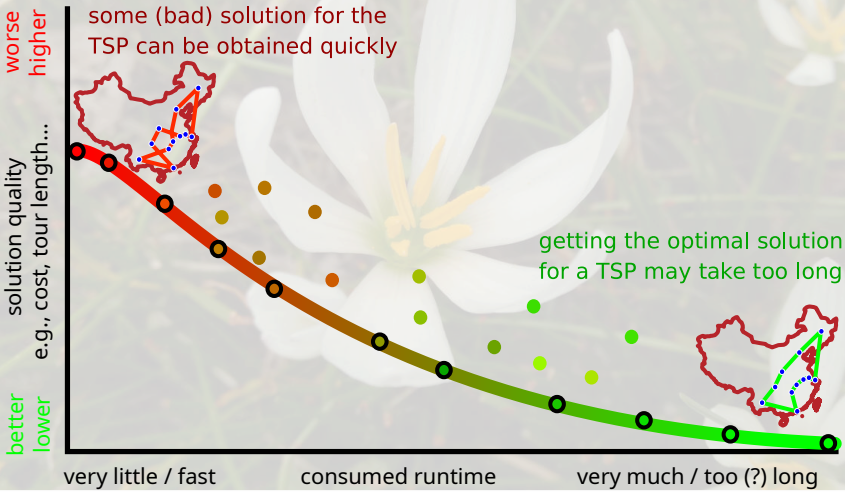
# Traveling Salesperson Problem

- In an Traveling Salesperson Problem (TSP), we want to find the shortest path through $n$ locations and back to the start[4,97,130,134,212,222].



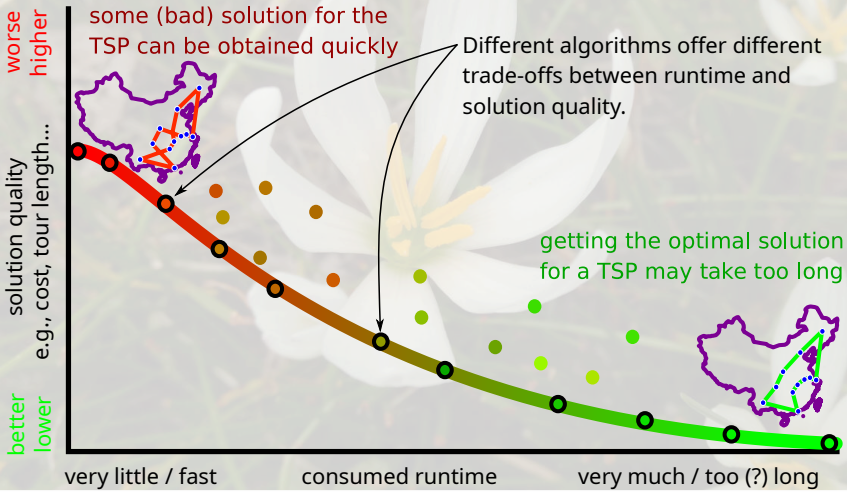
# Traveling Salesperson Problem

- In an Traveling Salesperson Problem (TSP), we want to find the shortest path through $n$ locations and back to the start[4,97,130,134,212,222].



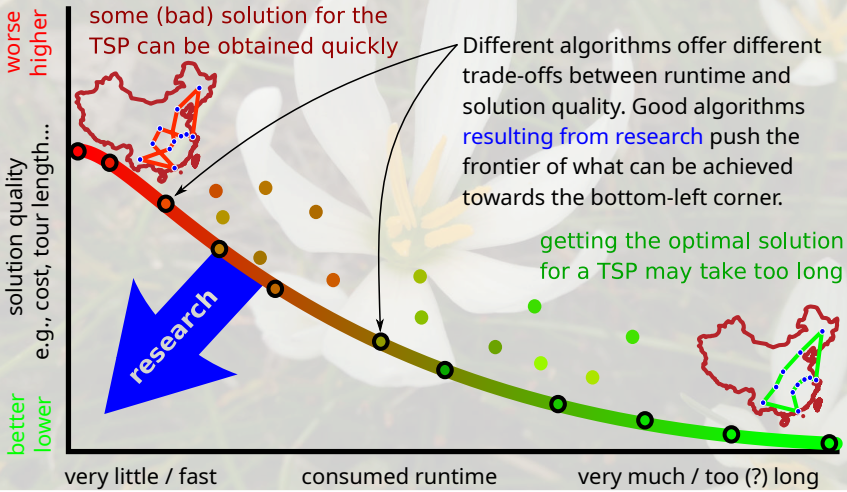
# Traveling Salesperson Problem

- In an Traveling Salesperson Problem (TSP), we want to find the shortest path through $n$ locations and back to the start[4,97,130,134,212,222].



# Traveling Salesperson Problem

- In an Traveling Salesperson Problem (TSP), we want to find the shortest path through $n$ locations and back to the start[4,97,130,134,212,222].



# Randomly Guessing Solutions



# *Almost* Solving Hard Problems




- For many problems, it is very easy to "guess" a solution.

# *Almost* Solving Hard Problems




- For many problems, it is very easy to "guess" a solution.
- This solution will (very likely) not be optimal.

# *Almost* Solving Hard Problems




- For many problems, it is very easy to "guess" a solution.
- This solution will (very likely) not be optimal.
- It will very likely be very bad.

# *Almost* Solving Hard Problems




- For many problems, it is very easy to "guess" a solution.
- This solution will (very likely) not be optimal.
- It will very likely be very bad.
- But it will be better than nothing.

## Random Sampling for the TSP

- Back to the Traveling Salesperson Problem (TSP).

## Random Sampling for the TSP



- Back to the Traveling Salesperson Problem (TSP).
- How do we "guess" a solution?

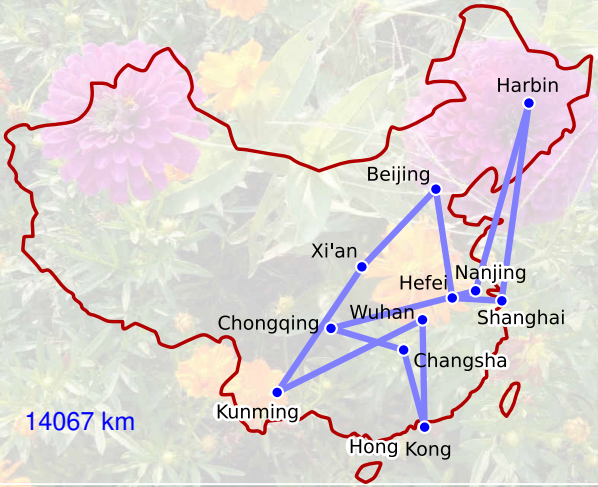
# Random Sampling for the TSP




- Back to the Traveling Salesperson Problem (TSP).
- How do we "guess" a solution?
- We could write down the cities in a random order.

# Random Sampling for the TSP

- We could write down the cities in a random order.



# Random Sampling for the TSP

- We could write down the cities in a random order.



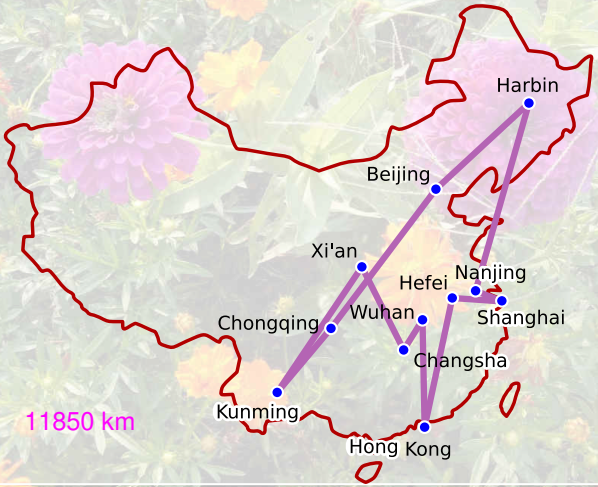
# Random Sampling for the TSP

- We could write down the cities in a random order.



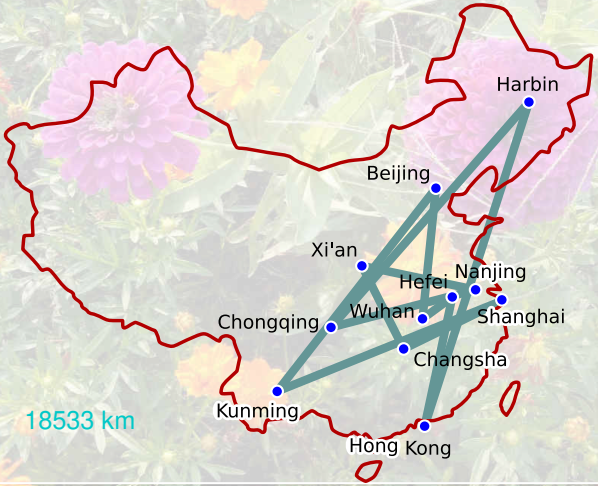
# Random Sampling for the TSP

- We could write down the cities in a random order.



# Random Sampling for the TSP

- We could write down the cities in a random order.



# Random Sampling for the TSP

- We could write down the cities in a random order.
- If we do this, we will get a bad result.



# Random Sampling for the TSP

- We could write down the cities in a random order.
- If we do this, we will get a bad result.
- But we will get it quickly.



# Random Sampling for the TSP

- We could write down the cities in a random order.
- If we do this, we will get a bad result.
- But we will get it quickly.
- And we can do this often.



# Random Sampling for the TSP



- We could write down the cities in a random order.
- If we do this, we will get a bad result.
- But we will get it quickly.
- And we can do this often.
- For this small problem, my computer can do this *millions* of times in a few seconds.

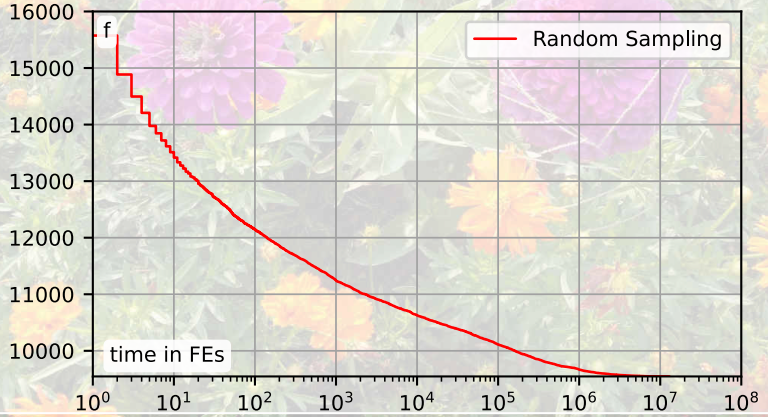
# Random Sampling for the TSP



- We could write down the cities in a random order.
- If we do this, we will get a bad result.
- But we will get it quickly.
- And we can do this often.
- For this small problem, my computer can do this *millions* of times in a few seconds.
- We can let the algorithm run for as much time as we can wait ... and then simply take the best random tour we found.

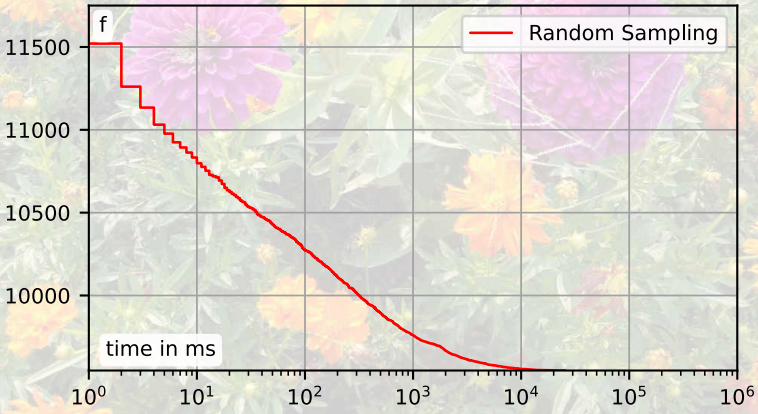
# Random Sampling for the TSP

- We could write down the cities in a random order.
- This is the improvement of the best-so-far solution of this so-called Random Sampling algorithm over the number of solutions it has generated and tested (the so-called objective function evaluations (FEs)).



# Random Sampling for the TSP

- We could write down the cities in a random order.
- This is the improvement of the best-so-far solution of this so-called Random Sampling algorithm over the consumed runtime in milliseconds.



# Random Sampling for the TSP

- We could write down the cities in a random order.
- And for this small problem, this random sampling eventually finds the optimal tour.




Principle 1: We can randomly construct solutions.

Principle 1: We can randomly construct solutions.

Problem 1: Guessing an optimal (or even just good) solution randomly is very unlikely.

# Why don't we use the A* Algorithm for the TSP?

- Before, we talked about the A* Algorithm for path finding.

# Why don't we use the A* Algorithm for the TSP?

- Before, we talked about the A* Algorithm for path finding.
- Isn't the TSP about finding a path?

## Why don't we use the A* Algorithm for the TSP?

- Before, we talked about the A* Algorithm for path finding.
- Isn't the TSP about finding a path?
- Why do we not use it for the TSP?

# Why don't we use the A* Algorithm for the TSP?

- Before, we talked about the A* Algorithm for path finding.
- Isn't the TSP about finding a path?
- Why do we not use it for the TSP?

1. The A* Algorithm needs memory exponential in the number of nodes in the final solution[106].

# Why don't we use the A* Algorithm for the TSP?

- Before, we talked about the A* Algorithm for path finding.
- Isn't the TSP about finding a path?
- Why do we not use it for the TSP?

1. The A* Algorithm needs memory exponential in the number of nodes in the final solution[106]. Which is the number of cities in the TSP...

# Why don't we use the A* Algorithm for the TSP?

- Before, we talked about the A* Algorithm for path finding.
- Isn't the TSP about finding a path?
- Why do we not use it for the TSP?

1. The A* Algorithm needs memory exponential in the number of nodes in the final solution[106]. Which is the number of cities in the TSP...
2. Also, it is a bit awkward: What are the heuristic and the goal state?

# Why don't we use the A* Algorithm for the TSP?

- Before, we talked about the A* Algorithm for path finding.
- Isn't the TSP about finding a path?
- Why do we not use it for the TSP?
- It will crash for any larger TSP.

1. The A* Algorithm needs memory exponential in the number of nodes in the final solution[106]. Which is the number of cities in the TSP...
2. Also, it is a bit awkward: What are the heuristic and the goal state?

# Why don't we use the A* Algorithm for the TSP?

- Before, we talked about the A* Algorithm for path finding.
- Isn't the TSP about finding a path?
- Why do we not use it for the TSP?
- It will crash for any larger TSP.
- But it can be done and used for our small TSP.

1. The A* Algorithm needs memory exponential in the number of nodes in the final solution[106]. Which is the number of cities in the TSP...
2. Also, it is a bit awkward: What are the heuristic and the goal state?

# Why don't we use the A* Algorithm for the TSP?

- Before, we talked about the A* Algorithm for path finding.
- Isn't the TSP about finding a path?
- Why do we not use it for the TSP?
- It will crash for any larger TSP.
- But it can be done and used for our small TSP.
- So let's do it.

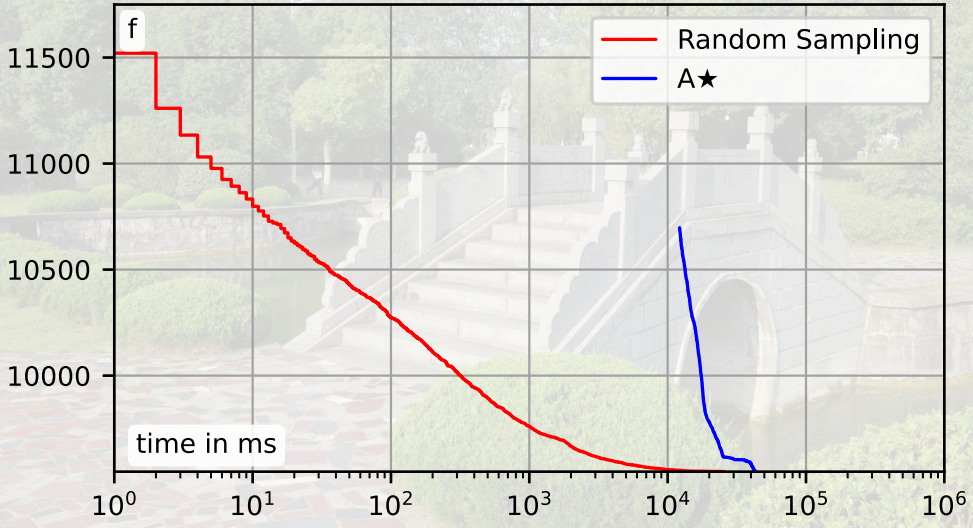
1. The A* Algorithm needs memory exponential in the number of nodes in the final solution[106]. Which is the number of cities in the TSP...
2. Also, it is a bit awkward: What are the heuristic and the goal state?

# Why don't we use the A* Algorithm for the TSP?

- Before, we talked about the A* Algorithm for path finding.
- Isn't the TSP about finding a path?
- Why do we not use it for the TSP?
- It will crash for any larger TSP.
- But it can be done and used for our small TSP.
- So let's do it. (I spare you all details.)

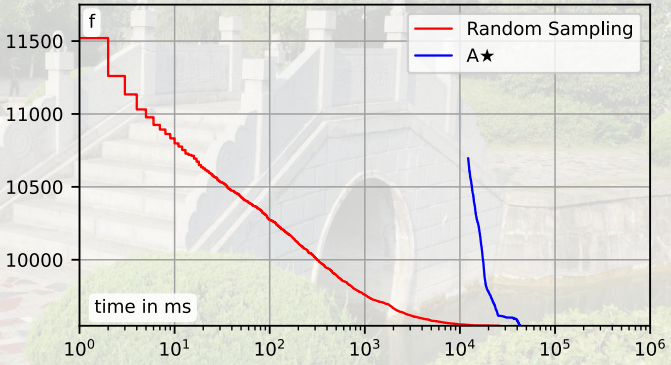
1. The A* Algorithm needs memory exponential in the number of nodes in the final solution[106]. Which is the number of cities in the TSP...
2. Also, it is a bit awkward: What are the heuristic and the goal state?

# Why don't we use the A* Algorithm for the TSP?



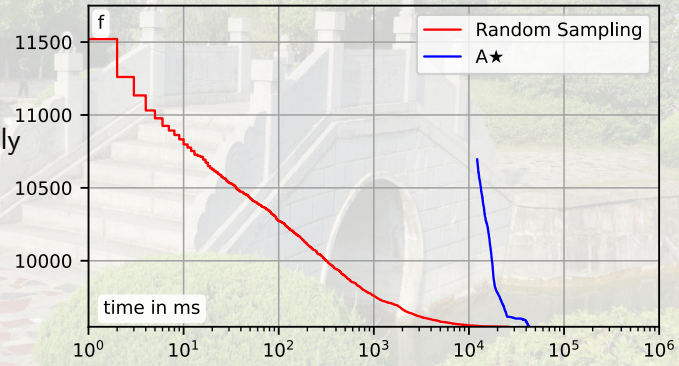
# Why don't we use the A$^\star$ Algorithm for the TSP?

- Because not only will it use too much memory for larger instances...



# Why don't we use the A$^\star$ Algorithm for the TSP?

- Because not only will it use too much memory for larger instances...
- ...it is also slower than randomly guessing on smaller instances.



# Random Sampling still is a bad algorithm

- Random sampling is a bad algorithm for the TSP (and basically all other problems, too).

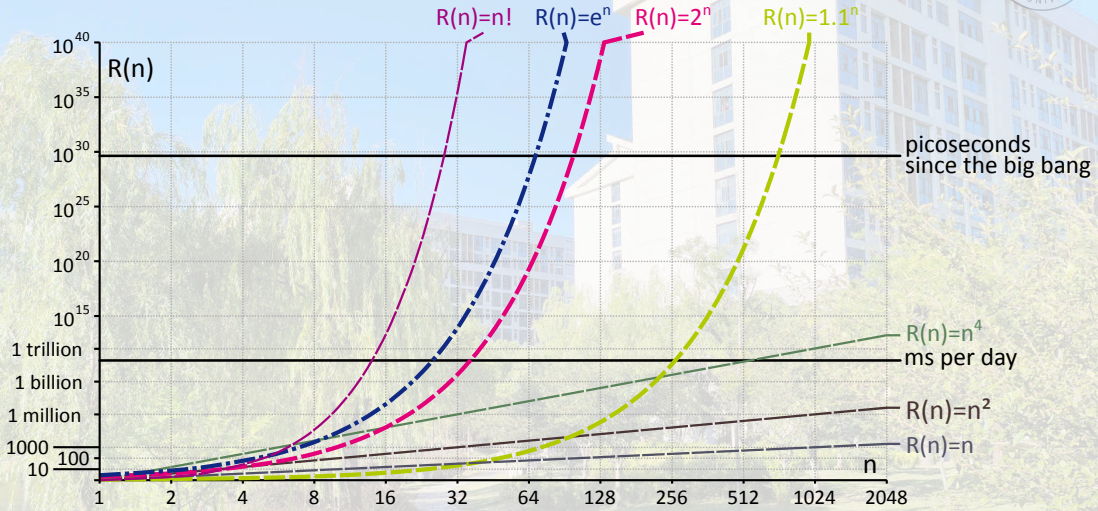
# Random Sampling still is a bad algorithm

- Random sampling is a bad algorithm for the TSP (and basically all other problems, too).
- It relies on randomly guessing good tours.

# Random Sampling still is a bad algorithm

- Random sampling is a bad algorithm for the TSP (and basically all other problems, too).
- It relies on randomly guessing good tours.
- If we have $n$ cities, then there are $1 * 2 * 3 * \cdots * (n-1) * n = n!$ tours.

# Random Sampling still is a bad algorithm



# Random Sampling still is a bad algorithm

- Random sampling is a bad algorithm for the TSP (and basically all other problems, too).
- It relies on randomly guessing good tours.
- If we have $n$ cities, then there are $1 * 2 * 3 * \cdots * (n-1) * n = n!$ tours.
- Factorial growth ($n!$) is even worse than exponential growth...

# Random Sampling still is a bad algorithm

- Random sampling is a bad algorithm for the TSP (and basically all other problems, too).
- It relies on randomly guessing good tours.
- If we have $n$ cities, then there are $1 * 2 * 3 * \cdots * (n-1) * n = n!$ tours.
- Factorial growth ($n!$) is even worse than exponential growth...
- So if we try to randomly sample the best possible tour, our chance is extremely small...

# Local Search: Using Information



## Random Sampling is a bad algorithm




- In each step, random sampling creates a completely random tour.

## Random Sampling is a bad algorithm




- In each step, random sampling creates a completely random tour.
- It does not gather any information.

## Random Sampling is a bad algorithm




- In each step, random sampling creates a completely random tour.
- It does not gather any information.
- It keeps the best tour, but does not use any information inside this tour.

# Random Sampling is a bad algorithm




- In each step, random sampling creates a completely random tour.
- It does not gather any information.
- It keeps the best tour, but does not use any information inside this tour.
- Just guessing answers randomly is not a good method.

# Random Sampling is a bad algorithm




- In each step, random sampling creates a completely random tour.
- It does not gather any information.
- It keeps the best tour, but does not use any information inside this tour.
- Just guessing answers randomly is not a good method.
- Clearly, seeing millions of tours, we should be able learn *something* and somehow use that to find better tours???

# Randomized Local Search



- But what could we do?

# Randomized Local Search

- But what could we do?
- "In the neighborhood of a solution, there will probably be better or worse solutions."

# Randomized Local Search



- But what could we do?
- "In the neighborhood of a solution, there will probably be better or worse solutions."
- neighborhood = solutions that are similar.

## Randomized Local Search



- But what could we do?
- "In the neighborhood of a solution, there will probably be better or worse solutions."
- neighborhood $=$ solutions that are similar.
- If we have a tour $x$, we could randomly pick two cities in the tour and swap them.

# Randomized Local Search



- But what could we do?
- "In the neighborhood of a solution, there will probably be better or worse solutions."
- neighborhood $=$ solutions that are similar.
- If we have a tour $x$, we could randomly pick two cities in the tour and swap them.
- If we do this, maybe we could get a better tour or a worse tour.

# Randomized Local Search



- But what could we do?
- "In the neighborhood of a solution, there will probably be better or worse solutions."
- neighborhood $=$ solutions that are similar.
- If we have a tour $x$, we could randomly pick two cities in the tour and swap them.
- If we do this, maybe we could get a better tour or a worse tour.
- We could keep a better tour, but throw away a worse one.

# Randomized Local Search



- But what could we do?
- "In the neighborhood of a solution, there will probably be better or worse solutions."
- neighborhood $=$ solutions that are similar.
- If we have a tour $x$, we could randomly pick two cities in the tour and swap them.
- If we do this, maybe we could get a better tour or a worse tour.
- We could keep a better tour, but throw away a worse one.
- Then we do the same with a better tour that we find, and with yet a better tour, and so on.

# Randomized Local Search



- But what could we do?
- "In the neighborhood of a solution, there will probably be better or worse solutions."
- neighborhood $=$ solutions that are similar.
- If we have a tour $x$, we could randomly pick two cities in the tour and swap them.
- If we do this, maybe we could get a better tour or a worse tour.
- We could keep a better tour, but throw away a worse one.
- Then we do the same with a better tour that we find, and with yet a better tour, and so on.
- This method is called randomized local search (RLS).

# Randomized Local Search for the TSP




15983 km

合肥, 北京, 西安, 南京, 哈尔滨,
香港, 长沙, 昆明, 武汉, 上海, 重庆

# Randomized Local Search for the TSP




16080 km

合肥, 北京, 西安, 南京, 哈尔滨,
香港, 长沙, 昆明, 武汉, 上海, 重庆

合肥, 哈尔滨, 西安, 南京, 北京,
香港, 长沙, 昆明, 武汉, 上海, 重庆

# Randomized Local Search for the TSP




16080 km

合肥, 北京, 西安, 南京, 哈尔滨,
香港, 长沙, 昆明, 武汉, 上海, 重庆

合肥, 哈尔滨, 西安, 南京, 北京,
香港, 长沙, 昆明, 武汉, 上海, 重庆

# Randomized Local Search for the TSP




15983 km

合肥, 北京, 西安, 南京, 哈尔滨,
香港, 长沙, 昆明, 武汉, 上海, 重庆

# Randomized Local Search for the TSP




15128 km

合肥, 北京, 西安, 南京, 哈尔滨,
香港, 长沙, 昆明, 武汉, 上海, 重庆

合肥, 北京, 西安, 南京, 上海,
香港, 长沙, 昆明, 武汉, 哈尔滨, 重庆

# Randomized Local Search for the TSP




15128 km

合肥, 北京, 西安, 南京, 上海,
香港, 长沙, 昆明, 武汉, 哈尔滨, 重庆

# Randomized Local Search for the TSP




16435 km

合肥, 北京, 西安, 南京, 上海,
香港, 长沙, 昆明, 武汉, 哈尔滨, 重庆

合肥, 北京, 西安, 南京, 香港,
上海, 长沙, 昆明, 武汉, 哈尔滨, 重庆

# Randomized Local Search for the TSP




16435 km

合肥, 北京, 西安, 南京, 上海,
香港, 长沙, 昆明, 武汉, 哈尔滨, 重庆

合肥, 北京, 西安, 南京, 香港,
上海, 长沙, 昆明, 武汉, 哈尔滨, 重庆

# Randomized Local Search for the TSP




15128 km

合肥, 北京, 西安, 南京, 上海,
香港, 长沙, 昆明, 武汉, 哈尔滨, 重庆

# Randomized Local Search for the TSP




12432 km

合肥, 北京, 西安, 南京, 上海,
香港, 长沙, 昆明, 武汉, 哈尔滨, 重庆

合肥, 北京, 哈尔滨, 南京, 上海,
香港, 长沙, 昆明, 武汉, 西安, 重庆

# Randomized Local Search for the TSP




12432 km

合肥, 北京, 哈尔滨, 南京, 上海,
香港, 长沙, 昆明, 武汉, 西安, 重庆

# Randomized Local Search for the TSP




14036 km

合肥, 北京, 哈尔滨, 南京, 上海,
香港, 长沙, 昆明, 武汉, 西安, 重庆

合肥, 北京, 哈尔滨, 武汉, 上海,
香港, 长沙, 昆明, 南京, 西安, 重庆

# Randomized Local Search for the TSP




14036 km

合肥, 北京, 哈尔滨, 南京, 上海,
香港, 长沙, 昆明, 武汉, 西安, 重庆

合肥, 北京, 哈尔滨, 武汉, 上海,
香港, 长沙, 昆明, 南京, 西安, 重庆

# Randomized Local Search for the TSP




12432 km

合肥, 北京, 哈尔滨, 南京, 上海,
香港, 长沙, 昆明, 武汉, 西安, 重庆

# Randomized Local Search for the TSP




10857 km

合肥, 北京, 哈尔滨, 南京, 上海,
香港, 长沙, 昆明, 武汉, 西安, 重庆

合肥, 北京, 哈尔滨, 南京, 上海,
香港, 长沙, 昆明, 重庆, 西安, 武汉

# Randomized Local Search for the TSP




10857 km

合肥, 北京, 哈尔滨, 南京, 上海,
香港, 长沙, 昆明, 重庆, 西安, 武汉

# Randomized Local Search for the TSP




10620 km

合肥, 北京, 哈尔滨, 南京, 上海,
香港, 长沙, 昆明, 重庆, 西安, 武汉

合肥, 北京, 哈尔滨, 南京, 上海,
长沙, 香港, 昆明, 重庆, 西安, 武汉

# Randomized Local Search for the TSP




10620 km

合肥, 北京, 哈尔滨, 南京, 上海,
长沙, 香港, 昆明, 重庆, 西安, 武汉

# Randomized Local Search for the TSP




10567 km

合肥, 北京, 哈尔滨, 南京, 上海,
长沙, 香港, 昆明, 重庆, 西安, 武汉

合肥, 北京, 哈尔滨, 上海, 南京,
长沙, 香港, 昆明, 重庆, 西安, 武汉

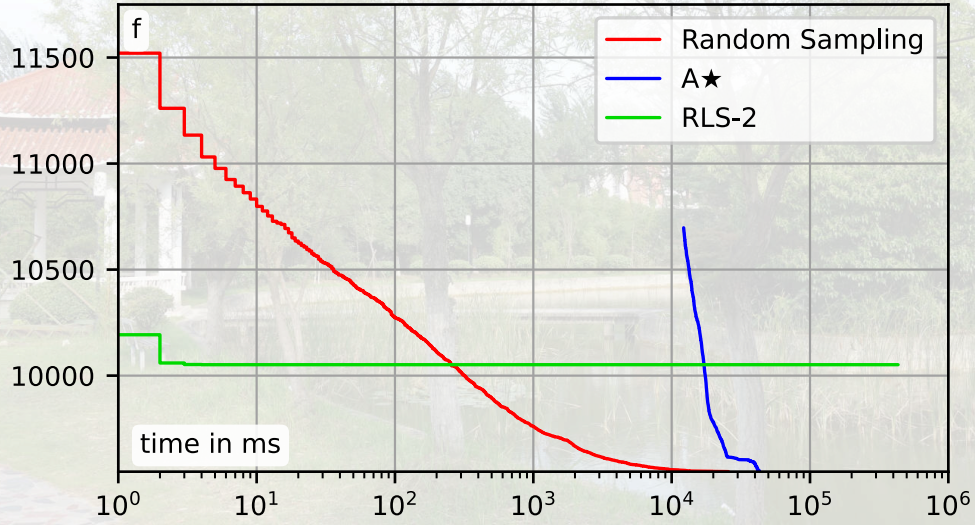
# Randomized Local Search for the TSP




10567 km

合肥, 北京, 哈尔滨, 上海, 南京,
长沙, 香港, 昆明, 重庆, 西安, 武汉

# Randomized Local Search for the TSP



# Randomized Local Search for the TSP



- RLS is much faster than random sampling.

# Randomized Local Search for the TSP



- RLS is much faster than random sampling.
- However...

# Randomized Local Search for the TSP

- RLS is much faster than random sampling.
- However, if our algorithm is at a tour $x$, then it can only find other tours by swapping two cities.

# Randomized Local Search for the TSP

- RLS is much faster than random sampling.
- However, if our algorithm is at a tour $x$, then it can only find other tours by swapping two cities.
- There are some tours that it cannot reach in one step.

# Randomized Local Search for the TSP




- RLS is much faster than random sampling.
- However, if our algorithm is at a tour $x$, then it can only find other tours by swapping two cities.
- There are some tours that it cannot reach in one step.
- If all tours that we can reach in one step are worse than $x$, we will never leave $x$.

# Randomized Local Search for the TSP

- RLS is much faster than random sampling.
- However, if our algorithm is at a tour $x$, then it can only find other tours by swapping two cities.
- There are some tours that it cannot reach in one step.
- If all tours that we can reach in one step are worse than $x$, we will never leave $x$.
- If $x$ is not the optimum, then we are stuck and won't ever find the optimal/best tour.



Principle 2: Applying a small random change to an existing solution sometimes can give us a better solution.



Principle 2: Applying a small random change to an existing solution sometimes can give us a better solution. This way, we can step-by-step get better solutions.



Principle 2: Applying a small random change to an existing solution sometimes can give us a better solution. This way, we can step-by-step get better solutions.

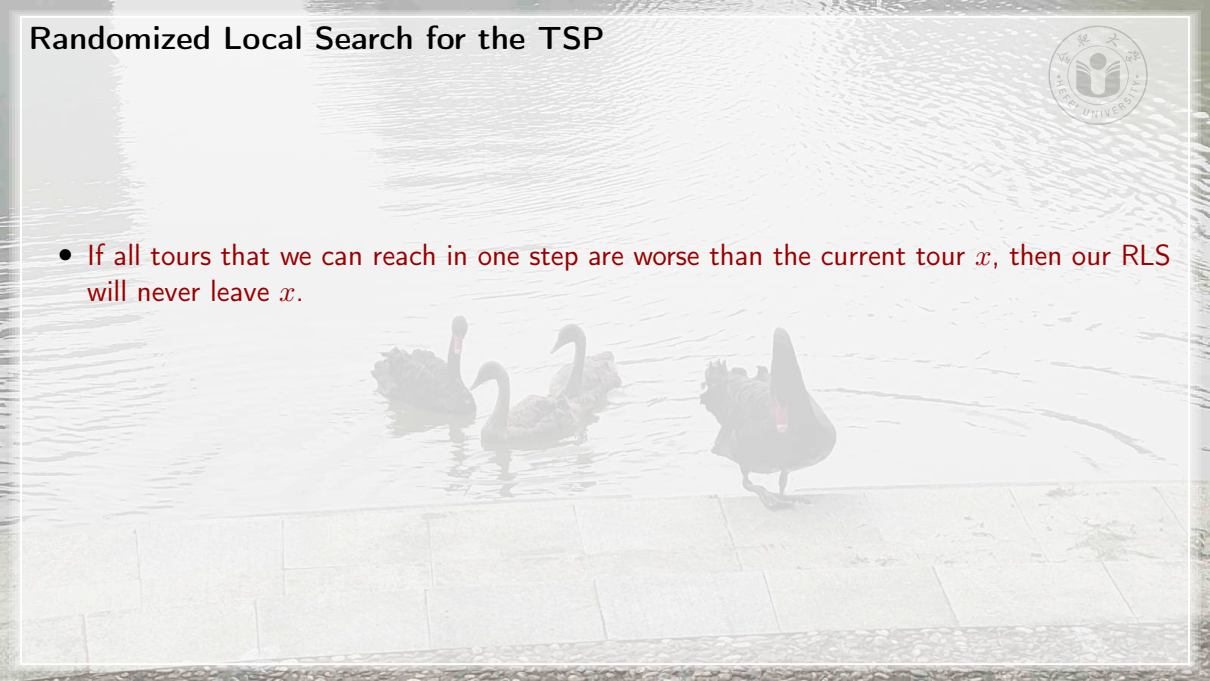
Problem 2: Sometimes, a small change is not enough.



Principle 2: Applying a small random change to an existing solution sometimes can give us a better solution. This way, we can step-by-step get better solutions.

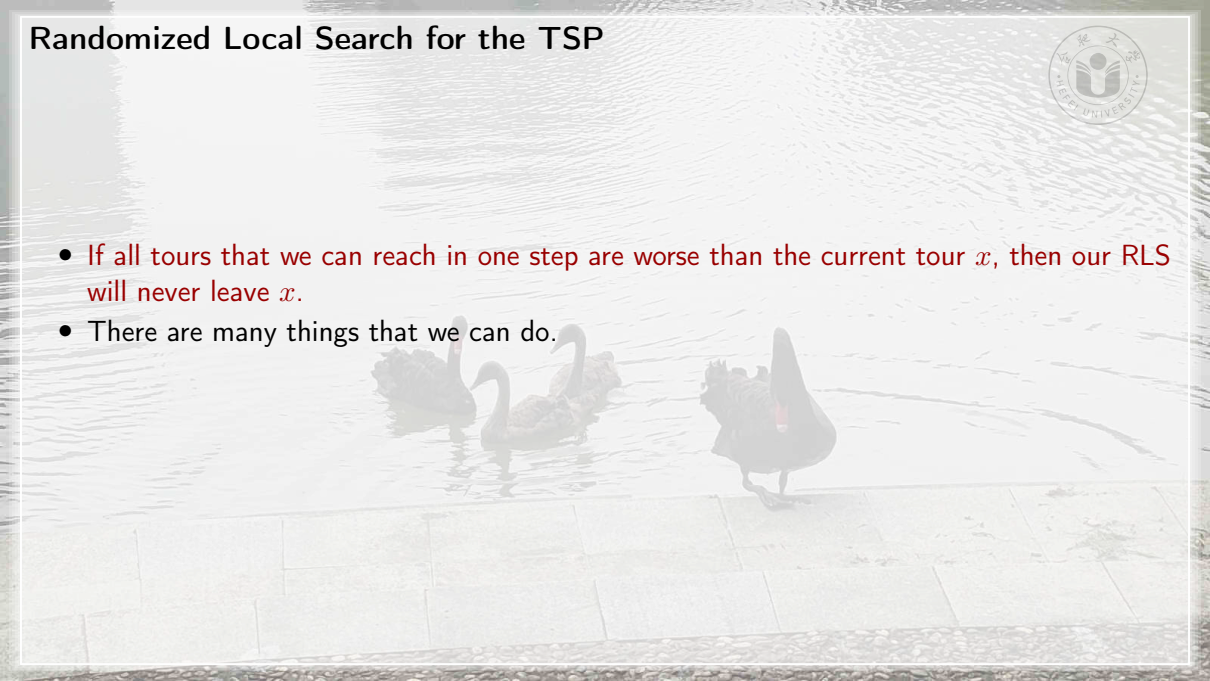
Problem 2: Sometimes, a small change is not enough. We can get stuck at a so-called *local* optimum.

# Randomized Local Search for the TSP



- If all tours that we can reach in one step are worse than the current tour $x$, then our RLS will never leave $x$.

# Randomized Local Search for the TSP



- If all tours that we can reach in one step are worse than the current tour $x$, then our RLS will never leave $x$.
- There are many things that we can do.

# Randomized Local Search for the TSP



- If all tours that we can reach in one step are worse than the current tour $x$, then our RLS will never leave $x$.
- There are many things that we can do, for example:
  1. We could restart the algorithm at a new starting point after some time.

# Randomized Local Search for the TSP



- If all tours that we can reach in one step are worse than the current tour $x$, then our RLS will never leave $x$.
- There are many things that we can do, for example:
  1. We could restart the algorithm at a new starting point after some time.
  2. Or we could sometimes allow it swap more cities in the current tour.

# Randomized Local Search with a Larger Neighborhood




15151 km

合肥, 重庆, 武汉, 长沙, 北京, 上海,
南京, 昆明, 香港, 西安, 哈尔滨

# Randomized Local Search with a Larger Neighborhood




15195 km

合肥, 重庆, 武汉, 长沙, 北京, 上海, 南京, 昆明, 香港, 西安, 哈尔滨

合肥, 重庆, 武汉, 长沙, 北京, 南京, 上海, 昆明, 香港, 西安, 哈尔滨

# Randomized Local Search with a Larger Neighborhood




15195 km

合肥, 重庆, 武汉, 长沙, 北京, 上海, 南京, 昆明, 香港, 西安, 哈尔滨

合肥, 重庆, 武汉, 长沙, 北京, 南京, 上海, 昆明, 香港, 西安, 哈尔滨

# Randomized Local Search with a Larger Neighborhood




15151 km

合肥, 重庆, 武汉, 长沙, 北京, 上海,
南京, 昆明, 香港, 西安, 哈尔滨

# Randomized Local Search with a Larger Neighborhood




14539 km

合肥, 重庆, 武汉, 长沙, 北京, 上海,
南京, 昆明, 香港, 西安, 哈尔滨

合肥, 南京, 武汉, 长沙, 北京, 上海,
昆明, 重庆, 香港, 西安, 哈尔滨

# Randomized Local Search with a Larger Neighborhood




14539 km

合肥, 南京, 武汉, 长沙, 北京, 上海,
昆明, 重庆, 香港, 西安, 哈尔滨

# Randomized Local Search with a Larger Neighborhood




16624 km

合肥, 南京, 武汉, 长沙, 北京, 上海,
昆明, 重庆, 香港, 西安, 哈尔滨

昆明, 南京, 武汉, 长沙, 北京, 上海,
合肥, 重庆, 香港, 西安, 哈尔滨

# Randomized Local Search with a Larger Neighborhood




16624 km

合肥, 南京, 武汉, 长沙, 北京, 上海,
昆明, 重庆, 香港, 西安, 哈尔滨

昆明, 南京, 武汉, 长沙, 北京, 上海,
合肥, 重庆, 香港, 西安, 哈尔滨

# Randomized Local Search with a Larger Neighborhood




14539 km

合肥, 南京, 武汉, 长沙, 北京, 上海,
昆明, 重庆, 香港, 西安, 哈尔滨

# Randomized Local Search with a Larger Neighborhood




13882 km

合肥, 南京, 武汉, 长沙, 北京, 上海,
昆明, 重庆, 香港, 西安, 哈尔滨

合肥, 南京, 武汉, 长沙, 北京, 上海,
重庆, 昆明, 香港, 西安, 哈尔滨

# Randomized Local Search with a Larger Neighborhood




13882 km

合肥, 南京, 武汉, 长沙, 北京, 上海,
重庆, 昆明, 香港, 西安, 哈尔滨

# Randomized Local Search with a Larger Neighborhood




10688 km

合肥, 南京, 武汉, 长沙, 北京, 上海,
重庆, 昆明, 香港, 西安, 哈尔滨

合肥, 南京, 武汉, 哈尔滨, 北京, 西安,
重庆, 昆明, 香港, 长沙, 上海

# Randomized Local Search with a Larger Neighborhood




10688 km

合肥, 南京, 武汉, 哈尔滨, 北京, 西安,
重庆, 昆明, 香港, 长沙, 上海

# Randomized Local Search with a Larger Neighborhood

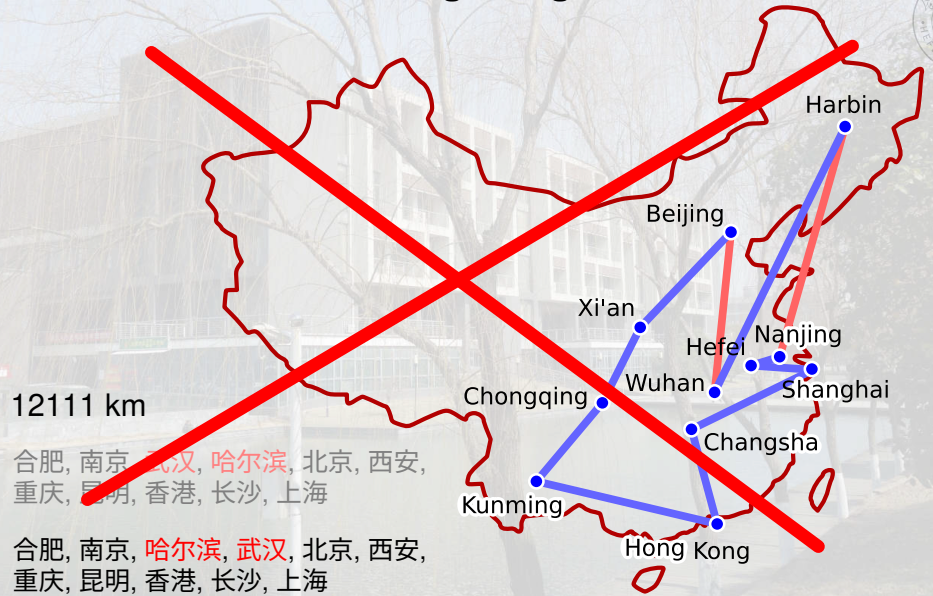



12111 km

合肥, 南京, 武汉, 哈尔滨, 北京, 西安, 重庆, 昆明, 香港, 长沙, 上海

合肥, 南京, 哈尔滨, 武汉, 北京, 西安, 重庆, 昆明, 香港, 长沙, 上海

# Randomized Local Search with a Larger Neighborhood




12111 km

合肥, 南京, 武汉, 哈尔滨, 北京, 西安,
重庆, 昆明, 香港, 长沙, 上海

合肥, 南京, 哈尔滨, 武汉, 北京, 西安,
重庆, 昆明, 香港, 长沙, 上海

# Randomized Local Search with a Larger Neighborhood




9547 km

合肥, 南京, 武汉, 哈尔滨, 北京, 西安,
重庆, 昆明, 香港, 长沙, 上海

合肥, 南京, 上海, 哈尔滨, 北京, 西安,
重庆, 昆明, 香港, 长沙, 武汉

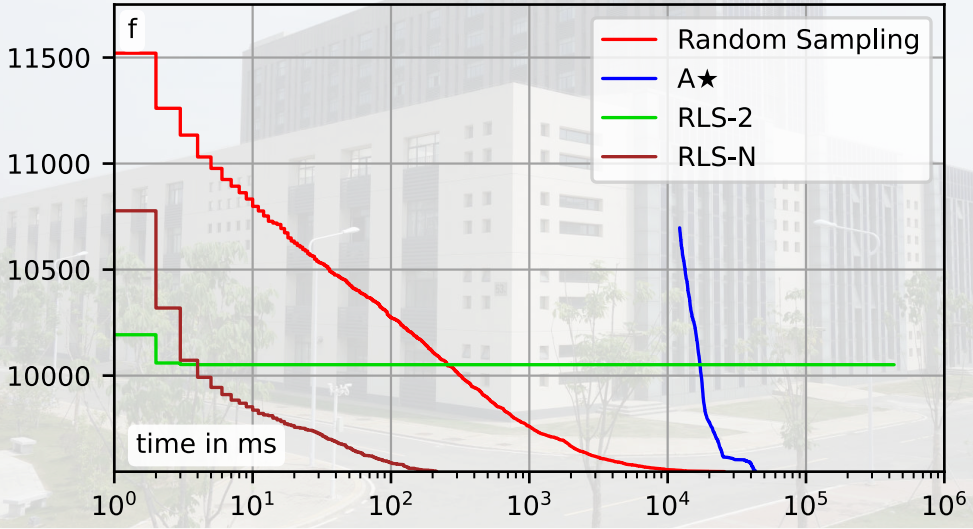
# Randomized Local Search with a Larger Neighborhood




9547 km

合肥, 南京, 上海, 哈尔滨, 北京, 西安,
重庆, 昆明, 香港, 长沙, 武汉

# Randomized Local Search with a Larger Neighborhood



# Randomized Local Search with a Larger Neighborhood



- If all tours that we can reach in one step are worse than the current tour $x$, then our RLS will never leave $x$.
- There are many things that we can do, for example:
  1. We could restart the algorithm at a new starting point after some time.
  2. Or we could sometimes allow it swap more cities in the current tour.

# Randomized Local Search with a Larger Neighborhood



- If all tours that we can reach in one step are worse than the current tour $x$, then our RLS will never leave $x$.
- There are many things that we can do, for example:
  1. We could restart the algorithm at a new starting point after some time.
  2. Or we could sometimes allow it swap more cities in the current tour.
- The larger neighborhood of solutions that can be reached in one step makes the search slower, but allows it to escape from local optima.




Principle 3: Larger changes can help us escape from local optima.

Principle 3: Larger changes can help us escape from local optima.

Problem 3: Larger changes are less likely to yield improvements (because the new solution is more different), so they slow down the search.

There are many more principles and problems surrounding (metaheuristic) optimization.

There are many more principles and problems surrounding (metaheuristic) optimization.

But we will leave it at what we have seen so far.

# Summary

## Summary on Algorithms



- We have learned some basic algorithmic principles.

## Summary on Algorithms




- We have learned some basic algorithmic principles.
- Current research tries to improve both the speed of algorithms as well as the solution quality.

## Summary on Algorithms

- We have learned some basic algorithmic principles.
- Current research tries to improve both the speed of algorithms as well as the solution quality.
- This requires carefully balancing the step-size of algorithms and to develop methods against getting stuck at local optima.

## Summary on the TSP



- Today, we can actually solve TSPs with tens of thousands of cities to optimality[4,48].

## Summary on the TSP

- Today, we can actually solve TSPs with tens of thousands of cities to optimality[4,48].
- For this, we actually use principles similar to the A* Algorithm, just a bit differently, in methods[4,49] like branch and bound (B&B)[138].

## Summary on the TSP



- Today, we can actually solve TSPs with tens of thousands of cities to optimality[4,48].
- For this, we actually use principles similar to the A* Algorithm, just a bit differently, in methods[4,49] like branch and bound (B&B)[138].
- We can get close-to-optimal solutions for TSPs with millions of cities.

## Summary on the TSP



- Today, we can actually solve TSPs with tens of thousands of cities to optimality[4,48].
- For this, we actually use principles similar to the A* Algorithm, just a bit differently, in methods[4,49] like branch and bound (B&B)[138].
- We can get close-to-optimal solutions for TSPs with millions of cities.
- For this, we use algorithms a bit similar to randomized local search (RLS), but with more targeted search steps, e.g., in the Lin-Kernighan Helsgaun (LKH) algorithm[100,101].

## Summary on the TSP



- Today, we can actually solve TSPs with tens of thousands of cities to optimality[4,48]. [but not all of them!]
- For this, we actually use principles similar to the A* Algorithm, just a bit differently, in methods[4,49] like branch and bound (B&B)[138].
- We can get close-to-optimal solutions for TSPs with millions of cities.

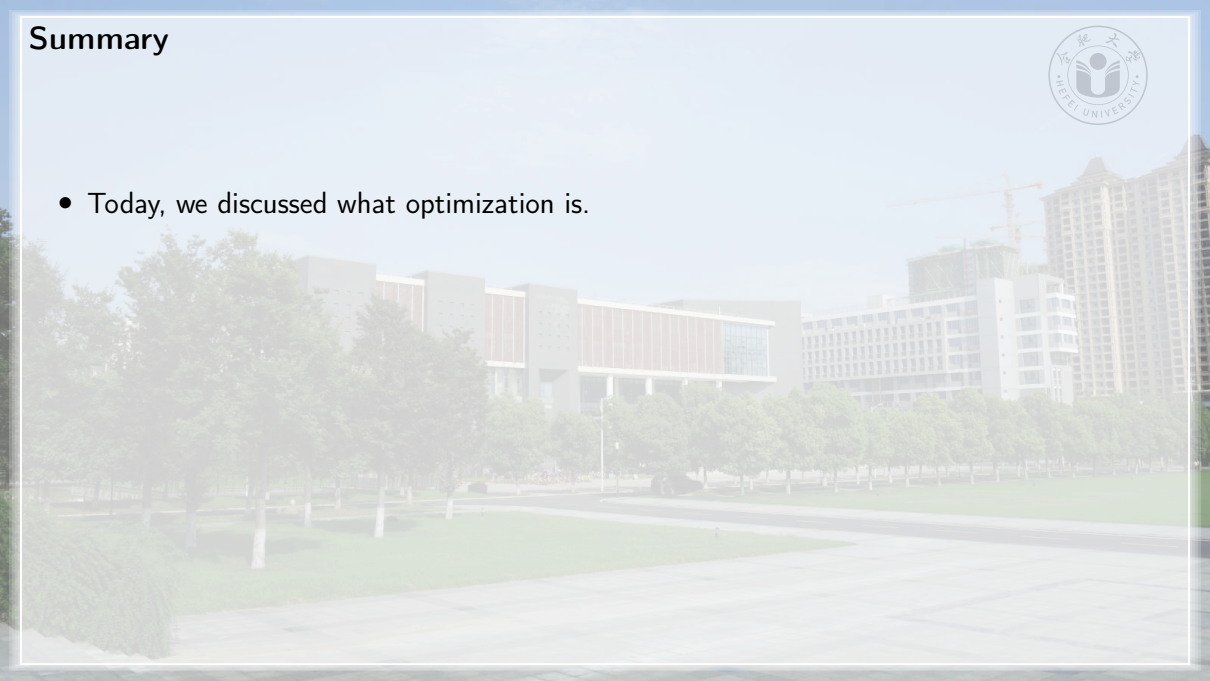
- For this, we use algorithms a bit similar to randomized local search (RLS), but with more targeted search steps, e.g., in the Lin-Kernighan Helsgaun (LKH) algorithm[100,101].

## Summary on the TSP



- Today, we can actually solve TSPs with tens of thousands of cities to optimality[4,48]. [but not all of them!]
- For this, we actually use principles similar to the A* Algorithm, just a bit differently, in methods[4,49] like branch and bound (B&B)[138].
- We can get close-to-optimal solutions for TSPs with millions of cities. [close-to-optimal, but not optimal (at least not always)!]
- For this, we use algorithms a bit similar to randomized local search (RLS), but with more targeted search steps, e.g., in the Lin-Kernighan Helsgaun (LKH) algorithm[100,101].

## Summary



- Today, we discussed what optimization is.

## Summary




- Today, we discussed what optimization is.
- Today, you also saw some of the basic principles and methods that are inside of the algorithms used in optimization and operations research.

## Summary




- Today, we discussed what optimization is.
- Today, you also saw some of the basic principles and methods that are inside of the algorithms used in optimization and operations research.
- You can imagine that the same principles that we tested on the TSP will work on many different kinds of problems.

## Summary




- Today, we discussed what optimization is.
- Today, you also saw some of the basic principles and methods that are inside of the algorithms used in optimization and operations research.
- You can imagine that the same principles that we tested on the TSP will work on many different kinds of problems.
- For example: As long as we can randomly construct and randomly modify a solution, we can attack the problem with randomized local search (RLS).

# Summary

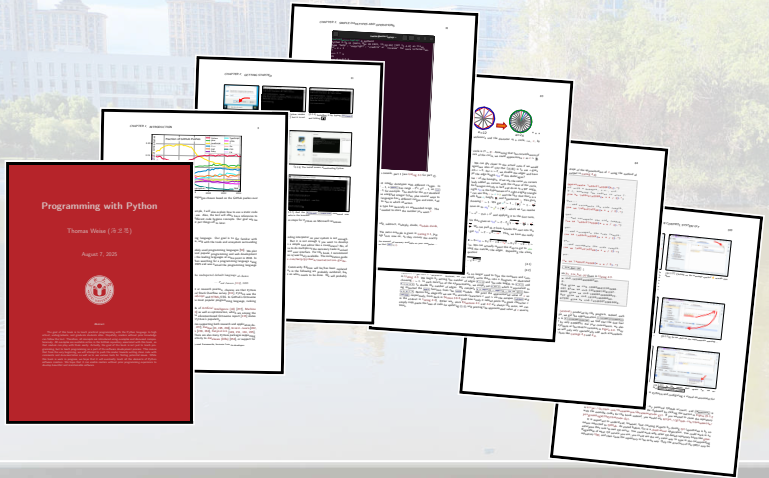



- Today, we discussed what optimization is.
- Today, you also saw some of the basic principles and methods that are inside of the algorithms used in optimization and operations research.
- You can imagine that the same principles that we tested on the TSP will work on many different kinds of problems.
- For example: As long as we can randomly construct and randomly modify a solution, we can attack the problem with randomized local search (RLS).
- Understanding the basic principles of optimization is not very hard.

# Advertisement

# Programming with Python

We have a freely available course book on *Programming with Python* at `https://thomasweise.github.io/programmingWithPython`, with focus on practical software development using the Python ecosystem of tools[211].



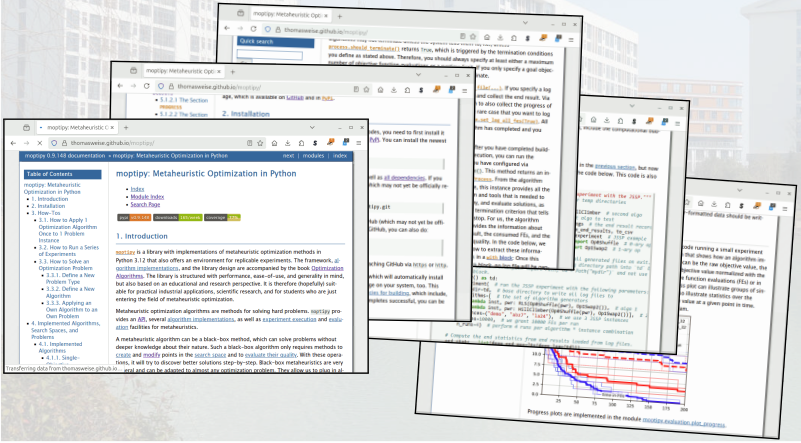
# Databases

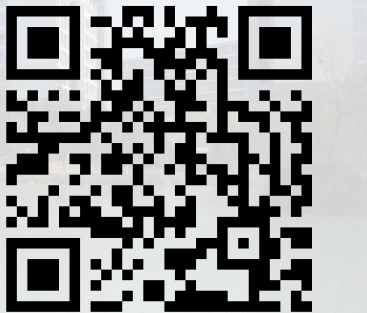
We have a freely available course book on *Databases* at `https://thomasweise.github.io/databases`, with actual practical examples using a real database management system (DBMS)[207].



# Metaheuristic Optimization in Python: moptipy

We offer moptipy[219], a mature open source Python package for metaheuristic optimization, which implements several algorithms, can run self-documenting experiments in parallel and in a distributed fashion, and offers statistical evaluation tools.



谢谢您们！

Thank you!

Vielen Dank!



# References I


[1] Joseph Adams, Egon Balas, and Daniel Zawack. "The Shifting Bottleneck Procedure for Job Shop Scheduling". *Management Science* 34(3):391–401, Mar. 1988. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0025-1909. doi:10.1287/mnsc.34.3.391 (cit. on p. 346).

[2] Miguel F. Anjos. *The QAPLIB Challenge*. Edinburgh, Scotland, UK: Miguel F. Anjos @ University of Edinburgh, Nov. 19, 2023. URL: https://www.miguelanjos.com/projects/2999 (visited on 2026-09-28). The document lists the 28 instances of QAPLIB[31,33] that had not yet been solved to optimality by the time of its writing. (Cit. on p. 350).

[3] Denis Antipov and Benjamin Doerr. "Precise Runtime Analysis for Plateaus". In: *15th International Conference on Parallel Problem Solving from Nature (PPSN XV)*. Vol. 2. Sept. 8–12, 2018, Coimbra, Portugal. Ed. by Anne Auger, Carlos M. Fonseca, Nuno Lourenço, Penousal Machado, Luís Paquete, and L. Darrell Whitley. Vol. 11102 of Lecture Notes in Computer Science (LNCS). Cham, Switzerland: Springer, 2018, pp. 117–128. ISSN: 0302-9743. ISBN: 978-3-319-99258-7. doi:10.1007/978-3-319-99259-4_10 (cit. on p. 349).

[4] David Lee Applegate, Robert E. Bixby, Vašek Chvátal, and William John Cook. *The Traveling Salesman Problem: A Computational Study*. 2nd ed. Vol. 17 of Princeton Series in Applied Mathematics. Princeton, NJ, USA: Princeton University Press, 2007. ISBN: 978-0-691-12993-8. This book also discusses the Concorde TSP solver. (Cit. on pp. 136, 160–170, 290–295, 343, 352).

[5] David Lee Applegate and William John Cook. "A Computational Study of the Job-Shop Scheduling Problem". *ORSA Journal on Computing* 3(2):149–156, May 1991. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0899-1499. doi:10.1287/ijoc.3.2.149. The Job Shop Scheduling Problem (JSSP) instances used were generated in Bonn in 1986. (Cit. on p. 346).

[6] Paul Gustav Heinrich Bachmann. *Die Analytische Zahlentheorie / Dargestellt von Paul Bachmann*. Vol. Zweiter Theil of Zahlentheorie: Versuch einer Gesamtdarstellung dieser Wissenschaft in ihren Haupttheilen. Leipzig, Sachsen, Germany: B. G. Teubner, 1894. ISBN: 978-1-4181-6963-3. URL: http://gallica.bnf.fr/ark:/12148/bpt6k994750 (visited on 2023-12-13) (cit. on p. 356).

[7] Thomas Bäck, David B. Fogel, and Zbigniew "Zbyszek" Michalewicz, eds. *Handbook of Evolutionary Computation*. Bristol, England, UK: IOP Publishing Ltd and Oxford, Oxfordshire, England, UK: Oxford University Press, 1997. ISBN: 978-0-7503-0392-7 (cit. on p. 343).

# References II


[8] Thomas Bartz-Beielstein, Carola Doerr, Daan van den Berg, Jakob Bossek, Sowmya Chandrasekaran, Tome Eftimov, Andreas Fischbach, Pascal Kerschke, William La Cava, Manuel López-Ibáñez, Katherine Mary Malan, Jason Hall Moore, Boris Naujoks, Patryk Orzechowski, Vanessa Volz, Markus Wagner, and Thomas Weise (汤卫思). "Benchmarking in Optimization: Best Practice and Open Issues". (abs/2007.03488), Dec. 18, 2020. doi:10.48550/arXiv.2007.03488. URL: https://arxiv.org/abs/2007.03488 (visited on 2025-07-25). arXiv:2007.03488v2 [cs.NE] 16 Dec 2020 (cit. on p. 343).

[9] Roberto Battiti and Giampietro Tecchiolli. "Local Search with Memory: Benchmarking RTS". *OR Spectrum: Quantitative Approaches in Management. Formerly Operations-Research-Spektrum* 17:67–86, June 1995. London, England, UK: Springer Nature Limited. ISSN: 0171-6468. doi:10.1007/BF01719249 (cit. on p. 351).

[10] Roberto Battiti and Giampietro Tecchiolli. "Simulated Annealing and Tabu Search in the Long Run: A Comparison on QAP Tasks". *Computers & Mathematics with Applications* 28(6):1–8, Sept. 1994. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0898-1221. doi:10.1016/0898-1221(94)00147-2 (cit. on pp. 351, 352).

[11] Roberto Battiti and Giampietro Tecchiolli. "The Reactive Tabu Search". 6(2):126–140, May 1994. doi:10.1287/ijoc.6.2.126. Applications of Reactive Tabu Search (RTS) to the 0-1 Knapsack and Quadratic Assignment Problem (QAP). (Cit. on pp. 351, 352).

[12] Roberto Battiti and Giampietro Tecchiolli. "Training Neural Nets with the Reactive Tabu Search". *IEEE Transactions on Neural Networks* 6(5):1185–1200, Sept. 1995. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: 1045-9227. doi:10.1109/72.410361. See also "Training Neural Nets with the Reactive Tabu Search, Technical Report UTM 421, Trento, Alto Adige/Südtirol, Italy: University of Trento, 11 1993" (cit. on p. 351).

[13] James C. Bean. "Genetic Algorithms and Random Keys for Sequencing and Optimization". *ORSA Journal on Computing* 6(2):107–220, Spr. 1994. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0899-1499. doi:10.1287/ijoc.6.2.154. Applies the Random Keys encoding to scheduling and the QAP. See [14]. (Cit. on pp. 307, 343, 350).

[14] James C. Bean. *Genetics and Random Keys for Sequencing and Optimization*. Tech. rep. 92-43. Ann Arbor, MI, USA: University of Michigan, Department of Industrial & Operations Engineering, June 30, 1992. URL: https://www.math.wm.edu/~rrkinc/homework/RdmKeyGA.pdf (visited on 2026-10-01). See [13]. (Cit. on pp. 307, 343, 350).

# References III


[15] John Edward Beasley. "OR-Library: Distributing Test Problems by Electronic Mail". *The Journal of the Operational Research Society (JORS)* 41:1069–1072, Nov. 1990. London, England, UK: Taylor and Francis Ltd. and London, England, UK: Springer Nature Limited. ISSN: 0160-5682. doi:10.1057/jors.1990.166. See the website[16] (cit. on pp. 308, 346, 349).

[16] John Edward Beasley. *Welcome to OR-Library*. Uxbridge, Middlesex, England, UK: Brunel University of London, June 1990–Feb. 2018. URL: https://people.brunel.ac.uk/~mastjjb/jeb/info.html (visited on 2026-09-30). See the article[15]. Contains instances of the JSSP, 1D- and 2D-Packing, Knapsack, periodic TSPs, amongst others. (Cit. on pp. 308, 346, 349).

[17] Andreas Beham, Michael Affenzeller, and Erik Pitzer. "Metaheuristic Algorithms for the Quadratic Assignment Problem: Performance and Comparison". In: *Innovative Technologies in Management and Science*. Ed. by Ryszard Klempous and Jan Nikodem. Vol. 10 of Topics in Intelligent Engineering and Informatics (TIEI). Cham, Switzerland: Springer, 2015, pp. 171–190. ISSN: 2193-9411. ISBN: 978-3-319-12651-7. doi:10.1007/978-3-319-12652-4_10. Algorithms like stochastic local search (SLS)/iterated local search (ILS), Simulated Annealing (SA), Tabu Search (TS), Variable Neighborhood Search (VNS), evolutionary algorithms (EAs), Memetic Algorithms (MAs) are compared and, with the exception of SA and EA, all perform best on some instance groups of QAPLIB. (Cit. on p. 350).

[18] Richard Ernest Bellman. *Dynamic Programming*. Princeton Landmarks in Mathematics and Physics. Princeton, NJ, USA: Princeton University Press, 1957–2003. ISBN: 978-0-691-14668-3 (cit. on p. 356).

[19] Bengt-Erik Bengtsson. "Packing Rectangular Pieces— A Heuristic Approach". *The Computer Journal* 25(3):353–357, Aug. 1982. Oxford, Oxfordshire, England, UK: Oxford University Press. ISSN: 0010-4620. doi:10.1093/comjnl/25.3.353 (cit. on p. 341).

[20] Tim Berners-Lee. *Re: Qualifiers on Hypertext links...* Geneva, Switzerland: World Wide Web project, European Organization for Nuclear Research (CERN) and Newsgroups: alt.hypertext, Aug. 6, 1991. URL: https://www.w3.org/People/Berners-Lee/1991/08/art-6484.txt (visited on 2025-02-05) (cit. on p. 354).

[21] Tim Berners-Lee, Larry Masinter, and Mark P. McCahill. *Uniform Resource Locators (URL)*. Request for Comments (RFC) 1738. Wilmington, DE, USA: Internet Engineering Task Force (IETF), Dec. 1994. URL: https://www.ietf.org/rfc/rfc1738.txt (visited on 2025-02-05) (cit. on p. 353).

# References IV


[22] Muhammed Beyaz, Tansel Dökeroğlu, and Ahmet Cosar. "Robust Hyper-Heuristic Algorithms for the Offline Oriented/Non-Oriented 2D Bin Packing Problems". *Applied Soft Computing (ASOC)* 36:236–245, Nov. 2015. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 1568-4946. doi:10.1016/j.asoc.2015.06.063 (cit. on p. 341).

[23] Christian Bierwirth. "A Generalized Permutation Approach to Job Shop Scheduling with Genetic Algorithms". *OR Spectrum: Quantitative Approaches in Management. Formerly Operations-Research-Spektrum* 17(2–3):87–92, June 1995. London, England, UK: Springer Nature Limited. ISSN: 0171-6468. doi:10.1007/BF01719250. URL: https://www.researchgate.net/publication/240263036 (visited on 2026-09-30). Proposes a permutations-with-repetitions based encoding for the JSSP, see also[24]. (Cit. on pp. 309, 318, 346, 355).

[24] Christian Bierwirth, Dirk C. Mattfeld, and Herbert Kopfer. "On Permutation Representations for Scheduling Problems". In: *4th International Conference on Parallel Problem Solving from Nature (PPSN IV)*. Sept. 22–24, 1996, Berlin, Germany. Ed. by Hans-Michael Voigt, Werner Ebeling, Ingo Rechenberg, and Hans-Paul Schwefel. Vol. 1141 of Lecture Notes in Computer Science (LNCS). Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany, 1996, pp. 310–318. ISSN: 0302-9743. ISBN: 978-3-540-61723-5. doi:10.1007/3-540-61723-X_995. URL: https://www.researchgate.net/publication/2753293 (visited on 2026-09-30). Proposes a permutations-with-repetitions based encoding for the JSSP, see also[23]. (Cit. on pp. 309, 318, 346, 355).

[25] Jacek Błażewicz, Wolfgang Domschke, and Erwin Pesch. "The Job Shop Scheduling Problem: Conventional and New Solution Techniques". *European Journal of Operational Research* 93(1):1–33, Aug. 1996. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/0377-2217(95)00362-2 (cit. on p. 346).

[26] Joshua Bloch. *Effective Java*. Reading, MA, USA: Addison-Wesley Professional, May 2008. ISBN: 978-0-321-35668-0 (cit. on p. 346).

[27] Christian Blum and Verena Schmid. "Solving the 2D Bin Packing Problem by Means of a Hybrid Evolutionary Algorithm". In: *International Conference on Computational Science (ICCS'2013)*. June 5–7, 2013, Barcelona, Spain. Ed. by Vassil Alexandrov, Michael Lees, Valeria V. Krzhizhanovskaya, Jack J. Dongarra, and Peter M. A. Sloot. Vol. 18 of Procedia Computer Science. Amsterdam, The Netherlands: Elsevier B.V., 2013, pp. 899–908. ISSN: 1877-0509. doi:10.1016/j.procs.2013.05.255. An EA for the oriented two-dimensional Bin Packing Problem with free cuts (2BP|O|F) using the heuristic by Wong and Lee [225]. (Cit. on p. 341).

# References V


[28] Borko Bošković and Janez Brest. "Computational Search of Long Skew-Symmetric Binary Sequences with High Merit Factors". *MENDEL – Soft Computing Journal* 28(2):17–24, Dec. 2022. Brno, Czech Republic: Brno University of Technology (Vysoké učení technické v Brně). ISSN: 1803-3814. doi:10.13164/mendel.2022.2.017 (cit. on p. 347).

[29] Borko Bošković, Franc Brglez, and Janez Brest. "Low-Autocorrelation Binary Sequences: On Improved Merit Factors and Runtime Predictions to Achieve Them". *Applied Soft Computing (ASOC)* 56:262–285, July 2017. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 1568-4946. doi:10.1016/J.ASOC.2017.02.024 (cit. on p. 347).

[30] Janez Brest and Borko Bošković. "A Heuristic Algorithm for a Low Autocorrelation Binary Sequence Problem with Odd Length and High Merit Factor". *IEEE Access* 6:4127–4134, Jan. 2018. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: 2169-3536. doi:10.1109/ACCESS.2018.2789916 (cit. on p. 347).

[31] Rainer E. Burkard, Eranda Çela, Stefan E. Karisch, Franz Rendl, Miguel F. Anjos, and Peter M. Hahn. *QAPLIB – A Quadratic Assignment Problem Library – Problem Instances and Solutions*. Edinburgh, Scotland, UK: University of Edinburgh, Apr. 20, 2012–Mar. 31, 2022. doi:10.7488/ds/3428. This is the 2012 status of the QAPLIB dataset from[33] (cit. on pp. 306, 310, 350).

[32] Rainer E. Burkard, Eranda Çela, Panos Miltiades Pardalos, and Leonidas S. Pitsoulis. "The Quadratic Assignment Problem". In: *Handbook of Combinatorial Optimization*. Ed. by Panos Miltiades Pardalos, Ding-Zhu Du, and Ronald Lewis Graham. 1st ed. Boston, MA, USA: Springer, 1998, pp. 1713–1809. ISBN: 978-1-4613-7987-4. doi:10.1007/978-1-4613-0303-9_27 (cit. on p. 350).

[33] Rainer E. Burkard, Stefan E. Karisch, and Franz Rendl. "QAPLIB – A Quadratic Assignment Problem Library". *Journal of Global Optimization* 10(4):391–403, June 1997. London, England, UK: Springer Nature Limited. ISSN: 0925-5001. doi:10.1023/A:1008293323270. This article introduces QAPLIB, a set of benchmark problems for the QAP, now hosted at [31]. (Cit. on pp. 306, 310, 350).

[34] Rainer E. Burkard and Josef Offermann. "Entwurf von Schreibmaschinentastaturen mittels quadratischer Zuordnungsprobleme". *Zeitschrift für Operations Research (ZOR). now Mathematical Methods of Operations Research (MMOR)* 21:B121–B132, Aug. 1977. Heidelberg, Baden-Württemberg, Germany: Physica-Verlag; now London, England, UK: Springer Nature Limited. ISSN: 0340-9422. doi:10.1007/BF01918175. Keyboard layout modeled as QAPs. (Cit. on p. 350).

# References VI


[35] Jacques Carlier and Éric Pinson. "An Algorithm for Solving the Job-Shop Problem". *Management Science* 35(2):164–176, 1989. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0025-1909. doi:10.1287/mnsc.35.2.164. A B&B method for the JSSP. (Cit. on p. 342).

[36] Eduardo Carvalho Pinto and Carola Doerr. *Towards a More Practice-Aware Runtime Analysis of Evolutionary Algorithms*. arXiv.org: Computing Research Repository (CoRR) abs/1812.00493. Ithaca, NY, USA: Cornell Universiy Library, Dec. 3, 2018. doi:10.48550/arXiv.1812.00493. URL: https://arxiv.org/abs/1812.00493 (visited on 2025-08-08). arXiv:1812.00493v1 [cs.NE] 3 Dec 2018 (cit. on p. 340).

[37] Vladimír Černý. "Thermodynamical Approach to the Traveling Salesman Problem: An Efficient Simulation Algorithm". *Journal of Optimization Theory and Applications* 45(1):41–51, Jan. 1985. New York, NY, USA: Springer Science+Business Media, LLC. ISSN: 0022-3239. doi:10.1007/BF00940812 (cit. on p. 351).

[38] Noureddine Chabini and Rachid Beguenane. "FPGA-Based Designs of the Factorial Function". In: *IEEE Canadian Conference on Electrical and Computer Engineering (CCECE'2022)*. Sept. 18–20, 2022, Halifax, NS, Canada. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE), 2022, pp. 16–20. ISBN: 978-1-6654-8432-9. doi:10.1109/CCECE49351.2022.9918302 (cit. on p. 354).

[39] Bo Chen, Chris N. Potts, and Gerhard J. Woeginger. "A Review of Machine Scheduling: Complexity, Algorithms and Approximability". In: *Handbook of Combinatorial Optimization*. Ed. by Panos Miltiades Pardalos, Ding-Zhu Du, and Ronald Lewis Graham. 1st ed. Boston, MA, USA: Springer, 1998, pp. 1493–1641. ISBN: 978-1-4613-7987-4. doi:10.1007/978-1-4613-0303-9_25. See also pages 21–169 in volume 3/3 by Norwell, MA, USA: Kluwer Academic Publishers. Discusses, amongst others, that the JSSP is $\mathcal{NP}$-hard. (Cit. on pp. 346, 356).

[40] Stephen Chen, James Montgomery, and Antonio Bolufé-Röhler. "Measuring the Curse of Dimensionality and its Effects on Particle Swarm Optimization and Differential Evolution". *Applied Intelligence – The International Journal of Research on Intelligent Systems for Real Life Complex Problems* 42:514–526, Apr. 2015. London, England, UK: Springer Nature Limited. ISSN: 0924-669X. doi:10.1007/s10489-014-0613-2 (cit. on p. 356).

# References VII


[41] Jiayang Chen (陈嘉阳), Zhize Wu (吴志泽), Sarah Louise Thomson, and Thomas Weise (汤卫思). "Frequency Fitness Assignment: Optimization Without Bias for Good Solution Outperforms Randomized Local Search on the Quadratic Assignment Problem". In: *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. Nov. 20–22, 2024, Porto, Portugal. Ed. by Francesco Marcelloni, Kurosh Madani, Niki van Stein, and Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024, pp. 27–37. ISSN: 2184-3236. ISBN: 978-989-758-721-4. doi:10.5220/0012888600003837 (cit. on pp. 344, 348, 350).

[42] T. C. Edwin Cheng, Bo Peng, and Zhipeng Lü. "A Hybrid Evolutionary Algorithm to Solve the Job Shop Scheduling Problem". *Annals of Operations Research* 242(2):223–237, July 2016. London, England, UK: Springer Nature Limited. ISSN: 0254-5330. doi:10.1007/s10479-013-1332-5. A MA utlizing TS (cit. on p. 347).

[43] Philippe Chrétienne, Edward G. Coffman, Jan Karel Lenstra, and Zhen Liu, eds. *Scheduling Theory and Its Applications*. Chichester, West Sussex, England, UK: John Wiley and Sons Ltd., Sept. 1995. ISBN: 978-0-471-94059-3 (cit. on p. 346).

[44] Nestor M. Cid-Garcia and Yasmin A. Rios-Solis. "Positions and Covering: A Two-Stage Methodology to Obtain Optimal Solutions for the 2D-Bin Packing Problem". *PLoS ONE* 15(4):e0229358, Apr. 2020. San Francisco, CA, USA: Public Library of Science (PLOS). ISSN: 1932-6203. doi:10.1371/journal.pone.0229358. An exact method for the two-dimensional Bin Packing Problem with free cuts and rotation (2BP|R|F) and the 2BP|O|F. (Cit. on p. 341).

[45] Stephen Arthur Cook. "The Complexity of Theorem-Proving Procedures". In: *Third Annual ACM Symposium on Theory of Computing (STOC'1971)*. May 3–5, 1971, Shaker Heights, OH, USA. Ed. by Michael A. Harrison, Ranan B. Banerji, and Jeffrey D. Ullman. New York, NY, USA: Association for Computing Machinery (ACM), 1971, pp. 151–158. ISBN: 978-1-4503-7464-4. doi:10.1145/800157.805047 (cit. on pp. 348, 356).

[46] William John Cook. *Concorde TSP Solver*. Waterloo, ON, Canada: University of Waterloo, 1997. URL: https://math.uwaterloo.ca/tsp/concorde (visited on 2026-09-26) (cit. on p. 343).

[47] William John Cook. *In Pursuit of the Traveling Salesman: Mathematics at the Limits of Computation*. Princeton, NJ, USA: Princeton University Press, 2012–Nov. 9, 2014. ISBN: 978-0-691-16352-9 (cit. on pp. 343, 347).

[48] William John Cook. *World TSP*. Waterloo, ON, Canada: University of Waterloo, 2013–Oct. 25, 2025. URL: http://www.math.uwaterloo.ca/tsp/world (visited on 2026-01-03) (cit. on pp. 290–295, 347, 352).

# References VIII


[49] William John Cook, Daniel G. Espinoza, and Marcos Goycoolea. *Computing with Domino-Parity Inequalities for the TSP*. Tech. rep. Atlanta, GA, USA: Georgia Institute of TechnologyIndustrial and Systems Engineering, 2005. URL: https://www.math.uwaterloo.ca/~bico/papers/DP_paper.pdf (visited on 2026-01-03). See also[49]. (Cit. on pp. 290–295, 313).

[50] Thomas H. Cormen, Charles E. Leiserson, Ronald Linn Rivest, and Clifford Stein. "Asymptotic Notation". In: *Introduction to Algorithms*. 3rd ed. Cambridge, MA, USA: MIT Press, 2009. Chap. 3.1, pp. 43–53. ISBN: 978-0-262-03384-8. Discusses $\Theta$, $\mathcal{O}$, $\Omega$, $o$, and $\omega$ (cit. on pp. 356, 357).

[51] Cemre Çubukçuoğlu, Pirouz Nourian, M. Fatih Tasgetiren, I. Sevil Sariyildiz, and Shervin Azadi. "Hospital Layout Design Renovation as a Quadratic Assignment Problem with Geodesic Distances". *Journal of Building Engineering* 44:102952, Dec. 2021. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 2352-7102. doi:10.1016/j.jobe.2021.102952 (cit. on p. 350).

[52] *Definition of Operations Research*. University of Western Ontario, London, ON, Canada: International Federation of Operational Research Societies (IFORS), 2020. URL: https://www.ifors.org/what-is-or (visited on 2026-01-01) (cit. on p. 349).

[53] Paul Deitel, Harvey Deitel, and Abbey Deitel. *Internet & World Wide Web: How to Program*. 5th ed. Hoboken, NJ, USA: Pearson Education, Inc., Nov. 2011. ISBN: 978-0-13-299045-5 (cit. on p. 354).

[54] Maxence Delorme, Manuel Iori, and Silvano Martello. "Bin Packing and Cutting Stock Problems: Mathematical Models and Exact Algorithms". *European Journal of Operational Research* 255(1):1–20, Nov. 2016. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/j.ejor.2016.04.030 (cit. on p. 342).

[55] Ebru Demirkol, Sanjay V. Mehta, and Reha Uzsoy. "Benchmarks for Shop Scheduling Problems". *European Journal of Operational Research* 109(1):137–141, Aug. 1998. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/S0377-2217(97)00019-2 (cit. on p. 346).

[56] Miroslav Dimitrov. *New Classes of Binary Sequences with High Merit Factor*. arXiv.org: Computing Research Repository (CoRR). Ithaca, NY, USA: Cornell Universiy Library, June 24, 2022. doi:10.48550/arXiv.2206.12070. URL: https://arxiv.org/pdf/2206.12070 (visited on 2026-09-21). arXiv:2206.12070v1 [cs.IT] 24 Jun 2022 (cit. on p. 347).

# References IX


[57] Miroslav Dimitrov, Tsonka Baitcheva, and Nikolay Nikolov. "Efficient Generation of Low Autocorrelation Binary Sequences". *IEEE Signal Processing Letters* 27:341–345, 2020. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: 1558-2361. doi:10.1109/LSP.2020.2972127 (cit. on p. 347).

[58] Slobodan Dmitrović. *Modern C for Absolute Beginners: A Friendly Introduction to the C Programming Language*. New York, NY, USA: Apress Media, LLC, Mar. 2024. ISBN: 979-8-8688-0224-9 (cit. on p. 343).

[59] Benjamin Doerr, Carola Doerr, and Franziska Ebel. "From Black-Box Complexity to Designing New Genetic Algorithms". *Theoretical Computer Science* 567:87–104, Feb. 2015. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0304-3975. doi:10.1016/J.TCS.2014.11.028 (cit. on p. 349).

[60] Carola Doerr, Furong Ye, Naama Horesh, Hao Wang, and Ofer M. ShirThomas Bäck. "Benchmarking Discrete Optimization Heuristics with IOHprofiler". In: *21st Genetic and Evolutionary Computation Conference Companion (GECCO'2019)*. July 13–17, 2019, Prague, Czech Republic. Ed. by Manuel López-Ibáñez, Anne Auger, and Thomas Stützle. New York, NY, USA: Association for Computing Machinery (ACM), 2019, pp. 1798–1806. ISBN: 978-1-4503-6748-6. doi:10.1145/3319619.3326810. See also[61,62] (cit. on p. 314).

[61] Carola Doerr, Furong Ye, Naama Horesh, Hao Wang, and Ofer M. ShirThomas Bäck. "Benchmarking Discrete Optimization Heuristics with IOHprofiler". *Applied Soft Computing (ASOC)* 88:106027, Mar. 2020. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 1568-4946. doi:10.1016/j.asoc.2019.106027. See also[60,62] (cit. on pp. 314, 340, 345, 346, 348).

[62] Carola Doerr, Furong Ye, Naama Horesh, Hao Wang, and Ofer M. ShirThomas Bäck. "Benchmarking Discrete Optimization Heuristics with IOHprofiler". (abs/1912.09237), Jan. 31, 2021. doi:10.48550/arXiv.1912.09237. URL: https://arxiv.org/abs/1912.09237 (visited on 2024-06-27). arXiv:1912.09237v2 [cs.NE] 31 Jan 2021, see also[60,61] (cit. on p. 314).

[63] Marco Dorigo, Mauro Birattari, and Thomas Stützle. "Ant Colony Optimization". *IEEE Computational Intelligence Magazine (CIM)* 1(4):28–39, Nov. 2006. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: 1556-603X. doi:10.1109/MCI.2006.329691. URL: https://www.researchgate.net/publication/308953674 (visited on 2025-07-25) (cit. on p. 341).

[64] Marco Dorigo, Vittorio Maniezzo, and Alberto Colorni. "Ant System: Optimization by a Colony of Cooperating Agents". *IEEE Transactions on Systems, Man, and Cybernetics – Part B: Cybernetics* 26(1):29–41, Feb. 1996. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: 1083-4419. doi:10.1109/3477.484436. URL: https://www.researchgate.net/publication/5589170 (visited on 2025-07-26) (cit. on p. 341).

# References X


[65] Marco Dorigo and Thomas Stützle. *Ant Colony Optimization*. Cambridge, MA, USA: MIT Press, 2004. ISBN: 978-0-262-04219-2 (cit. on p. 341).

[66] Johann Dréo, Alain Pétrowski, Patrick Siarry, and Éric D. Taillard. *Metaheuristics for Hard Optimization – Methods and Case Studies*. Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany, 2006. ISBN: 978-3-540-23022-9. doi:10.1007/3-540-30966-7. Discusses SA, TS, EAs, and Ant Colony Optimization (ACO). (Cit. on p. 350).

[67] Stefan Droste, Thomas Jansen, and Ingo Wegener. "On the Analysis of the $(1+1)$ Evolutionary Algorithm". *Theoretical Computer Science* 276(1-2):51–81, Apr. 2002. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0304-3975. doi:10.1016/S0304-3975(01)00182-7 (cit. on pp. 340, 346, 347, 351).

[68] Stefan Droste, Thomas Jansen, and Ingo Wegener. "Upper and Lower Bounds for Randomized Search Heuristics in Black-Box Optimization". *Theoretical Computer Science* 39(4):525–544, July 2006. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0304-3975. doi:10.1007/S00224-004-1177-Z (cit. on p. 349).

[69] Jacques Dutka. "The Early History of the Factorial Function". *Archive for History of Exact Sciences* 43(3):225–249, Sept. 1991. Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany. ISSN: 0003-9519. doi:10.1007/BF00389433. Communicated by Umberto Bottazzini (cit. on p. 354).

[70] Harald Dyckhoff and Ute Finke. *Cutting and Packing in Production and Distribution: A Typology and Bibliography*. Contributions to Management Science. Heidelberg, Baden-Württemberg, Germany: Physica-Verlag, 1992. ISSN: 1431-1941. ISBN: 978-3-642-63487-1. doi:10.1007/978-3-642-58165-6 (cit. on pp. 8–13, 342).

[71] Kelly Easton, George L. Nemhauser, and Michael A. Trick. "The Traveling Tournament Problem Description and Benchmarks". In: *7th International Conference on Principles and Practice of Constraint Programming (CP'01)*. Nov. 26–Dec. 1, 2001, Paphos, Cyprus. Ed. by Toby Walsh. Vol. 2239 of Lecture Notes in Computer Science (LNCS). Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany, 2001, pp. 580–584. ISSN: 0302-9743. ISBN: 978-3-540-42863-3. doi:10.1007/3-540-45578-7_43 (cit. on p. 353).

[72] Alwalid N. Elshafei. "Hospital Layout as a Quadratic Assignment Problem". *The Journal of the Operational Research Society (JORS)* 28(1):167–179, Apr. 1977. London, England, UK: Taylor and Francis Ltd. and London, England, UK: Springer Nature Limited. ISSN: 0160-5682. doi:10.1057/jors.1977.29 (cit. on p. 350).

# References XI


[73] Bernhard Eschermann and Hans-Joachim Wunderlich. "Optimized Synthesis of Self-Testable Finite State Machines". In: *20th International Symposium Fault-Tolerant Computing (FTCS'1990): Digest of Papers*. June 26–28, 1990, Newcastle Upon Tyne, England, UK. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE), 1990. ISBN: 978-0-8186-2051-5. doi:10.1109/FTCS.1990.89393 (cit. on p. 350).

[74] Thomas A. Feo and Mauricio G. C. Resende. "Greedy Randomized Adaptive Search Procedures". *Journal of Global Optimization* 6(2):109–133, Mar. 1995. London, England, UK: Springer Nature Limited. ISSN: 0925-5001. doi:10.1007/BF01096763. Introduces Greedy Randomized Adaptive Search Procedures (GRASPs) and applies them to the set covering problem, among others, and lists many problem domains. (Cit. on p. 344).

[75] Luca Ferrari and Enrico Pirozzi. *Learn PostgreSQL*. 2nd ed. Birmingham, England, UK: Packt Publishing Ltd, Oct. 2023. ISBN: 978-1-83763-564-1 (cit. on p. 349).

[76] Paola Festa and Mauricio G. C. Resende. "An Annotated Bibliography of GRASP—Part I: Algorithms". *International Transactions on Operational Research (ITOR)* 16(1):1–24, Jan. 2009. Chichester, West Sussex, England, UK: John Wiley and Sons Ltd. ISSN: 0969-6016. doi:10.1111/j.1475-3995.2009.00663.x. See also[77] (cit. on pp. 316, 344).

[77] Paola Festa and Mauricio G. C. Resende. "An Annotated Bibliography of GRASP—Part II: Applications". *International Transactions on Operational Research (ITOR)* 16(2):131–172, Mar. 2009. Chichester, West Sussex, England, UK: John Wiley and Sons Ltd. ISSN: 0969-6016. doi:10.1111/j.1475-3995.2009.00664.x. See also[76] (cit. on pp. 316, 344).

[78] Simon Fischer. "A Polynomial Upper Bound for a Mutation-Based Algorithm on the Two-Dimensional Ising Model". In: *6th Genetic and Evolutionary Computation Conference (GECCO'2004), Part I*. June 26–30, 2004, Seattle, WA, USA. Ed. by Kalyanmoy Deb, Riccardo Poli, Wolfgang Banzhaf, Hans-Georg Beyer, Edmund K. Burke, Paul J. Darwen, Dipankar Dasgupta, Dario Floreano, James A. Foster, Mark Harman, Owen Holland, Pier Luca Lanzi, Lee Spector, Andrea Tettamanzi, Dirk Thierens, and Andrew M. Tyrrell. Vol. 3102 of Lecture Notes in Computer Science (LNCS). Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany, 2004, pp. 1100–1112. ISSN: 0302-9743. ISBN: 978-3-540-22344-3. doi:10.1007/978-3-540-24854-5_108 (cit. on pp. 345, 346).

# References XII


[79] Simon Fischer and Ingo Wegener. "The One-Dimensional Ising Model: Mutation versus Recombination". *Theoretical Computer Science* 344(2-3):208–225, Nov. 2005. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0304-3975. doi:10.1016/J.TCS.2005.04.002 (cit. on p. 345).

[80] Henry Fisher and Gerald L. Thompson. "Probabilistic Learning Combinations of Local Job-Shop Scheduling Rules". In: *Industrial Scheduling*. Ed. by John F. Muth and Gerald L. Thompson. Englewood Cliffs, NJ, USA: Prentice-Hall, 1963. Chap. 3.2, pp. 225–251 (cit. on p. 346).

[81] Charles Fleurent and Jacques A. Ferland. "Genetic Hybrids for the Quadratic Assignment Problem". In: *Quadratic Assignment and Related Problems, Proceedings of a DIMACS Workshop*. May 20–21, 1993, New Brunswick, NJ, USA: Rutgers University. Ed. by Panos Miltiades Pardalos and Henry Wolkowicz. Providence, RI, USA: American Mathematical Society (AMS), 1994, pp. 173–187. ISBN: 978-0-8218-6607-8. A MA using the TS from[195] as local search performs well. (Cit. on pp. 347, 350).

[82] Tobias Friedrich, Pietro Simone Oliveto, Dirk Sudholt, and Carsten Witt. "Analysis of Diversity-Preserving Mechanisms for Global Exploration". *Evolutionary Computation* 17(4):455–476, Win. 2009. Cambridge, MA, USA: MIT Press. ISSN: 1063-6560. doi:10.1162/evco.2009.17.4.17401 (cit. on p. 353).

[83] Tobias Friedrich, Francesco Quinzan, and Markus Wagner. "Escaping Large Deceptive Basins of Attraction with Heavy-Tailed Mutation Operators". In: *20th Genetic and Evolutionary Computation Conference (GECCO'2018)*. July 15–19, 2018, Kyōto, Kyōto-fu, Japan. Ed. by Hernán E. Aguirre and Keiki Takadama. New York, NY, USA: Association for Computing Machinery (ACM), 2018, pp. 293–300. ISBN: 978-1-4503-5618-3. doi:10.1145/3205455.3205515 (cit. on pp. 346, 353).

[84] José E. Gallardo, Carlos Cotta, and Antonio J. Fernández. "A Memetic Algorithm for the Low Autocorrelation Binary Sequence Problem". In: *9th Genetic and Evolutionary Computation Conference (GECCO'2007)*. July 7–11, 2007, London, England, UK. Ed. by Hod Lipson. New York, NY, USA: Association for Computing Machinery (ACM), 2007, pp. 1226–1233. ISBN: 978-1-59593-697-4. doi:10.1145/1276958.1277195 (cit. on p. 347).

[85] Michael R. Garey and David Stifler Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. New York, NY, USA: W. H. Freeman and Company, 1979. ISBN: 978-0-7167-1045-5 (cit. on p. 352).

# References XIII


[86] Mitsuo Gen, Yasuhiro Tsujimura, and Erika Kubota. "Solving Job-Shop Scheduling Problems by Genetic Algorithm". In: *Humans, Information and Technology: Proceedings of the 1994 IEEE International Conference on Systems, Man and Cybernetics (SMC'1994)*. Vol. 2. Oct. 2–5, 1994, San Antonio, TX, USA. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE), 1994, pp. 1577–1582. ISBN: 978-0-7803-2129-8. doi:10.1109/ICSMC.1994.400072. Proposes a permutations-with-repetitions based encoding for the JSSP. (Cit. on pp. 318, 346, 355).

[87] Fred Glover. "Tabu Search – Part I". *ORSA Journal on Computing* 1(3):190–206, Sum. 1989. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0899-1499. doi:10.1287/ijoc.1.3.190 (cit. on p. 352).

[88] Fred Glover. "Tabu Search – Part II". *ORSA Journal on Computing* 2(1):4–32, Win. 1990. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0899-1499. doi:10.1287/ijoc.2.1.4 (cit. on p. 352).

[89] Fred Glover and Gary A. Kochenberger, eds. *Handbook of Metaheuristics*. Vol. 57 of International Series in Operations Research & Management Science (ISOR). Cham, Switzerland: Springer, 2003. ISSN: 0884-8289. ISBN: 978-0-306-48056-0. doi:10.1007/b101874 (cit. on p. 348).

[90] Fred Glover, Éric D. Taillard, and Dominique de Werra. "A User's Guide to Tabu Search". *Annals of Operations Research* 41(1):3–28, Mar. 1993. London, England, UK: Springer Nature Limited. ISSN: 0254-5330. doi:10.1007/BF02078647. Special issue on TS. (Cit. on p. 352).

[91] José Fernando Gonçalves and Mauricio G. C. Resende. "A Biased Random Key Genetic Algorithm for two-dimensional Bin Packing Problem (2BP) and 3D Bin Packing Problems". *International Journal of Production Economics (IJPE)* 145(2):500–510, Oct. 2013. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0925-5273. doi:10.1016/j.ijpe.2013.04.019. Applies the Biased Random Key Genetic Algorithm (BRKGA) to the 2BP|R|F and 2BP|O|F as well as 3D packing. Continuous vectors are translated to permutations and orientation data which is then the input of a placement heuristic. (Cit. on pp. 341, 343).

[92] José Fernando Gonçalves and Mauricio G. C. Resende. "An Extended Akers Graphical Method with a Biased Random-Key Genetic Algorithm for Job-Shop Scheduling". *International Transactions on Operational Research (ITOR)* 21(2):215–246, Mar. 2014. Chichester, West Sussex, England, UK: John Wiley and Sons Ltd. ISSN: 0969-6016. doi:10.1111/itor.12044. URL: http://mauricio.resende.info/doc/brkga-jss2011.pdf (visited on 2026-09-30). The BRKGA basically is a MA using a continuous encoding of permutations that are decoded to schedules using the method from[23,24,86,185] and then refined via a TS. (Cit. on pp. 343, 346).

# References XIV


[93] José Fernando Gonçalves and Mauricio G. C. Resende. "Biased Random-Key Genetic Algorithms for Combinatorial Optimization". *Journal of Heuristics* 17:487–525, Oct. 2010. London, England, UK: Springer Nature Limited. ISSN: 1381-1231. doi:10.1007/s10732-010-9143-1 (cit. on p. 343).

[94] Michael T. Goodrich. *A Gentle Introduction to NP-Completeness*. Irvine, CA, USA: University of California, Irvine, Apr. 2022. URL: https://ics.uci.edu/~goodrich/teach/cs165/notes/NPComplete.pdf (visited on 2025-08-01) (cit. on p. 356).

[95] Ronald Lewis Graham, Eugene Leighton Lawler, Jan Karel Lenstra, and Alexander Hendrik George Rinnooy Kan. "Optimization and Approximation in Deterministic Sequencing and Scheduling: A Survey". Annals of Discrete Mathematics (5):287–326. Ed. by Peter Ladislaw Hammer, Ellis Lane Johnson, and Bernhard H. Korte, 1979. ISSN: 0167-5060. doi:10.1016/S0167-5060(08)70356-X. An early review on JSSPs and FSSPs. (Cit. on pp. 344, 346).

[96] Martin Grötschel, Michael Jünger, and Gerhard Reinelt. "Optimal Control of Plotting and Drilling Machines: A Case Study". *Zeitschrift für Operations Research (ZOR). now Mathematical Methods of Operations Research (MMOR)* 35(1):61–84, Jan. 1991. Heidelberg, Baden-Württemberg, Germany: Physica-Verlag; now London, England, UK: Springer Nature Limited. ISSN: 0340-9422. doi:10.1007/BF01415960. This paper tackles circuit board plotting and drilling problems by modelling (parts of) them as symmetric Traveling Salesperson Problems (STSPs). (Cit. on p. 352).

[97] Gregory Z. Gutin and Abraham P. Punnen, eds. *The Traveling Salesman Problem and its Variations*. Vol. 12 of Combinatorial Optimization (COOP). New York, NY, USA: Springer New York, May 2002. ISSN: 1388-3011. doi:10.1007/b101971 (cit. on pp. 160–170, 352).

[98] Pierre Hansen, Nenad Mladenović, and José Andrés Moreno Pérez. "Variable Neighbourhood Search: Methods and Applications. Invited Survey". *4OR* 6(4):319–360, Dec. 2008. ISSN: 1619-4500. doi:10.1007/s10288-008-0089-1. URL: https://www.researchgate.net/publication/225155945 (visited on 2026-10-02) (cit. on p. 354).

[99] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. "A Formal Basis for the Heuristic Determination of Minimum Cost Paths". *IEEE Transactions on Systems Science and Cybernetics* 4(2):100–107, July 31, 1968. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: 0536-1567. doi:10.1109/TSSC.1968.300136 (cit. on pp. 122–127, 341).

# References XV


[100] Keld Helsgaun. "General $k$-opt Submoves for the Lin–Kernighan TSP Heuristic". *Mathematical Programming Computation* 1(2-3):119–163, July 2009. London, England, UK: Springer Nature Limited. ISSN: 1867-2949. doi:10.1007/s12532-009-0004-6. URL: https://www2.seas.gwu.edu/~simhaweb/champalg/tsp/papers/Helsgaun2.pdf (visited on 2026-09-26) (cit. on pp. 290–295, 347).

[101] Keld Helsgaun. "An Effective Implementation of the Lin-Kernighan Traveling Salesman Heuristic". 126(1):106–130, Oct. 2000. doi:10.1016/S0377-2217(99)00284-2. URL: http://akira.ruc.dk/~keld/research/LKH/LKH-2.0/DOC/LKH_REPORT.pdf (visited on 2026-09-26) (cit. on pp. 290–295, 347).

[102] André Henning. "Praktische Job-Shop Scheduling-Probleme". PhD thesis. Jena, Thüringen, Germany: Friedrich-Schiller-Universität Jena, Aug. 2002. URL: http://www.db-thueringen.de/servlets/DocumentServlet?id=873 (visited on 2026-09-30). Alternate url: https://nbn-resolving.org/urn:nbn:de:gbv:27-20060809-115700-4. Applies MA-like algorithms to the JSSP. (Cit. on pp. 346, 347).

[103] Holger H. Hoos and Thomas Stützle. *Stochastic Local Search: Foundations & Applications*. The Morgan Kaufmann Series in Artificial Intelligence. Amsterdam, The Netherlands: Elsevier B.V., 2005. ISBN: 978-1-55860-872-6. doi:10.1016/B978-1-55860-872-6.X5016-1 (cit. on pp. 348, 351).

[104] Holger H. Hoos and Thomas Stützle. *SATLIB – The Satisfiability Library*. 1.4.4. Vancouver, BC, Canada: The University of British Columbia (UBC), May 11, 2001. URL: https://www.cs.ubc.ca/~hoos/SATLIB/index-ubc.html (visited on 2026-09-20). See paper[105] (cit. on pp. 320, 351).

[105] Holger H. Hoos and Thomas Stützle. "SATLIB: An Online Resource for Research on SAT". In: *SAT2000 – Highlights of Satisfiability Research in the Year 2000*. Ed. by Ian Philip Gent, Toby Walsh, and Hans van Maaren. Vol. 63 of Frontiers in Artificial Intelligence and Applications. Amsterdam, The Netherlands: IOS Press BV, Jan. 2000, pp. 283–292. ISSN: 0922-6389. ISBN: 978-1-58603-061-2. See the website[104] (cit. on pp. 320, 351).

[106] ."Why is the Complexity of A* Exponential in Memory?" In: *Stack Overflow*. Ed. by Paul. New York, NY, USA: Stack Exchange Inc., Nov. 11, 2009–July 17, 2013. URL: https://stackoverflow.com/questions/1715401 (visited on 2026-01-03) (cit. on pp. 194–203).

[107] John Hunt. *A Beginners Guide to Python 3 Programming*. 2nd ed. Undergraduate Topics in Computer Science (UTICS). Cham, Switzerland: Springer, 2023. ISSN: 1863-7310. ISBN: 978-3-031-35121-1. doi:10.1007/978-3-031-35122-8 (cit. on p. 349).

# References XVI


[108] *Information Technology – Universal Coded Character Set (UCS)*. International Standard ISO/IEC 10646:2020. Geneva, Switzerland: International Organization for Standardization (ISO) and International Electrotechnical Commission (IEC), Dec. 2020 (cit. on p. 338).

[109] Manuel Iori, Vinícius Loti de Lima, Silvano Martello, Flávio K. Miyazaw, and Michele Monaci. "Exact Solution Techniques for Two-Dimensional Cutting and Packing". *European Journal of Operational Research* 289(2):399–415, Mar. 2021. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/j.ejor.2020.06.050. See also arXiv:2004.12619v1 [math.OC] 27 Apr 2020 at doi:10.1016/j.ejor.2020.06.050. (Cit. on p. 341).

[110] Christoph Jabs, Dieter Vandesande, Jeremias Berg, Johannes Fichte, George Katsirelos, Ruben Martins, André Schidler, and Dominik Schreiber. *MaxSAT Evaluation: Complete Benchmark Collection*. 2026th ed. Geneva, Switzerland: zenodo (hosted by European Organization for Nuclear Research (CERN)), July 9, 2026. doi:10.5281/zenodo.21283148. All versions of this dataset can be reached directly via doi:10.5281/zenodo.21283147. See also https://github.com/maxsat-evaluations/data and https://maxsat-evaluations.github.io. (Cit. on p. 348).

[111] Dean Jacobs, Jan Prins, Peter Siegel, and Kenneth Wilson. "Monte Carlo Techniques in Code Optimization". In: *15th Annual Workshop on Microprogramming (MICRO 15)*. Oct. 5–7, 1982. Ed. by Joseph Allen Fisher, William J. Tracz, and Bill Hopkins. Palo Alto, CA, USA: Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE) and New York, NY, USA: Association for Computing Machinery (ACM), Oct. 1982, pp. 143–148. doi:10.5555/800036.800944. See [112] (cit. on p. 321).

[112] Dean Jacobs, Jan Prins, Peter Siegel, and Kenneth Wilson. "Monte Carlo Techniques in Code Optimization". *ACM SIGMICRO Newsletter* 13(4):143–148, Dec. 1982. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 1050-916X. doi:10.1145/1014194.800944. See [111] (cit. on pp. 321, 351).

[113] Stefan Jakobs. "On Genetic Algorithms for the Packing of Polygons". *European Journal of Operational Research* 88(1):165–181, Jan. 1996. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/0377-2217(94)00166-9 (cit. on p. 345).

[114] Wassim Jaziri, ed. *Tabu Search*. London, England, UK: IntechOpen Limited, Aug. 1, 2008. ISBN: 978-3-902613-34-9. doi:10.5772/65. URL: https://www.intechopen.com/books/6114 (visited on 2026-09-30) (cit. on p. 352).

# References XVII


[115] David Stifler Johnson, Gregory Z. Gutin, Lyle Andrew McGeoch, Anders Yeo, Weixiong Zhang, and Alexei Zverovitch. *Experimental Analysis of Heuristics for the ATSP*. Ed. by Gregory Z. Gutin and Abraham P. Punnen. New York, NY, USA, May 2002. doi:10.1007/0-306-48213-4_10. URL: https://archive.dimacs.rutgers.edu/Challenges/TSP/papers/atspchap.pdf (visited on 2026-09-25) (cit. on p. 352).

[116] David Stifler Johnson and Lyle Andrew McGeoch. *Experimental Analysis of Heuristics for the STSP*. Ed. by Gregory Z. Gutin and Abraham P. Punnen. New York, NY, USA, May 2002. doi:10.1007/0-306-48213-4_9. URL: https://archive.dimacs.rutgers.edu/Challenges/TSP/papers/stspchap.pdf (visited on 2026-09-25) (cit. on p. 352).

[117] David Stifler Johnson, Lyle Andrew McGeoch, Fred Glover, and Cesar Rego. *8th DIMACS Implementation Challenge: The Traveling Salesman Problem*. June 1, 2000–Nov. 19, 2013. URL: https://archive.dimacs.rutgers.edu/Challenges/TSP (visited on 2026-09-25) (cit. on p. 352).

[118] Selmer Martin Johnson. "Optimal Two- and Three-Stage Production Schedules with Setup Times Included". *Naval Research Logistics Quarterly* 1(1):61–68, Mar. 1954. Chichester, West Sussex, England, UK: John Wiley and Sons Ltd. ISSN: 0894-069X. doi:10.1002/nav.3800010110. URL: https://www.rand.org/content/dam/rand/pubs/papers/2008/P402.pdf (visited on 2023-12-06) (cit. on p. 344).

[119] Scott Kirkpatrick, C. Daniel Gelatt, Jr., and Mario P. Vecchi. "Optimization by Simulated Annealing". *Science Magazine* 220(4598):671–680, May 13, 1983. Washington, D.C., USA: American Association for the Advancement of Science (AAAS). ISSN: 0036-8075. doi:10.1126/science.220.4598.671. URL: https://www.researchgate.net/publication/6026283 (visited on 2025-08-08) (cit. on p. 351).

[120] Robert Klein. *Scheduling of Resource-Constrained Projects*. Vol. 10 of Operations Research/Computer Science Interfaces Series (ORCS). New York, NY, USA: Springer New York, 2000. ISSN: 1387-666X. ISBN: 978-0-7923-8637-7. doi:10.1007/978-1-4615-4629-0 (cit. on p. 344).

[121] Donald Ervin Knuth. "Big Omicron and Big Omega and Big Theta". *ACM SIGACT News* 8(2):18–24, Apr.–June 1976. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 0163-5700. doi:10.1145/1008328.1008329 (cit. on pp. 356, 357).

[122] Donald Ervin Knuth. *Fundamental Algorithms*. 3rd ed. Vol. 1 of The Art of Computer Programming. Reading, MA, USA: Addison-Wesley Professional, 1997. ISBN: 978-0-201-89683-1 (cit. on pp. 356, 357).

# References XVIII


[123] Donald Ervin Knuth. *Sorting and Searching*. Vol. 3 of The Art of Computer Programming. Reading, MA, USA: Addison-Wesley Professional, 1998. ISBN: 978-0-201-89685-5 (cit. on p. 356).

[124] Tjalling Charles Koopmans and Martin Beckmann. "Assignment Problems and the Location of Economic Activities". *Econometrica* 25(1):53–76, 1957. New Haven, CT, USA: The Econometric Society and Chichester, West Sussex, England, UK: John Wiley and Sons Ltd. ISSN: 0012-9682. doi:10.2307/1907742 (cit. on p. 350).

[125] Jakob Krarup and Peter Mark Pruzan. "Computer-aided Layout Design". In: *Mathematical Programming in Use*. Ed. by Michel Louis Balinski and Claude Lemaréchal. Vol. 9 of. Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany, 1978. Chap. 6, pp. 75–94. ISSN: 0303-3929. ISBN: 978-3-642-00796-5. doi:10.1007/BFb0120827. Uses QAPs as model for designing architectural layouts. (Cit. on p. 350).

[126] Mark W. Krentel. "The Complexity of Optimization Problems". *Journal of Computer and System Sciences* 36(3):490–509, June 1988. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0022-0000. Explores the hardness (optimization) problems such TSP, Clique, and Coloring, where computing the optimal value is $\mathcal{NP}$-complete. (Cit. on p. 352).

[127] Edmund Landau. *Handbuch der Lehre von der Verteilung der Primzahlen*. Leipzig, Sachsen, Germany: B. G. Teubner, 1909. ISBN: 978-0-8218-2650-8 (cit. on p. 356).

[128] Eugene Leighton Lawler, Jan Karel Lenstra, and Alexander Hendrik George Rinnooy Kan. "Recent Developments in Deterministic Sequencing and Scheduling: A Survey". In: *Deterministic and Stochastic Scheduling: Proceedings of the NATO Advanced Study and Research Institute on Theoretical Approaches to Scheduling Problems*. June 6–17, 1981, Durham, England, UK. Ed. by Michael Anthony Howe Dempster, Jan Karel Lenstra, and Alexander Hendrik George Rinnooy Kan. Vol. 84 of Nato Science Series C: Mathematical and Physical Science (ASIC). London, England, UK: Springer Nature Limited, Apr. 1982, pp. 35–73. ISBN: 978-90-277-1397-1. doi:10.1007/978-94-009-7801-0_3. The JSSP is $\mathcal{NP}$-hard. (Cit. on p. 346).

# References XIX


[129] Eugene Leighton Lawler, Jan Karel Lenstra, Alexander Hendrik George Rinnooy Kan, and David B. Shmoys. "Sequencing and Scheduling: Algorithms and Complexity". In: *Production Planning and Inventory*. Ed. by Stephen C. Graves, Alexander Hendrik George Rinnooy Kan, and Paul H. Zipkin. Vol. IV of Handbooks of Operations Research and Management Science. Amsterdam, The Netherlands: Elsevier B.V., 1993. Chap. 9, pp. 445–522. ISSN: 0927-0507. ISBN: 978-0-444-87472-6. doi:10.1016/S0927-0507(05)80189-6. URL: http://alexandria.tue.nl/repository/books/339776.pdf (visited on 2023-12-06). Discusses, amongst others, that the JSSP is $\mathcal{NP}$-hard. (Cit. on pp. 346, 356).

[130] Eugene Leighton Lawler, Jan Karel Lenstra, Alexander Hendrik George Rinnooy Kan, and David B. Shmoys. *The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization*. Estimation, Simulation, and Control – Wiley-Interscience Series in Discrete Mathematics and Optimization. Chichester, West Sussex, England, UK: Wiley Interscience, Sept. 1985. ISSN: 0277-2698. ISBN: 978-0-471-90413-7 (cit. on pp. 136, 160–170, 352).

[131] Stephen R. Lawrence. "Resource Constrained Project Scheduling: An Experimental Investigation of Heuristic Scheduling Techniques". PhD thesis. Pittsburgh, PA, USA: Graduate School of Industrial Administration (GSIA), Carnegie-Mellon University, 1984 (cit. on p. 346).

[132] Kent D. Lee and Steve Hubbard. *Data Structures and Algorithms with Python*. Undergraduate Topics in Computer Science (UTICS). Cham, Switzerland: Springer, 2015. ISSN: 1863-7310. ISBN: 978-3-319-13071-2. doi:10.1007/978-3-319-13072-9 (cit. on p. 349).

[133] Xueping Li and Kaike Zhang. "Single Batch Processing Machine Scheduling with Two-Dimensional Bin Packing Constraints". *International Journal of Production Economics (IJPE)* 196:113–121, Feb. 2018. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0925-5273. doi:10.1016/j.ijpe.2017.11.015. Mentions some scheduling tasks of Additive Manufacturing jobs can be viewed as instance of the 2BP|R|F. (Cit. on p. 341).

[134] Tianyu Liang (梁天宇), Zhize Wu (吴志泽), Jörg Lässig, Daan van den Berg, Sarah Louise Thomson, and Thomas Weise (汤卫思). "Addressing the Traveling Salesperson Problem with Frequency Fitness Assignment and Hybrid Algorithms". *Soft Computing* 28(17-18):9495–9508, July 2024. London, England, UK: Springer Nature Limited. ISSN: 1432-7643. doi:10.1007/S00500-024-09718-8 (cit. on pp. 160–170, 344, 348, 352).

# References XX


[135] Tianyu Liang (梁天宇), Zhize Wu (吴志泽), Jörg Lässig, Daan van den Berg, and Thomas Weise (汤卫思). "Solving the Traveling Salesperson Problem using Frequency Fitness Assignment". In: *IEEE Symposium Series on Computational Intelligence (SSCI'2022)*. Dec. 4–7, 2022, Singapore. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE), 2022. ISBN: 978-1-6654-8769-6. doi:10.1109/SSCI51031.2022.10022296 (cit. on pp. 344, 348).

[136] Tianyu Liang (梁天宇), Zhize Wu (吴志泽), Matthias Thürer, Markus Wagner, and Thomas Weise (汤卫思). "Generating Small Instances with Interesting Features for the Traveling Salesperson Problem". In: *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. Nov. 20–22, 2024, Porto, Portugal. Ed. by Francesco Marcelloni, Kurosh Madani, Niki van Stein, and Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024, pp. 173–180. ISSN: 2184-3236. ISBN: 978-989-758-721-4. doi:10.5220/0012888800003837 (cit. on p. 348).

[137] Shen Lin and Brian W. Kernighan. "An Effective Heuristic Algorithm for the Traveling-Salesman Problem". *Operations Research* 21(2):498–516, Mar.–Apr. 1973. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0030-364X. doi:10.1287/opre.21.2.498. URL: https://www.cs.princeton.edu/~bwk/btl.mirror/tsp.pdf (visited on 2026-09-26) (cit. on p. 347).

[138] John D. C. Little, Katta G. Murty, Dura W. Sweeny, and Caroline Karel. *An Algorithm for the Traveling Salesman Problem*. Sloan Working Papers 07-63. Massachusetts Institute of Technology (MIT): Massachusetts Institute of Technology (MIT), Sloan School of Management, Mar. 1963. URL: http://dspace.mit.edu/bitstream/handle/1721.1/46828/algorithmfortrav00litt.pdf (visited on 2026-01-03) (cit. on pp. 290–295, 342).

[139] Dequan Liu (刘德全) and Hongfei Teng (滕弘飞). "An Improved BL-Algorithm for EAGenetic Algorithm of the Orthogonal Packing of Rectangles". *European Journal of Operational Research* 112(2):413–420, Jan. 1999. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/S0377-2217(97)00437-2 (cit. on p. 345).

[140] Andrea Lodi, Silvano Martello, and Michele Monaci. "Two-Dimensional Packing Problems: A Survey". *European Journal of Operational Research* 141(2):241–252, Sept. 2002. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/S0377-2217(02)00123-6 (cit. on pp. 341, 342).

# References XXI


[141] Andrea Lodi, Silvano Martello, and Daniele Vigo. "Heuristic and Metaheuristic Approaches for a Class of Two-Dimensional Bin Packing Problems". *INFORMS Journal on Computing* 11(4):345–357, Aut. 1999. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 1091-9856. doi:10.1287/ijoc.11.4.345 (cit. on p. 341).

[142] Andrea Lodi, Silvano Martello, and Daniele Vigo. "Recent Advances on Two-Dimensional Bin Packing Problems". *Discrete Applied Mathematics* 123(1–3):379–396, Nov. 2002. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0166-218X. doi:10.1016/S0166-218X(01)00347-X (cit. on pp. 341, 342).

[143] Eliane Maria Loiola, Nair Maria Maia de Abreu, Paulo Oswaldo Boaventura-Netto, Peter M. Hahn, and Tania Querido. "A Survey for the Quadratic Assignment Problem". *European Journal of Operational Research* 176(2):657–690, 2007. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/j.ejor.2005.09.032 (cit. on p. 350).

[144] Mariana A. Londe, Luciana S. Pessoa, Carlos E. Andrade, and Mauricio G. C. Resende. *Biased Random-Key Genetic Algorithms: A Review*. arXiv.org: Computing Research Repository (CoRR) abs/2312.00961v2. Ithaca, NY, USA: Cornell Universiy Library, Dec. 6, 2023. doi:10.48550/arXiv.2312.00961. URL: https://arxiv.org/abs/2312.00961v2 (visited on 2024-06-27). arXiv:2312.00961v2 [cs.NE] 06 Dec 2023; See [145] (cit. on p. 326).

[145] Mariana A. Londe, Luciana S. Pessoa, Carlos E. Andrade, and Mauricio G. C. Resende. "Biased Random-Key Genetic Algorithms: A Review". *European Journal of Operational Research* 321(1):1–22, Feb. 2025. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/j.ejor.2024.03.030. See [144] (cit. on pp. 326, 343).

[146] Marc Loy, Patrick Niemeyer, and Daniel Leuck. *Learning Java*. 5th ed. Sebastopol, CA, USA: O'Reilly Media, Inc., Mar. 2020. ISBN: 978-1-4920-5627-0 (cit. on p. 346).

[147] Peter Luschny. *A New Kind of Factorial Function*. Highland Park, NJ, USA: The OEIS Foundation Inc., Oct. 4, 2015. URL: https://oeis.org/A000142/a000142.pdf (visited on 2024-09-29) (cit. on p. 354).

[148] Mark Lutz. *Learning Python*. 6th ed. Sebastopol, CA, USA: O'Reilly Media, Inc., Mar. 2025. ISBN: 978-1-0981-7130-8 (cit. on p. 349).

[149] Reinhard Männer and Bernard Manderick, eds. *Parallel Problem Solving from Nature 2 (PPSN-II)*. Sept. 28–30, 1992, Brussels, Belgium. Amsterdam, The Netherlands: Elsevier B.V., 1992.

# References XXII


[150] Francesco Marcelloni, Kurosh Madani, Niki van Stein, and Joaquim Filipe, eds. *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. Nov. 20–22, 2024, Porto, Portugal. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024. ISSN: 2184-3236. ISBN: 978-989-758-721-4. doi:10.5220/0000195000003837.

[151] Silvano Martello and Daniele Vigo. "Exact Solution of the Two-Dimensional Finite Bin Packing Problem". 44(3):388–399, Mar. 1998. doi:10.1287/mnsc.44.3.388 (cit. on pp. 341, 342).

[152] Stephan Mertens. "Exhaustive Search for Low-Autocorrelation Binary Sequences". *Journal of Physics A: Mathematical and General* 29(18):L473, Sept. 1996. Bristol, England, UK: IOP Publishing Ltd. ISSN: 0305-4470. doi:10.1088/0305-4470/29/18/005 (cit. on p. 347).

[153] Peter Merz and Bernd Freisleben. "A Comparison of MAMemetic Algorithms, TSTabu Search, and ACOAnt Colonies for the QAPQuadratic Assignment Problem". In: *Congress on Evolutionary Computation (CEC'1999)*. July 6–9, 1999, Washington, D.C., USA. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE), 1999, pp. 2063–2070. doi:10.1109/CEC.1999.785529 (cit. on p. 350).

[154] Burkhard Militzer, Michele Zamparelli, and Dieter Beule. "Evolutionary Search for Low Autocorrelated Binary Sequences". *IEEE Transactions on Evolutionary Computation* 2(1):34–39, Apr. 1998. Los Alamitos, CA, USA: IEEE Computer Society. ISSN: 1089-778X. doi:10.1109/4235.728212 (cit. on p. 347).

[155] Alfonsas Misevičius. "A Tabu Search Algorithm for the Quadratic Assignment Problem". *Computational Optimization and Applications* 30:95–111, Jan. 2005. London, England, UK: Springer Nature Limited. ISSN: 0926-6003. doi:10.1007/s10589-005-4562-x (cit. on p. 352).

[156] Alfonsas Misevičius. "A Modified Simulated Annealing Algorithm for the Quadratic Assignment Problem". *Informatica* 14(4):497–514, Jan. 2003. Vilnius, Lithuania: Vilnius University, Institute of Data Science and Digital Technologies (DMSTI), Lithuanian Academy of Sciences. ISSN: 0868-4952. doi:10.15388/Informatica.2003.037 (cit. on p. 351).

[157] Alfonsas Misevičius. *An Implementation of the Iterated Tabu Search Algorithm for the Quadratic Assignment Problem*. Working Paper. Kaunas, Lithuania: Kaunas University of Technology, 2008. See also[158] (cit. on p. 328).

# References XXIII


[158] Alfonsas Misevičius. "An Implementation of the Iterated Tabu Search Algorithm for the Quadratic Assignment Problem". *OR Spectrum: Quantitative Approaches in Management. Formerly Operations-Research-Spektrum* 34(3):665–690, July 2012. London, England, UK: Springer Nature Limited. ISSN: 0171-6468. doi:10.1007/s00291-011-0274-z. See also[157] (cit. on pp. 327, 350, 352).

[159] Nenad Mladenović and Pierre Hansen. "Variable Neighborhood Search". *Computers & Operations Research* 24(11):1097–1100, Nov. 1997. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0305-0548. doi:10.1016/S0305-0548(97)00031-2 (cit. on p. 354).

[160] Wai Ho Mow, Ke-Lin Du, and Wei Hsiang Wu. "New Evolutionary Search for Long Low Autocorrelation Binary Sequences". *IEEE Transactions on Aerospace and Electronic Systems* 51(1):290–303, Jan. 2015. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: 0018-9251. doi:10.1109/TAES.2014.130518 (cit. on p. 347).

[161] Heinz Mühlenbein. "How Genetic Algorithms Really Work: Mutation and Hillclimbing". In: *Parallel Problem Solving from Nature 2 (PPSN-II)*. Sept. 28–30, 1992, Brussels, Belgium. Ed. by Reinhard Männer and Bernard Manderick. Amsterdam, The Netherlands: Elsevier B.V., 1992, pp. 15–26. URL: https://www.researchgate.net/publication/284805798 (visited on 2025-08-15) (cit. on p. 349).

[162] Siegfried Nijssen and Thomas Bäck. "An Analysis of the Behavior of Simplified Evolutionary Algorithms on Trap Functions". *IEEE Transactions on Evolutionary Computation* 7(1):11–22, Feb. 2003. Los Alamitos, CA, USA: IEEE Computer Society. ISSN: 1089-778X. doi:10.1109/TEVC.2002.806169 (cit. on p. 351).

[163] Eugeniusz Nowicki and Czesław Smutnicki. "A Fast Taboo Search Algorithm for the Job Shop Problem". *Management Science* 42(6):783–938, June 1996. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0025-1909. doi:10.1287/mnsc.42.6.797 (cit. on p. 352).

[164] Regina O. Obe and Leo S. Hsu. *PostgreSQL: Up and Running*. 3rd ed. Sebastopol, CA, USA: O'Reilly Media, Inc., Oct. 2017. ISBN: 978-1-4919-6336-4 (cit. on p. 349).

[165] Tom Packebusch and Stephan Mertens. "Low Autocorrelation Binary Sequences". *Journal of Physics A: Mathematical and Theoretical* 49(16):165001, Mar. 2016. Bristol, England, UK: IOP Publishing Ltd. ISSN: 1751-8121. doi:10.1088/1751-8113/49/16/165001 (cit. on p. 347).

[166] Panos Miltiades Pardalos, Ding-Zhu Du, and Ronald Lewis Graham, eds. *Handbook of Combinatorial Optimization*. 1st ed. Boston, MA, USA: Springer, 1998. ISBN: 978-1-4613-7987-4.

# References XXIV


[167] Panos Miltiades Pardalos, Oleg V. Shylo, and Alkis Vazacopoulos. "Solving Job Shop Scheduling Problems Utilizing the Properties of Backbone and "Big Valley"". *Computational Optimization and Applications* 47(1):61–76, Sept. 2010. London, England, UK: Springer Nature Limited. ISSN: 0926-6003. doi:10.1007/s10589-008-9206-5. An algorithm similar to SA. (Cit. on pp. 346, 351).

[168] Francisco Parreño, Ramón Alvarez-Valdés, José Fernando Oliveira, and José Manuel Tamarit. "A Hybrid GRASP/VND Algorithm for Two- and Three-Dimensional Bin Packing". *Annals of Operations Research* 179(1):203–220, Sept. 2010. London, England, UK: Springer Nature Limited. ISSN: 0254-5330. doi:10.1007/s10479-008-0449-4 (cit. on pp. 341, 344, 353).

[169] Amit Patel. *Amit's A* Pages*. Stanford, CA, USA: Stanford University, 1997–2025. URL: https://theory.stanford.edu/~amitp/GameProgramming (visited on 2026-01-02) (cit. on pp. 122–127, 341).

[170] Bo Peng, Zhipeng Lü, and T. C. Edwin Cheng. "A Tabu Search/Path Relinking Algorithm to Solve the Job Shop Scheduling Problem". *Computers & Operations Research* 53:154–164, Jan. 2015. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0305-0548. doi:10.1016/j.cor.2014.08.006. URL: http://arxiv.org/abs/1402.5613 (visited on 2026-09-30) (cit. on pp. 346, 352).

[171] Ferdinando Pezzella and Emanuela Merelli. "A Tabu Search Method Guided by Shifting Bottleneck for the Job Shop Scheduling Problem". *European Journal of Operational Research* 120(2):297–310, Jan. 2000. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/S0377-2217(99)00158-7. URL: https://www2.cs.sfu.ca/CourseCentral/827/havens/papers/topic%5C%2310(JobShop)/Tabu%5C%20With%5C%20Shifting.pdf (visited on 2026-09-30) (cit. on p. 352).

[172] Martin Pincus. "Letter to the Editor – A Monte Carlo Method for the Approximate Solution of Certain Types of Constrained Optimization Problems". *Operations Research* 18(6):1225–1228, Nov.–Dec. 1970. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0030-364X. doi:10.1287/opre.18.6.1225 (cit. on p. 351).

[173] Marcelo Pinto, Cristóvão Silva, Matthias Thürer, and Samuel Moniz. "Survey in Operations Research and Management Science. Nesting and Scheduling Optimization of Additive Manufacturing Systems: Mapping the Territory". *Computers & Operations Research* 165(106592), Feb. 2024. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0305-0548. doi:10.1016/j.cor.2024.106592. Mentions some scheduling tasks of Additive Manufacturing jobs can be viewed as instance of the 2BP|R|F. (Cit. on p. 341).

# References XXV


[174] Alexander Podlich, Thomas Weise (汤卫恩), Manfred Menze, and Christian Gorldt. "Intelligente Wechselbrückensteuerung für die Logistik von Morgen". *Electronic Communications of the EASST (ECEASST)* 17(Communication in Distributed Systems), Feb. 27, 2009. The Netherlands: European Association of Software Science and Technology (EASST). ISSN: 1863-2122. doi:10.14279/tuj.eceasst.17.205. In the proceedings of *Workshops der Wissenschaftlichen Konferenz Kommunikation in Verteilten Systemen 2009* (WowKiVS 2009), 3 6, 2009, Kassel, Hessen, Germany (cit. on pp. 8–13).

[175] *PostgreSQL Essentials: Leveling Up Your Data Work*. Sebastopol, CA, USA: O'Reilly Media, Inc., Mar. 2024 (cit. on p. 349).

[176] S. D. Prestwich. *Improved Branch-and-Bound for Low Autocorrelation Binary Sequences*. arXiv.org: Computing Research Repository (CoRR) abs/1305.6187. Ithaca, NY, USA: Cornell Universiy Library, Apr. 3, 2024. doi:10.48550/arXiv.1305.6187. URL: https://arxiv.org/abs/1305.6187 (visited on 2013-07-23). arXiv:1305.6187v2 [cs.AI] 23 Jul 2013 (cit. on p. 347).

[177] *Programming Languages – C, Working Document of SC22/WG14*. International Standard ISO/IEC9899:2017 C17 Ballot N2176. Geneva, Switzerland: International Organization for Standardization (ISO) and International Electrotechnical Commission (IEC), Nov. 2017. URL: https://files.lhmouse.com/standards/ISO%20C%20N2176.pdf (visited on 2024-06-29) (cit. on p. 343).

[178] Sanatan Rai and George Vairaktarakis. "NP-Complete Problems and Proof Methodology". In: *Encyclopedia of Optimization*. Ed. by Christodoulos A. Floudas and Panos Miltiades Pardalos. 2nd ed. Boston, MA, USA: Springer, Sept. 2008, pp. 2675–2682. ISBN: 978-0-387-74758-3. doi:10.1007/978-0-387-74759-0_462 (cit. on p. 356).

[179] Soraya B. Rana and L. Darrell Whitley. "Genetic Algorithm Behavior in the MAXSAT Domain". In: *5th International Conference on Parallel Problem Solving from Nature (PPSN V)*. Sept. 27–30, 1998, Amsterdam, The Netherlands. Ed. by Ágoston Endre Eiben, Thomas Bäck, Marc Schoenauer, and Hans-Paul Schwefel. Lecture Notes in Computer Science (LNCS). 1998, pp. 785–794. ISSN: 0302-9743. ISBN: 978-3-540-65078-2. doi:10.1007/BFb0056920 (cit. on p. 348).

[180] Gerhard Reinelt. "TSPLIB – A Traveling Salesman Problem Library". *ORSA Journal on Computing* 3(4):376–384, Aut. 1991. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0899-1499. doi:10.1287/ijoc.3.4.376 (cit. on p. 352).

# References XXVI


[181] Gerhard Reinelt. *TSPLIB95*. Tech. rep. Heidelberg, Baden-Württemberg, Germany: Universität Heidelberg, Institut für Angewandte Mathematik, 1995–Apr. 28, 2016. URL: https://github.com/shredderzwj/TSPLIB (visited on 2026-09-25). The original Uniform Resource Locator (URL) of the repository has become defunct in 2026. Therefore, we provide the repository listed as main URL which contains the data. Additionally, you may also check https://github.com/thomasWeise/tspSuite of our own project *TSPSuite*, which contains all the TSPLib instances as well as Java code to load them. (Cit. on p. 352).

[182] Günter Rudolph. *Convergence Properties of Evolutionary Algorithms*. Vol. 35 of Forschungsergebnisse zur Informatik. Hamburg, Germany: Verlag Dr. Kovač, 1997. ISSN: 1435-6260. ISBN: 978-3-86064-554-3 (cit. on p. 347).

[183] Sartaj Sahni and Teofilo Francisco Gonzalez Arce. "P-complete Approximation Problems". *Journal of the ACM (JACM)* 23(3):555–565, July 1976. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 0004-5411. doi:10.1145/321958.321975. URL: https://sites.cs.ucsb.edu/~teo/papers/JACM-PCom.pdf (visited on 2026-02-21) (cit. on p. 350).

[184] Guntram Scheithauer. *Introduction to Cutting and Packing Optimization: Problems, Modeling Approaches, Solution Methods*. Vol. 263 of International Series in Operations Research & Management Science (ISOR). Cham, Switzerland: Springer, 2018. ISSN: 0884-8289. ISBN: 978-3-319-64402-8. doi:10.1007/978-3-319-64403-5 (cit. on pp. 8–13, 342).

[185] Guoyong Shi, Hitoshi Iima, and Nobuo Sannomiya. "New Encoding Scheme for Solving Job Shop Problems by Genetic Algorithm". In: *35th IEEE Conference on Decision and Control (CDC'96)*. Vol. 4. Dec. 11–13, 1996. Kōbe, Hyōgo-ken, Japan: Institute of Electrical and Electronics Engineers (IEEE), 1997, pp. 4395–4400. ISBN: 978-0-7803-3590-5. doi:10.1109/CDC.1996.577484. URL: https://www.researchgate.net/publication/224238934 (visited on 2026-09-30). Proposes a permutations-with-repetitions based encoding for the JSSP. (Cit. on pp. 318, 346, 355).

[186] Oleg V. Shylo. *Job Shop Scheduling (Personal Homepage)*. Knoxville, TN, USA: Oleg V. Shylo, Aug. 2019. URL: http://optimizizer.com/jobshop.php (visited on 2026-09-30). Contains many JSSP benchmark instances. (Cit. on pp. 332, 346).

# References XXVII


[187] Oleg V. Shylo and Hesam Shams. *Boosting Binary Optimization via Binary Classification: A Case Study of Job Shop Scheduling*. arXiv.org: Computing Research Repository (CoRR) abs/1808.10813. Proposes a TS using regression to adapt the tabu tenures for each variable of the solution vector of a JSSP separately. Many results are available in the GitHub repository https://github.com/quasiquasar/gta-jobshop-data; other results are published Shylo's website http://optimizizer.com/jobshop.php. Ithaca, NY, USA: Cornell Universiy Library, Aug. 31, 2018. doi:10.48550/arXiv.1808.10813. URL: https://arxiv.org/abs/1808.10813 (visited on 2026-09-30) (cit. on pp. 346, 352).

[188] Hossein M. Soroush. "Scheduling in Stochastic Bicriteria Single Machine Systems with Set-up Times". *International Journal of Planning and Scheduling* 1(1–2):109–145, Jan. 2011. Geneva, Switzerland: Inderscience Publishers. ISSN: 2044-494X. doi:10.1504/IJPS.2011.044605. A scheduling problem is phrased as QAP. (Cit. on p. 350).

[189] Leon Steinberg. "The Backboard Wiring Problem: A Placement Algorithm". *SIAM Review* 3(1):37–50, Jan. 1961. Philadelphia, PA, USA: Society for Industrial and Applied Mathematics Publications (SIAM). ISSN: 0036-1445. doi:10.1137/1003003. The problem is modeled as a QAP (cit. on p. 350).

[190] Robert H. Storer, S. David Wu, and Renzo Vaccari. "New Search Spaces for Sequencing Problems with Application to Job Shop Scheduling". *Management Science* 38(10):1495–1509, Oct. 1992. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0025-1909. doi:10.1287/mnsc.38.10.1495 (cit. on p. 346).

[191] Thomas Strecker and Leonhard Hennig. "Automatic Layouting of Personalized Newspaper Pages". In: *Operations Research Proceedings 2008: Selected Papers of the Annual International Conference of the German Operations Research Society (GOR)*. Sept. 3–5, 2008, Augsburg, Bayern, Germany: University of Augsburg. Ed. by Bernhard Fleischmann, Karl-Heinz Borgwardt, Robert Klein, and Axel Tuma. Vol. 2008 of Operations Research Proceedings (ORP), Gesellschaft für Operations Research e.V., (GOR). Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany, 2009, pp. 469–474. ISSN: 0721-5924. doi:10.1007/978-3-642-00142-0_76 (cit. on p. 341).

[192] Thomas Stützle. "Iterated Local Search for the Quadratic Assignment Problem". *European Journal of Operational Research* 174(3):1519–1539, Nov. 2006. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/j.ejor.2005.01.066 (cit. on pp. 345, 350).

# References XXVIII


[193] Éric D. Taillard. "Benchmarks for Basic Scheduling Problems". *European Journal of Operational Research* 64(2):278–285, Jan. 1993. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/0377-2217(93)90182-M. URL: https://arodes.hes-so.ch/record/8224/files/Preprint.pdf (visited on 2026-02-20) (cit. on p. 346).

[194] Éric D. Taillard. "Comparison of Iterative Searches for the Quadratic Assignment Problem". *Location Science. Now incorporated into Computers & Operations Research* 3(2):87–105, Aug. 1995. Amsterdam, The Netherlands: Elsevier B.V. doi:10.1016/0966-8349(95)00008-6 (cit. on p. 352).

[195] Éric D. Taillard. "Robust Taboo Search for the Quadratic Assignment Problem". *Parallel Computing* 17(4–5):443–455, July 1991. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0167-8191. doi:10.1016/S0167-8191(05)80147-4 (cit. on pp. 317, 333, 350, 352).

[196] Éric D. Taillard and Luca M. Gambardella. *Adaptive Memories for the Quadratic Assignment Problem*. Tech. rep. IDSIA-87-97. Lugano, Switzerland: Istituto Dalle Molle Di Studi Sull Intelligenza Artificiale (IDSIA), Sept. 10, 1997. URL: https://mistic.iict-heig-vd.ch/taillard/articles.dir/TaillardGambardella1997.pdf (visited on 2026-09-28). A MA using TS from [195] as well as a ACO-based approach refining its solutions with a very simple SLS are presented. (Cit. on p. 350).

[197] Alkin Tezuysal and Ibrar Ahmed. *Database Design and Modeling with PostgreSQL and MySQL*. Birmingham, England, UK: Packt Publishing Ltd, July 2024. ISBN: 978-1-80323-347-5 (cit. on p. 349).

[198] Dirk Thierens, David Edward Goldberg, and Angela Guimarães Pereira. "Domino Convergence, Drift, and the Temporal-Salience Structure of Problems". In: pp. 535–540. doi:10.1109/ICEC.1998.700085 (cit. on p. 340).

[199] Sarah Louise Thomson, Gabriela Ochoa, Daan van den Berg, Tianyu Liang (梁天宇), and Thomas Weise (汤卫思). "Entropy, Search Trajectories, and Explainability for Frequency Fitness Assignment". In: *Parallel Problem Solving from Nature (PPSN XVIII)*. Vol. 1. Sept. 14–18, 2024, Hagenberg, Mühlkreis, Austria. Ed. by Michael Affenzeller, Stephan M. Winkler, Anna V. Kononova, Heike Trautmann, Tea Tušar, Penousal Machado, and Thomas Bäck. Vol. 15148 of Lecture Notes in Computer Science (LNCS). Cham, Switzerland: Springer. ISSN: 0302-9743. ISBN: 978-3-031-70054-5. doi:10.1007/978-3-031-70055-2_23. Explores RLS, RLS with FFA (FRLS), and SA on the QAP. (Cit. on pp. 344, 348).

[200] Ulrich Wilhelm Thonemann and Andreas Bölte. *An Improved Simulated Annealing Algorithm for the Quadratic Assignment Problem*. Working Paper. Paderborn, Nordrhein-Westfalen, Germany: School of Business, Department of Production and Operations Research, University of Paderborn, 1994 (cit. on p. 351).

# References XXIX


[201] Marco Tomassini. "Complex Networks Analysis of the Energy Landscape of the Low Autocorrelation Binary Sequences Problem". *Physica A: Statistical Mechanics and its Applications* 577:126089, Sept. 1, 2021. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0378-4371. doi:10.1016/j.physa.2021.126089. Lists the LABS as $\mathcal{NP}$-hard. (Cit. on p. 347).

[202] Daan van den Berg, Florian Braam, Mark Moes, Emiel Suilen, and Sandjai Bhulai. "Almost Squares in Almost Squares: Solving the Final Instance". In: *5th International Conference on Data Analytics (DATA ANALYTICS'2016)*. Oct. 9–13, 2016, Venice, Venezia, Veneto, Italy. Ed. by Sandjai Bhulai and Ivana Semanjski. Wilmington, DE, USA: International Academy, Research, and Industry Association (IARIA), 2016, pp. 69–74. ISSN: 2308-4464. ISBN: 978-1-61208-510-4. URL: https://math.vu.nl/~sbhulai/publications/data_analytics2016b.pdf (visited on 2026-10-01). This poses a very interesting 2BP|R|F problem, where "almost squars," i.e., rectangles with sizes $(i, i+1)$, must be placed in an almost square. (Cit. on pp. 341, 342).

[203] Clarissa Van Hoyweghen, David Edward Goldberg, and Bart Naudts. "From Twomax To The Ising Model: Easy And Hard Symmetrical Problems". In: *4th Genetic and Evolutionary Computation Conference (GECCO'2002)*. July 9–13, 2002, New York, NY, USA. Ed. by William B. Langdon, Erick Cantú-Paz, Keith E. Mathias, Rajkumar Roy, David Davis, Riccardo Poli, Karthik Balakrishnan, Vasant G. Honavar, Günter Rudolph, Joachim Wegener, Larry Bull, Mitchell A. Potter, Alan C. Schultz, Julian F. Miller, Edmund K. Burke, and Natasa Jonoska. Burlington, MA, USA/San Mateo, CA, USA: Morgan Kaufmann Publishers, 2002, pp. 626–633. ISBN: 978-1-55860-878-8 (cit. on p. 353).

[204] Dieter Vandesande, Christoph Jabs, Jeremias Berg, Johannes Fichte, George Katsirelos, Ruben Martins, André Schidler, and Dominik Schreiber. *MaxSAT Evaluation 2026: Solvers, Benchmarks & Results*. v1. Geneva, Switzerland: zenodo (hosted by European Organization for Nuclear Research (CERN)), July 14, 2026. doi:10.5281/zenodo.21358014. See also https://maxsat-evaluations.github.io. (Cit. on p. 348).

[205] Kristian Verduin, Sarah Louise Thomson, and Daan van den Berg. "Too Constrained for Genetic Algorithms. Too Hard for Evolutionary Computing. The Traveling Tournament Problem". In: *15th International Joint Conference on Computational Intelligence (IJCCI'23)*. Nov. 13–15, 2023, Rome, Italy. Ed. by Niki van Stein, Francesco Marcelloni, H. K. Lam, Marie Cottrell, and Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2023, pp. 246–257. ISSN: 2184-3236. ISBN: 978-989-758-674-3. doi:10.5220/0012192100003595 (cit. on p. 353).

# References XXX


[206] Petr Vilím, Philippe Laborie, and Paul Shaw. "Failure-Directed Search for Constraint-Based Scheduling". In: *12th International Conference on the Integration of AI and OR Techniques in Constraint Programming (CPAIOR'2015)*. Ed. by Laurent Michel. Vol. 9075 of Theoretical Computer Science and General Issues (LNTCS), sub-series of Lecture Notes in Computer Science (LNCS). Barcelona, Spain: Springer, 2015, pp. 437–453. ISSN: 2512-2010. ISBN: 978-3-319-18007-6. doi:10.1007/978-3-319-18008-3_30. URL: https://www.researchgate.net/publication/280223626 (visited on 2026-09-30) (cit. on p. 346).

[207] Thomas Weise (汤卫思). *Databases*. Hefei, Anhui, China (中国安徽省合肥市): Hefei University (合肥大学), School of Artificial Intelligence and Big Data (人工智能与大数据学院), 2025. URL: https://thomasweise.github.io/databases (visited on 2025-01-05) (cit. on pp. 303, 343).

[208] Thomas Weise (汤卫思). *Global Optimization Algorithms – Theory and Application*. self-published, 2009. URL: https://www.researchgate.net/publication/200622167 (visited on 2025-07-25) (cit. on pp. 343, 347, 348, 351).

[209] Thomas Weise (汤卫思). *jsspInstancesAndResults: Results, Data, and Instances of the Job Shop Scheduling Problem*. San Francisco, CA, USA: GitHub Inc, 2019–2020. URL: https://github.com/thomasWeise/jsspInstancesAndResults (visited on 2025-08-20) (cit. on p. 346).

[210] Thomas Weise (汤卫思). *Programming with C*. Hefei, Anhui, China (中国安徽省合肥市): Hefei University (合肥大学), School of Artificial Intelligence and Big Data (人工智能与大数据学院), 2026. URL: https://thomasweise.github.io/programmingWithC (visited on 2026-05-10) (cit. on p. 343).

[211] Thomas Weise (汤卫思). *Programming with Python*. Hefei, Anhui, China (中国安徽省合肥市): Hefei University (合肥大学), School of Artificial Intelligence and Big Data (人工智能与大数据学院), 2024–2026. URL: https://thomasweise.github.io/programmingWithPython (visited on 2025-01-05) (cit. on pp. 302, 349).

[212] Thomas Weise (汤卫思), Raymond Chiong, Jörg Lässig, Ke Tang (唐珂), Shigeyoshi Tsutsui, Wenxiang Chen (陈文祥), Zbigniew "Zbyszek" Michalewicz, and Xin Yao (姚新). "Benchmarking Optimization Algorithms: An Open Source Framework for the Traveling Salesman Problem". *IEEE Computational Intelligence Magazine (CIM)* 9(3):40–52, Aug. 2014. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: 1556-603X. doi:10.1109/MCI.2014.2326101. Applies a multitude of algorithms including SLSes, EAs, MAs, ACO, and EDAs to the TSPLib instances. (Cit. on pp. 160–170, 343, 352, 354).

# References XXXI


[213] Thomas Weise (汤卫思), Xinlu Li (李新路), Yan Chen (陈岩), and Zhize Wu (吴志泽). "Solving Job Shop Scheduling Problems without using a Bias for Good Solutions". In: *23rd Genetic and Evolutionary Computation Conference (GECCO'2021), Companion Volume*. July 10–14, 2021, Lille, France. Ed. by Krzysztof Krawiec. New York, NY, USA: Association for Computing Machinery (ACM), 2021, pp. 1459–1466. ISBN: 978-1-4503-8351-6. doi:10.1145/3449726.3463124. Applies Frequency Fitness Assignment (FFA) to the JSSP. (Cit. on p. 355).

[214] Thomas Weise (汤卫思), Li Niu (牛力), and Ke Tang (唐珂). "AOAB – Automated Optimization Algorithm Benchmarking". In: *12th Annual Conference Companion on Genetic and Evolutionary Computation (GECCO'2010)*. July 7–11, 2010, Portland, OR, USA. Ed. by Martin Pelikan and Jürgen Branke. New York, NY, USA: Association for Computing Machinery (ACM), 2010, pp. 1479–1486. ISBN: 978-1-4503-0073-5. doi:10.1145/1830761.1830763 (cit. on p. 343).

[215] Thomas Weise (汤卫思), Alexander Podlich, and Christian Gorldt. "Solving Real-World Vehicle Routing Problems with Evolutionary Algorithms". In: *Natural Intelligence for Scheduling, Planning and Packing Problems*. Ed. by Raymond Chiong and Sandeep Dhakal. Vol. 250 of Studies in Computational Intelligence (SCI). Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany, Sept. 2009. Chap. 2, pp. 29–53. ISSN: 1860-949X. ISBN: 978-3-642-04038-2. doi:10.1007/978-3-642-04039-9_2 (cit. on pp. 8–13).

[216] Thomas Weise (汤卫思), Alexander Podlich, Manfred Menze, and Christian Gorldt. "Optimierte Güterverkehrsplanung mit Evolutionären Algorithmen". *Industrie Management – Zeitschrift für industrielle Geschäftsprozesse* 10(3):37–40, June 2009. Berlin, Germany: GITO mbH Verlag (cit. on pp. 8–13).

[217] Thomas Weise (汤卫思), Alexander Podlich, Kai Reinhard, Christian Gorldt, and Kurt Geihs. "Evolutionary Freight Transportation Planning". In: *Applications of Evolutionary Computing (EvoWorkshops 2009): Proceedings of EvoCOMNET, EvoENVIRONMENT, EvoFIN, EvoGAMES, EvoHOT, EvoIASP, EvoINTERACTION, EvoMUSART, EvoNUM, EvoSTOC, EvoTRANSLOG*. Apr. 15–17, 2009, Tübingen, Baden-Württemberg, Germany. Ed. by Mario Giacobini, Anthony Brabazon, Stefano Cagnoni, Gianni A. Caro, Anikó Ekárt, Anna Isabel Esparcia-Alcázar, Muddassar Farooq, Andreas Fink, and Penousal Machado. Vol. 5484 of Theoretical Computer Science and General Issues (LNTCS), sub-series of Lecture Notes in Computer Science (LNCS). Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany, Apr. 2009, pp. 768–777. ISSN: 2512-2010. doi:10.1007/978-3-642-01129-0_87 (cit. on pp. 8–13).

[218] Thomas Weise (汤卫思), Mingxu Wan (万明绪), Pu Wang (王璞), Ke Tang (唐珂), Alexandre Devert, and Xin Yao (姚新). "Frequency Fitness Assignment". *IEEE Transactions on Evolutionary Computation* 18(2):226–243, Apr. 2014. Los Alamitos, CA, USA: IEEE Computer Society. ISSN: 1089-778X. doi:10.1109/TEVC.2013.2251885. The article introducing FFA. (Cit. on p. 344).

# References XXXII


[219] Thomas Weise (汤卫思) and Zhize Wu (吴志泽). "Replicable Self-Documenting Experiments with Arbitrary Search Spaces and Algorithms". In: *25th Conference on Genetic and Evolutionary Computation (GECCO'2023), Companion Volume*. July 15–19, 2023, Lisbon, Portugal. Ed. by Sara Silva and Luís Paquete. New York, NY, USA: Association for Computing Machinery (ACM), 2023, pp. 1891–1899. ISBN: 979-8-4007-0120-7. doi:10.1145/3583133.3596306. Introduces the experiment execution and data collection facilities of moptipy. (Cit. on pp. 304, 343, 348).

[220] Thomas Weise (汤卫思), Zhize Wu (吴志泽), Xinlu Li (李新路), and Yan Chen (陈岩). "Frequency Fitness Assignment: Making Optimization Algorithms Invariant under Bijective Transformations of the Objective Function Value". *IEEE Transactions on Evolutionary Computation* 25(2):307–319, Apr. 2021. Los Alamitos, CA, USA: IEEE Computer Society. ISSN: 1089-778X. doi:10.1109/TEVC.2020.3032090 (cit. on pp. 340, 344, 346, 347, 349, 351, 353).

[221] Thomas Weise (汤卫思), Zhize Wu (吴志泽), Xinlu Li (李新路), Yan Chen (陈岩), and Jörg Lässig. "Frequency Fitness Assignment: Optimization without Bias for Good Solutions can be Efficient". *IEEE Transactions on Evolutionary Computation* 27(4):980–992, 2023. Los Alamitos, CA, USA: IEEE Computer Society. ISSN: 1089-778X. doi:10.1109/TEVC.2022.3191698. Applies many FFA-based algorithm variants to problems such as OneMax, LeadingOnes, Linear Harmonic Function (LinHarm), TwoMax, Trap, Jump, Plateau, Ising Model, and N-Queens. (Cit. on pp. 340, 344, 346, 351, 353).

[222] Thomas Weise (汤卫思), Yuezhong Wu (吴越钟), Raymond Chiong, Ke Tang (唐珂), and Jörg Lässig. "Global versus Local Search: The Impact of Population Sizes on Evolutionary Algorithm Performance". *Journal of Global Optimization* 66(3):511–534, Feb. 2016. London, England, UK: Springer Nature Limited. ISSN: 0925-5001. doi:10.1007/s10898-016-0417-5 (cit. on pp. 160–170, 352).

[223] L. Darrell Whitley. "The GENITOR Algorithm and Selection Pressure: Why Rank-Based Allocation of Reproductive Trials is Best". In: *3rd International Conference on Genetic Algorithms (ICGA'1989)*. June 1989, Fairfax, VA, USA: George Mason University. Ed. by J. David Schaffer. Burlington, MA, USA/San Mateo, CA, USA: Morgan Kaufmann Publishers, 1989, pp. 116–123. ISBN: 978-1-55860-066-9. URL: https://www.researchgate.net/publication/2527551 (visited on 2025-08-08) (cit. on p. 347).

[224] James M. Wilson. "Gantt Charts: A Centenary Appreciation". *European Journal of Operational Research* 149(2):430–437, Sept. 2003. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/S0377-2217(02)00769-5 (cit. on p. 344).

# References XXXIII


[225] L. Wong and Lai-Soon Lee. "Heuristic Placement Routines for Two-Dimensional Bin Packing Problem". 5(4):334–341, Dec. 2009. doi:/10.3844/jmssp.2009.334.341. URL: https://www.thescipub.com/pdf/jmssp.2009.334.341 (visited on 2026-10-02). A heuristic for the 2BP|R|F and the 2BP|O|F (cit. on p. 309).

[226] CAO Xiang (曹翔), Zhize Wu (吴志泽), Daan van den Berg, and Thomas Weise (汤卫思). "Randomized Local Search vs. NSGA-II vs. Frequency Fitness Assignment on The Traveling Tournament Problem". In: *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. Nov. 20–22, 2024, Porto, Portugal. Ed. by Francesco Marcelloni, Kurosh Madani, Niki van Stein, and Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024, pp. 38–49. ISSN: 2184-3236. ISBN: 978-989-758-721-4. doi:10.5220/0012891500003837 (cit. on pp. 348, 353, 355).

[227] Zhou Xu and Chung-Yee Lee. "New Lower Bound and Exact Method for the Continuous Berth Allocation Problem". *Operations Research* 66(3):ii–iv, 597–892, May–June 2018. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0030-364X. doi:10.1287/opre.2017.1687. URL: https://ira.lib.polyu.edu.hk/bitstream/10397/78451/1/Xu_New_Lower_Bound.pdf (visited on 2026-10-01) (cit. on p. 342).

[228] Takeshi Yamada and Ryohei Nakano. "A Genetic Algorithm Applicable to Large-Scale Job-Shop Instances". In: *Parallel Problem Solving from Nature 2 (PPSN-II)*. Sept. 28–30, 1992, Brussels, Belgium. Ed. by Reinhard Männer and Bernard Manderick. Amsterdam, The Netherlands: Elsevier B.V., 1992, pp. 281–290. URL: https://www.researchgate.net/publication/220701684 (visited on 2026-09-30) (cit. on p. 346).

[229] Kinza Yasar and Craig S. Mullins. *Definition: Database Management System (DBMS)*. Newton, MA, USA: TechTarget, Inc., June 2024. URL: https://www.techtarget.com/searchdatamanagement/definition/database-management-system (visited on 2025-01-11) (cit. on p. 343).

[230] François Yergeau. *UTF-8, A Transformation Format of ISO 10646*. Request for Comments (RFC) 3629. Wilmington, DE, USA: Internet Engineering Task Force (IETF), Nov. 2003. URL: https://www.ietf.org/rfc/rfc3629.txt (visited on 2025-02-05). See Unicode and[108] (cit. on p. 353).

# References XXXIV


[231] Jiazheng Zeng (曾嘉政), Thomas Weise (汤卫思), Zhize Wu (吴志泽), and Markus Wagner. "When Frequency Fitness Assignment Fails: Trapped States in Frequency-Guided Local Search". In: *28th Genetic and Evolutionary Computation Conference (GECCO'2026)*. July 13–17, 2026, San José, Costa Rica. Ed. by Leonardo Trujillo and Ting Hu. New York, NY, USA: Association for Computing Machinery (ACM), 2026, pp. 233–242. ISBN: 979-8-4007-2487-9. doi:10.1145/3795095.3805098. Explores $(1+1)$ evolutionary algorithm ($(1+1)$ EA), $(1+1)$ EA with FFA ($(1+1)$ FEA), RLS, and FRLS as well as hybrid algorithms on a variety of discrete benchmarks such as OneMax, LeadingOnes, LinHarm, Binary Integer (BinInt), TwoMax, Trap, Jump, Plateau, Ising Model, N-Queens and also applies them to the search for Low-Autocorrelation Binary Sequences (LABS). (Cit. on pp. 340, 344–351, 353).

[232] Chaoyong Zhang, Xinyu Shao, Yunqing Rao, and Haobo Qiu. "Some New Results on Tabu Search Algorithm Applied to the Job-Shop Scheduling Problem". In: *Tabu Search*. Ed. by Wassim Jaziri. London, England, UK: IntechOpen Limited, Aug. 1, 2008. Chap. 8. ISBN: 978-3-902613-34-9. doi:10.5772/5593. URL: https://www.intechopen.com/chapters/4596 (visited on 2026-09-30) (cit. on pp. 346, 352).

[233] Rui Zhao (赵睿), Tianyu Liang (梁天宇), Zhize Wu (吴志泽), Daan van den Berg, Matthias Thürer, and Thomas Weise (汤卫思). "Randomized Local Search on the 2D Rectangular Bin Packing Problem with Item Rotation". In: *26th Genetic and Evolutionary Computation Conference (GECCO'2024)*. July 14–18, 2024, Melbourne, VIC, Australia. Ed. by Xiaodong Li and Julia Handl. New York, NY, USA: Association for Computing Machinery (ACM), 2024, pp. 235–238. ISBN: 979-8-4007-0494-9. doi:10.1145/3638530.3654139 (cit. on pp. 8–13, 133–135, 341, 345, 348).

[234] Rui Zhao (赵睿), Zhize Wu (吴志泽), Daan van den Berg, Matthias Thürer, Tianyu Liang (梁天宇), Ming Tan (檀明), and Thomas Weise (汤卫思). "Randomized Local Search for Two-Dimensional Bin Packing and a Negative Result for Frequency Fitness Assignment". In: *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. Nov. 20–22, 2024, Porto, Portugal. Ed. by Francesco Marcelloni, Kurosh Madani, Niki van Stein, and Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024, pp. 15–26. ISSN: 2184-3236. ISBN: 978-989-758-721-4. doi:10.5220/0012888500003837 (cit. on pp. 8–13, 133–135, 341, 342, 345, 348).

# Glossary I

**$(1+1)$ EA** The $(1+1)$ EA is a local search algorithm that retains the best solution $x_c$ discovered so far during the search[36,67]. In each step, it applies a unary search operator to this best-so-far solution $x_c$ and derives a new solution $x_n$. If the new solution $x_n$ is *better or equally good* when compared with $x_c$, i.e., not worse, then it replaces it, i.e., is stored as the new $x_c$. If the search space are bit strings of length $n$, then the $(1+1)$ EA uses a unary search operator that flips each bit independently with probability $m/n$, where usually $m = 1$. This operator is the main difference to randomized local search (RLS). The $(1+1)$ EA is a special case of the $(\mu+\lambda)$ evolutionary algorithm ($(\mu+\lambda)$ EA) where $\mu = \lambda = 1$.

**$(1+1)$ FEA** The $(1+1)$ EA with FFA plugged in [220, 221, 231]. On the discrete bit-string domains, this algorithm has a very specific structure: Its unary search operator flips each bit with probability $1/n$, where $n$ is the dimension of the search space, distinguishing it from the unary operator in FRLS which flips exactly one bit. On other domains, such as permutations, the term is used synonymously with FRLS.

**$(\mu+\lambda)$ EA** The $(\mu+\lambda)$ EA is an evolutionary algorithm (EA) where, in each generation, $\lambda$ offspring solutions are generated from the current population of $\mu$ parent solutions. The offspring and parent populations are merged, yielding $\mu+\lambda$ solutions, from which then the best $\mu$ solutions are ratained to form the parent population of the next generation. If the search space is the bit strings of length $n$, then this algorithm usually applies a mutation operator flipping each bit independently with probability $1/n$.

**BinInt** The *Binary Integer Problem* is a benchmark problem for bit-string based black-box search[198,231]. It is very similar to OneMax: It tries to maximize the number of 1 bits in the bit string of a chosen dimension $n$. Different from OneMax, where each bit has the same weight in the objective function, the bits of BinInt have a exponentially increasing weight: The bit string is treated as a binary-encoded integer whose value should be maximized. The objective function $f(x)$ of the minimization variant of this problem can be defined as $f(x) = 2^n - 1 - \sum_{i=1}^{n} 2^{i-1} \cdot x_i$. The problem is unimodal, i.e., has only a single global optimum and no local optimum. It is fully separable, i.e., we can optimize each variable separately.

**LinHarm** The Linear Harmonic Function is a benchmark problem for bit-string based black-box search[61,67,231]. It is very similar to OneMax: It tries to maximize the number of 1 bits in the bit string of a chosen dimension $n$. Different from OneMax, where each bit has the same weight in the objective function, the bits of LinHarm have a linear increasing weight. The objective function $f(x)$ of the minimization variant of this problem can be defined as $f(x) = \frac{1}{2}n(n+1) - \sum_{i=1}^{n} i \cdot x_i$. The problem is unimodal, i.e., has only a single global optimum and no local optimum. It is fully separable, i.e., we can optimize each variable separately. The $(1+1)$ EA can solve this problem with an expected runtime in $\Theta(n \ln n)$[61,67].

# Glossary II

**2BP** In the two-dimensional Bin Packing Problem (BP), the goal is to pack rectangles into rectangular bins without overlap and such that their edges are parallel to the edges of the bins[109,140,142]. The 2BP is strongly $\mathcal{NP}$-hard[142]. The variants of this problem include the 2BP|O|F, 2BP|R|F, 2BP|O|G, and 2BP|R|G[141,142]. Different objective function for the 2BP|O|F and 2BP|R|F are presented in [233]. 2BPs have many practical applications, including cutting (wood, glas)[142], packing (transportation, warehouse)[142], and newspaper page layout[191].

**2BP|O|F** In the oriented 2BP with free cuts, the rectangles to pack have fixed orientations, i.e., cannot be rotated, but can otherwise be arranged freely[141,142,151,234]. Permutation-based search spaces can be used for this problem by using the Improved Bottom Left (IBL) encoding. The state-of-the-art (SOTA) for this problem comprises[22,27,44,91,168,234].

**2BP|O|G** In the oriented 2BP with guillotine cuts, the rectangles to pack have fixed orientations, i.e., cannot be rotated, and must be arranged such that they can be extracted by edge-to-edge cuts[141,142].

**2BP|R|F** In this variant of the 2BP with free cuts, the rectangles to pack can be rotated by 90° if need be and can freely be arranged[19,141,142,233,234]. Besides packing and cutting, this problem also occurs in scheduling and build formation in additive manufacturing[133,173]. The almost-squares-in-almost-squares (Asqas) problem[202] is also a 2BP|R|F. Search spaces based on signed permutations can be used for this problem by using the IBL encoding. The SOTA for this problem comprises[22,44,91,233,234].

**2BP|R|G** In this variant of the 2BP with guillotine cuts, the rectangles to pack can be rotated by 90° if need be, but must be arranged such that they can be extracted by edge-to-edge cuts[141,142].

**A* Algorithm** The A* Algorithm is an a greedy best-first-first search for finding the shortest path between two locations[99,169]. The algorithm iteratively constructs the shortest path. It maintains a list of current candidate paths. For choosing the next candidate path whose end node should be expanded, it computes a value $f(p)$ of these paths $p$. $f(p)$ is the sum of the cost $g(p)$ so far incurred by the path as well as a heuristic $h(p)$ predicting the cost from the current end of the path to the goal. $h(p)$ must never overestimate that cost.

**ACO** *Ant Colony Optimization* is a nature-inspired optimization method for combinatorial problems where solutions are generated by "ants" that move from node to node in a graph choosing edges based on (1) the simulated pheromone on the edges and (2) a per-edge heuristic value[63–65]. If an ant produced a good solution, "pheromone" is distributed over the edges it visited, making it more likely to be re-visited by other ants.

# Glossary III

**Asqas** An "almost-square" be a rectangle with size $i \times (i+1)$ with $i \in \mathbb{N}_1$. The *almost-squares-in-almost-squares* problem[202] is a special case of the 2BP|R|F. Here, almost-square rectangles of sizes $1 \times 2$, $2 \times 3$, $\ldots$, $n \times (n+1)$ must be placed into an almost-square bin without waste. This requires that there exists a natural number $p \in \mathbb{N}_1$ such that $p(p+1) = \sum_{i=1}^{n} i(i+1)$[202]. There are exacty five Asqas instances[202], namely with bin sizes $1 \times 2$ for $n = 1$, $p = 1$, $5 \times 4$ for $n = 3$, $p = 4$, $15 \times 16$ for $n = 8$, $p = 15$, $55 \times 56$ for $n = 20$, $p = 55$, and $119 \times 120$ for $n = 34$, $p = 119$.

**ATSP** The general case for TSPs are *asymmetric Traveling Salesperson Problems*, where the $n \times n$ distance matrix $A$ does not need to be symmetric. In other words, it is possible that $\exists a, b \in 1..n$ with $A[a, b] \neq A[b, a]$. Notice that while such index tuple *may* exist, this is not always or necessarily the case. In this case, one may consider the STSP as a special case of the ATSP, while ATSPs may be considered as synonymous to TSPs.

**B&B** *Branch and bound* is a family of optimization methods that combines systematic exploration of the search space with both and objective function $f$ subject to minimization and a lower bound function. The idea is that we divide the search space into two sets. We then pick the set that seems more interesting and divide it again. We do this recursively until we have arrived at a set of size 1, which is our first solution $y$. We can compute its objective value $z = f(y)$. Now we continue with the sets that we have created and not yet explored. However, we can discard all sets for which the lower bound of the solution quality is $\geq z$, because they can never give us a better solution than $y$. If we have divided another set down to size 1 and it yields a better solution than $y$, of cause we update these values. Eventually, we have either discarded or fully expanded all sets – and then the solution we got must be optimal. B&B-algorithms have been applied to the TSP[138], the JSSP[35], the 2BP|O|F[151], and maritime logistics[227], for instance.

**BP** The *Bin Packing Problem*[70,184] is one of the classical problems from OR. The goal is to pack a set of objects into containers, using as few containers as possible. Different flavors of BPs can be distinguished, e.g., by the number of dimensions[54,140] that the objects and containers have (see, e.g., the 2BP), and/or whether the objects can be rotated[234], as well as other constraints and limitations. Both the one- and two-dimensional BP are strongly $\mathcal{NP}$-hard[142].

# Glossary IV

BRKGA The *Biased Random Key Genetic Algorithm* is a EA with a continuous search space $\mathbb{X} \subset [0,1]^n$ and a decoding function $\gamma$ translating this space to something else and biased, uniform, and elitist mating strategies[93,145]. The *bias* comes from the fact that one of the parents used for the binary operation is biased to be of higher fitness than the other parent[93]. Often, $\gamma$ is the Random Keys encoding[13,14] procedure translating $\mathbb{X}$ to permutations of the first $n$ natural numbers[92]. Sometimes, it is actually a MA further refining the translation result with a local search (LS)[92]. Sometimes the permutations are then further translated to more complex data structures, such as Gantt charts for the JSSP[92], or packing plans for 2BP 3D packing[91].

C is a programming language, which is very successful in system programming situations[58,177,210].

Concorde is an exact solver for the TSP[4,46,47], which could even solve the largest instance of TSPLib (with 85 900 cities) to optimality.

DB A *database* is an organized collection of structured information or data, typically stored electronically in a computer system. Databases are discussed in our book *Databases*[207].

DBMS A *database management system* is the software layer located between the user or application and the database (DB). The DBMS allows the user/application to create, read, write, update, delete, and otherwise manipulate the data in the DB[229].

EA An *evolutionary algorithm* is a metaheuristic optimization method that maintains a population of candidate solutions, which undergo selection (where better solutions are chosen with higher probability) and reproduction (where mutation and recombination create a new candidate solution from one or two existing ones, repectively)[7,208].

FE *Objective function evaluations* are an implementation-independent measure of runtime for optimization algorithms[8,212,214,219]. 1 FE equals to one evaluated candidate solution during the optimization process.

# Glossary V

FFA *Frequency Fitness Assignment* (中文: 频率适应度分配) is a algorithm plugin for optimization methods applied to discrete or combinatorial problems with not-too-many different possible objective values[218,220,221]. It replaces the objective values in all comparisons with their absolute encounter frequency so far during the search. FFA has successfully been applied to the Maximum Satisfiability (MaxSAT)[220,221], the TSP[134,135], and the QAP[41,199], amongst others. Its properties have been intensely researched on discrete benchmark functions such as OneMax, LeadingOnes, Trap, TwoMax, Jump, and Plateau[220,221,231]. The most common realizations of FFA so far are the $(1+1)$ FEA and FRLS.

FSSP The *Flow Shop Scheduling Problem* is a variant of a JSSP where all products go through the same sequence of machines[95,118].

Gantt chart Gantt charts are two-dimensional diagrams illustrating the steps of a process[120,224]. Their horizontal axis represents the time, the vertical axis represents work packages, machines, or workers. In the diagram area, bars located at certain vertical locations indicate that the work package, machine, or worker is processed or works at the time range covered by the bar as projected to the horizontal axis.

GRASP A *Greedy Randomized Adaptive Search Procedure* is a metaheuristic where each cycle consists of two main components[74,76,77]: First, a starting point is generated using a heuristic method (as opposed to being randomly sampled). Then, a LS is applied to refine the solution. Then, another cycle starts and proceeds in the same way. The overall best solution is retained as result. Many examples of applications of GRASPs are listed in [77]. GRASP in combination with VND has been applied to the 2BP|O|F in [168].

# Glossary VI



IBL The *improved bottom left* heuristic is a one-shot algorithm constructing a solution for 2BP[139]. For the 2BP|O|F problem variant, the input of the IBL algorithm is a permutation of the first $n$ natural numbers, each number identifying one rectangle to be packed. For the 2BP|R|F problem flavor, the permutation elements are signed, where a negative sign indicates a rotation by 90° [139]. In[233,234] it is adapted to a multi-bin situation as follows: The algorithm iterates over the (potentially signed) permutation $x$ and places the rectangles into the packing plan in the prescribed order. It first places the rectangle outside on top of the current bin $b$, with its bottom-right corner onto the top-right corner of the current bin $b$. It then moves the rectangle downwards and leftwards as far as possible, prioritizing the downward movement whenever possible. If the rectangle cannot be moved any further, it checks if it is completely contained in the bin $b$. If yes, it can remain there. Otherwise, a new bin is opened ($b \leftarrow b + 1$) and the rectangle is placed at its bottom-left corner. Once a new bin is opened, this bin is used for all further insertions. IBL is suitable as a decoding function $\gamma$ inside an EA[139]. Historically, it improves upon the BL algorithm[113]: That algorithm cannot represent all possible optimal packings using a permutation, but the IBL can[139].

ILS An *iterated local search* is a variant of the LS which applies cycles of local search and perturbation[192]. First, an initial solution is generated and refined by the local search. Then the cycle begins: A perturbation, i.e., a large search move that likely destroys a good part of the solution, is applied. The resulting new solution is refined by local search. Then, an acceptance criterion decides whether we should use this or the previous solution to continue in the next cycle. ILS has successfully been applied to the QAP.

Ising Model The Ising models are benchmark problems for bit-string based black-box search[61,78,79,231]. Here, a neighborhood is defined around each bit. The bit needs to have the same value as all bits in the neighborhood structure $T$. Thus, we try to achieve that all bits have the same value. The objective function $f(x)$ of the minimization variant of this problem family can be defined as $f(x, T) = \sum_{(i,j)\in T} |x_i - x_j|$, where $T$ contains the index pairs of neighboring bits and is defined differently for the Ising model instances Ising 1D and Ising 2D. The problems are highly epistatic, as all variables interact with some other variables.

Ising 1D The 1D is a benchmark problem for bit-string based black-box search[61,79,231]. The bits are arranged in a one-dimensional ring and we try to minimize the number of bits that have different values than their neighbors. Hence, the neighborhood structure $T$ is defined as $T_{\mathbf{1}D} = \{(i+1, (i+1 \bmod n)+1) \mid i \in [0..n-1]\}$. The problems are highly epistatic, as all variables interact with some other variables. The $(1+1)$ EA has an expected runtime in $\mathcal{O}(n^{\mathbf{3}})$ on 1-Dimensional Ising Model (Ising 1D)[79].

# Glossary VII

**Ising 2D** The 2D is a benchmark problem for bit-string based black-box search[61,78,231]. The bits are arranged in a two-dimensional torus and we try to minimize the number of bits that have different values than their neighbors. Hence, the neighborhood structure $T = T_{2D}$ is defined as the set of all $\{(\alpha + \beta N, \gamma + \delta N)\}$ pairs with $\alpha, \beta, \gamma, \delta \in 0..N-1$ and either $()\gamma = (\alpha + 1) \bmod N$ and $\delta = \beta$) or ($\gamma = \alpha$ and $\delta = (\beta + 1) \bmod N$) with $n = N^2$. The problems are highly epistatic, as all variables interact with some other variables. On a similar class of problems, the $(1 + 1)$ EA has an exponential expected runtime[78].

**Java** is another very successful programming language, with roots in the C family of languages[26,146].

**JSSP** The *Job Shop Scheduling Problem*[25,43,95,129] is one of the most prominent and well-studied scheduling tasks. In a JSSP instance, there are $m$ machines and $k$ jobs[193]. Each job must be processed once by each machine in a job-specific sequence and has a job-specific processing time on each machine. The goal is to find an assignment of jobs to machines that results in an overall shortest makespan, i.e., the schedule (Gantt chart) which can complete all the jobs in the shortest time. The JSSP is $\mathcal{NP}$-complete[39,128,129]. The 8 most common sets of JSSP benchmark instances are *abz*[1], *dmu*[55], *ft*[80], *la*[131], *orb*[5], *swv*[190], *ta*[193], and *yn*[228].Their data can be found (sometimes partially) in several repositories, such as the OR-Library[15,16], [186], and [209], the latter of which also contains many results from existing works. Encodings based on permutations with repetition are proposed in [23, 24, 86, 185] and used in [220]. The SOTA of the JSSP includes[92,102,167,170,187,206,232].

**Jump** The Jump problem is a benchmark problem for bit-string based black-box search[67,83,231]. Like the OneMax, it tries to maximize the number of 1 bits in a bit string of chosen length $n$. However, the solutions representing the last $\omega$ bits around the optimum to flip are deceptive, meaning that they are actually worse and getting worse than surrounding solutions. The objective function $f(x)$ of the minimization variant is $n - \|x\|_1$ if $\|x\|_1 = n$ or $\|x\|_1 \leq n - \omega$ hold, and $\omega + \|x\|_1$ otherwise, where $\|x\|_1 = \sum_{i=1}^{n} x_i$. This means that, the larger $\omega$, the larger the deceptive basin around the optimum that an algorithm needs to cross. An $(1 + 1)$ EA will converge to the resulting local optimum and thus have an expected runtime of $\Theta(n^\omega + n \ln n)$[67].In experiments, the $(1 + 1)$ FEA achieves polynomial runtime on Jump, seemingly in $\mathcal{O}(n^2 \ln n)$[220,221,231].

# Glossary VIII



LABS *Low-Autocorrelation Binary Sequences* are bit strings of length $n$ whose autocorrelation is as low as possible[28–30,56,57,84,152,154,160,165,176]. This means that sub-sequences of any length $1 \leq k < n$ should ideally never repeat. The objective function $f : \{0,1\}^n \mapsto \mathbb{N}_0$ represents the energy of a bit string $x$ and is subject to minimization. We can define it as $f(x) = \sum_{k=1}^{n-1} \left(\sum_{i=0}^{n-1-k} s_i \cdot s_{i+k}\right)^2$ with $s_j = 2x_j - 1$. Such sequences are needed for communication engineering[29,30,160], cryptography[56,160], as well as modulation pulses in radar and sonar ranging[165]. The LABS is $\mathcal{NP}$-hard[201]. The SOTA on LABS contains[28,30,56,57,160].

LeadingOnes The LeadingOnes problem is a benchmark problem for bit-string based black-box search[182,223,231]. It tries to maximize the number of *leading* 1 bits in the bit string of a chosen dimension $n$. The objective function $f(x)$ of the minimization variant of this problem can be defined as $f(x) = n - \sum_{i=1}^{n} \prod_{j=1}^{i} x_j$. Different from OneMax, the contribution to the objective value of the bit at index $i$ thus depends on the state of all the bits *before* this index. The problem is unimodal, i.e., has only a single global optimum and no local optimum. Different from OneMax, it is not separable, i.e., we can optimize each variable separately. The expected runtime of $(1+1)$ EA on LeadingOnes is in $\Theta(n^2)$[67]. The average runtime $(1+1)$ FEA on LeadingOnes seems to be slower by factor proportional to $n$, i.e., seems to be in $\mathcal{O}(n^3)$[220].

LK The *Lin-Kernighan* (LK) algorithm[137] is a SLS method for the TSP. The LK algorithm performs iterative $k$-opt moves, i.e., in each step, it constructs the two sets $X$ and $Y$ of $k$ edges such that removing the edges in $X$ from the current tour $T$ and then adding the edges from $Y$ to $T$ results in a better tour.The LK algorithm is one of the most classical approaches to the TSP.

LKH The *Lin-Kernighan-Helsgaun* algorithm[47,100,101] is a SLS method for the TSP based on the Lin-Kernighan (LK) algorithm[137]. LKH-1 was proposed in [101] and LKH-2 in [100]. It holds the record in the *World TSP*[48] as of 2025.

LS A *local search* is an algorithm where a single "current" candidate solution is maintained and new solutions are selected from its neighborhood in each step. Most LSes are randomized, i.e., stochastic local search (SLS).

MA A *Memetic Algorithm* is an EA where each new solution is refined by a LS before it enters the population[208]. As LS, we can use, e.g., RLS, but TS[42,81] is maybe a better choise. Some SOTA algorithms in some domains may be classified as MAs, e.g., Lin-Kernighan Helsgaun (LKH) basically uses a population that is refined via a local search. MAs also worked well on the JSSP[42,102] and the TSP[81].

# Glossary IX



**MaxSAT** The goal of satisfiability problems is to find an assignment for $n$ Boolean variables that make a given Boolean formula $F : \{0,1\}^n \mapsto \{0,1\}$ become true. In the *Maximum Satisfiability* problem (MaxSAT)[103], $F$ is given in conjunctive normal form, i.e., the variables appear either directly or negated in $m$ "or" clauses, which are all combined into one "and." The objective function $f(x)$, subject to minimization, computes the number of clauses which are false under the variable setting $x$. If $f(x) = 0$, then all clauses of $F$ are true, which solves the problem. The MaxSAT problem is $\mathcal{NP}$-complete[45] and exhibits low epistasis but deceptiveness[179], making it hard for local searches. Typical benchmark instances can be found in SATLIB, which contains smaller-scale instances, and the up-to-date MaxSAT Evaluation (MSE), providing SOTA benchmarks, algorithms, and results.

**metaheuristic** A *metaheuristic* is an optimization algorithm which performs a cycle of trial and error during which candidate solutions are sampled from the solution space $\mathbb{Y}$ and evaluated using an objective function $f : \mathbb{Y} \mapsto \mathbb{R}$[89,208]. The results of the objective function are used to decide which solution is promising. The neighborhoods around promising solutions *or* similarly-structured solutions are then sampled in the next cycle. Prominent metaheuristics are SLSes, SA, TS, and EAs.

**moptipy** is the *Metaheuristic Optimization in Python* library[219]. It has been used in several different research works, including[41,134–136,199,220,233,234]. Algorithms implemented by moptipy include EAs, $(1+1)$ EA, RLS, SA, and the $(1+1)$ FEA, amongst others. Learn more at https://thomasweise.github.io/moptipy and https://thomasweise.github.io/moptipyapps.

**MSE** The *MaxSAT Evaluation* is a regularly updated set of benchmark instances for the MaxSAT[110]. It also includes solver software and benchmark results[204].

**N-Queens** The N-Queens Problem is a benchmark problem for bit-string based black-box search[61,231]. The goal is to arrange $N$ queens on an $N \times N$ chessboard so that none can attack the other. The search space has thus dimension $n = N^2$, with each bit standing for a field and encoding whether or not a queen is placed there. The objective function $f(x)$ of the minimization variant is $N - \left(\sum_{i=1}^{n} x_i\right) - N \sum_{\xi} \max\left\{0, Q_\xi(x) - 1\right\}$ where $Q_\xi(x)$ is the number of queens in the horizontal, vertical, or diagonale $\xi$. The problem is quite epistatic.

# Glossary X



**OneMax** OneMax is the simplest possible benchmark problem for bit-string based black-box search[68,161,231]. The goal is to maximize the number of 1 bits in the bit string of a chosen dimension $n$. The objective function $f(x)$ of the minimization variant of this problem can be defined as $f(x) = n - \sum_{i=1}^{n} x_i$. The problem is unimodal, i.e., has only a single global optimum and no local optimum. It is fully separable, i.e., we can optimize each variable separately. The expected runtime of $(1+1)$ EA on OneMax is in $\Theta(n \ln n)$[59,161]. The average runtime $(1+1)$ FEA on OneMax seems to be slower by factor proportional to $n$, i.e., seems to be in $\mathcal{O}(n^2 \ln n)$[220].

**OR** Operations Research (or Operational Research) is the application of sciences such as mathematics and computer science to the management and organization of systems, organizations, enterprises, factories, or projects. It encompasses the development and application of problem-solving methods and techniques (such as mathematical optimization, simulation, queueing theory and other stochastic models) with the goal to improve decision-making and efficiency[52].

**OR-Library** is a library of benchmark instances for various OR problems[15,16] located at https://people.brunel.ac.uk/~mastjjb/jeb/info.html. It contains instances of the JSSP, 1D- and 2D-Packing, Knapsack, periodic TSPs, amongst others.

**Plateau** The Plateau problem is a benchmark problem for bit-string based black-box search[3,231]. It is structured exactly like Jump, but instead of being deceptive, the last $\omega$ bits to flip around the optimum are neutral. The objective function $f(x)$ of the minimization variant is $n - \|x\|_1$ if $\|x\|_1 = n$ or $\|x\|_1 \leq n - \omega$ hold, and $\omega$ otherwise, where $\|x\|_1 = \sum_{i=1}^{n} x_i$. This means that, the larger $\omega$, the larger the neutral plane around the optimum that an algorithm needs to cross. An $(1+1)$ EA has an expected runtime of $\Theta(n^\omega)$ on plateaus[3]. The average runtime of the $(1+1)$ FEA on Plateaus is slower by a factor seemingly proportional in $n$.

**PostgreSQL** An open source object-relational DBMS[75,164,175,197]. See https://postgresql.org for more information.

**Python** The Python programming language[107,132,148,211], i.e., what you will learn about in our book[211]. Learn more at https://python.org.

# Glossary XI

QAP The *Quadratic Assignment Problem* is an optimization problem where the goal is to assign a set of $n$ facilities to a set of $n$ locations[32,41,124,143]. Such an assignment can be represented as a permutation $x$ of the first $n$ natural numbers, where $x_i$ specifies the location where facility $i$ should be placed. For each QAP instance, a distance matrix $A$ is given, where $A[a, b]$ specifies the distance from location $a$ to location $b$, as well as a flow matrix $B$, where $B[i, j]$ is the amount of material flowing (e.g., daily) from facility $i$ to facility $j$. The objective function $f(x)$ then rates a permutation $x$ as $f(x) = \sum_{i=\mathbf{1}}^{n} \sum_{j=\mathbf{1}}^{n} A[x_i, x_j] * B[i, j]$. The QAP is $\mathcal{NP}$-complete[66,183] and even among this problem class, is considered as particularly hard for metaheuristics[17,32,143,192]. The QAP has applications such as building layout[51,72,125], keyboard layout[34], circuit design[73], wiring[189], and scheduling[188]. Typical benchmark instances can be found in QAPLIB. SOTA algorithms for the QAP include[17,81,153,158,195,196].

QAPLIB is a library of 136 benchmark instances for the Quadratic Assignment Problem (QAP)[31,33], 28 of which have not yet been solved to optimality[2] (at least by 2023).

Random Keys encoding is a decoding procedure $\gamma : \mathbb{X} \mapsto \mathbb{Y}$ that maps a continuous search space $\mathbb{X} \subset [0, 1]^n$ which is a subset of the $n$-dimensional real numbers to the space $\mathbb{Y}$ of permutations of the first $n$ natural numbers[13,14]. For example, on $n = 5$, the vector $x = (0.49, 0.91, 0.33, 0.75, 0.51)$ is translated to the permutation $y = (3, 1, 5, 4, 2)$, because $x_{\mathbf{3}} = 0.33$ has the smallest value, putting 3 at the beginning, $x_{\mathbf{1}} = 0.49$ the second-smallest, placing 1 second, ..., and $x_{\mathbf{2}} = 0.91$ being the largest value meaning that the last number in $y$ will be $2$[14]. The advantage of the encoding is that we can search in $\mathbb{X}$ applying arbitrary mathematical operators, crossover, and mutation procedures, but will always get correct permutations. The disadvantage is that $|\mathbb{X}| \gg |\mathbb{Y}|$ and that there actually is no meaning underpinning moves in $\mathbb{X}$ in relation to their projected result in $\mathbb{Y}$.

RLS Randomized local search retains the best solution $x_c$ discovered so far during the search and, in each step, it applies a unary search operator to this best-so-far solution $x_c$ and derives a new solution $x_n$. If the new solution $x_n$ is *better or equally good* when compared with $x_c$, i.e., not worse, then it replaces it, i.e., is stored as the new $x_c$. If the search space are bit strings of length $n$, then RLS uses a unary search operator that flips exactly one bit. This operator is the main difference to $(1 + 1)$ EA.

FRLS The RLS with FFA plugged in [231]. On the discrete bit-string domains, this algorithm has a very specific structure: Its unary search operator flips exactly one bit, distinguishing it from the unary operator in the $(1 + 1)$ FEA, which flips each bit with probability $1/n$. On other domains, such as permutations, we use the term synonymously with $(1 + 1)$ FEA.

# Glossary XII



RTS *Reactive Tabu Search* is a variant of TS where the length of the tabu list is adapted to the problem and to the current evolution of the search, and an escape strategy for diversifying the search is used when that is not sufficient[9–12]. It has successfully been applied to a wide range of problems, from the QAP[10,11], to Knapsack problems[9,11], the N-K landscape[9] to the training of neural networks[12].

SA *Simulated Annealing* is a SLS that sometimes accepts a worse solution[37,112,119,172,208]. The probability to do so decreases over time and with the difference in objective values, i.e.,is the lower the worse the new solution is. SA performed well on a variety of problems, including the QAP[156,200] and the JSSP[167].

SATLIB is a library of benchmark instances for the Maximum Satisfiability (MaxSAT) available at https://www.cs.ubc.ca/~hoos/SATLIB/index-ubc.html[104,105].

SLS *Stochastic local search* is a local search (LS), one of the simplest metaheuristic search paradigms[103]: A single "current" candidate solution is maintained and new solutions are *stochastically* sampled from its neighborhood, i.e., in a way that involves random numbers. If these solutions are deemed interesting based on their objective value, they replace the "current". The cycle continues. Basic examples are RLS and $(1+1)$ EA. More complex examples that add mechanisms to escape local optima on top of the SLS paradigm are TS and SA.

SOTA State-of-the-Art, i.e., the best-performing methods for a field or problem type

STSP A *symmetric Traveling Salesperson Problem* is a special case of the TSP, where the $n \times n$ distance matrix $A$ is symmetric, i.e., $A[a,b] = A[b,a] \forall a, b \in 1..n$.

Trap The Trap problem is a benchmark problem for bit-string based black-box search[67,162,231]. It is structured exactly like OneMax, however the worst and the best solution are swapped. This makes the problem highly deceptive, as following the "gradient" in the search space leads the search away from the global optimum. For bit strings of length $n$, the objective function $f(x)$ of the minimization variant is 0 if $\|x\|_{\mathbf{1}} = 0$ and $n - \|x\|_{\mathbf{1}} + 1$ otherwise, where $\|x\|_{\mathbf{1}} = \sum_{i=\mathbf{1}}^{n} x_i$. An $(1+1)$ EA will converge to the resulting local optimum and thus have an expected runtime of $\Theta(n^n)$[67]. In experiments, the $(1+1)$ FEA achieves polynomial runtime on Trap, seemingly in $\mathcal{O}(n^{\mathbf{2}} \ln n)$[220,221,231].

# Glossary XIII



TS *Tabu Search* is one of the most successful SLSes[87,88,90,114]. In order to avoid returning to previous solutions, it declares certain solutions or search moves as "tabu." For example, if a bit in a discrete problem has been flipped by the search operator, it may be forbidden to flip the bit again for some time, thus preventing the algorithm from getting trapped in a cycle. This rule may be overridden if a new best-so-far solution is discovered that outperforms all previous solutions, in which case it would be accepted (this is the so-called aspiration criterion). TS has performed well on the QAP[10,11,155,158,194,195] and on the JSSP[163,170,171,187,232].

TSP In an instance of the *Traveling Salesperson Problem*, also known as *Traveling Salesman Problem*, a set of $n$ cities or locations as well as the distances between them are given[4,97,130,134,212,222]. The goal is to find the shortest round-trip tour that starts at one city, visits all the other cities one time each, and returns to the origin. The TSP is one of the most well-known $\mathcal{NP}$-hard combinatorial optimization problems[85,97,126]. The TSP has many applications[4], including in transportation, logistics, (circuit) plotting and drilling problems[96], telescope aiming, and genome sequencing. Solutions are often encoded as permutations $x$ of the first $n$ natural numbers, where each city gets a number and the order of numbers in the permutation corresponds to the order of cities in the tour. The distances between the $n$ cities are usually given in a $n \times n$ distance matrix $A$, where $A[a, b]$ is the distance from city $a$ to city $b$ with $a, b \in 1..n$. Then, the objective function subject to minimization is $f(x) = A[x_n, x_{\mathbf{1}}] + \sum_{i=\mathbf{1}}^{n-\mathbf{1}} A[x_i, x_{i+\mathbf{1}}]$. Typical benchmark instances can be found in TSPLib[180,181], the 8th DIMACS challenge[115–117], and the *World TSP* repository[48].

TSPLib is a library of benchmark instances for the Traveling Salesperson Problem (TSP)[180,181]. Its original URL was available until 2025 and then became defunct. The repository https://github.com/shredderzwj/TSPLIB contains the data. Additionally, you may also check https://github.com/thomasWeise/tspSuite, which contains all the TSPLib instances as well as Java code to load them. The TSPLib contains 110 instances of the symmetric Traveling Salesperson Problems (STSPs) with scales $n$ ranging from 14 to 85 900, but 93 instances have less than 2000 nodes. There also are 19 asymmetric Traveling Salesperson Problems (ATSPs) with scales $n \in 17..443$.

# Glossary XIV



**TTP** The *Traveling Tournament Problem* is the combinatorial optimization problem of both efficiently and fairly organizing a tournament of $n$ teams that play against each other in a pairwise fashion[71,226]. The efficient part boils down to arranging the games such that the total travel length is short, which is somewhat similar to the classical TSP. Initially, each team is at its home location. On each day, a team needs to travel if its scheduled game is not at its present location. On the last day, each team may need to travel back home unless their last game is a home game. The total travel length sums up the lengths of all travels over all teams. The fair part is represented in several constraints, such as doubleRoundRobin, compactness, maxStreak, and noRepeat. The TTP is $\mathcal{NP}$-hard[205].

**TwoMax** The TwoMax problem is a benchmark problem for bit-string based black-box search[83,203,231]. It has a global and a local optimum, both of about the same size, at opposite ends of the search space. For bit strings of length $n$, the objective function $f(x)$ of the minimization variant is 0 if $\|x\|_1 = n$ and $1 + n - \max\{\|x\|_1, n - \|x\|_1\}$ otherwise, where $\|x\|_1 = \sum_{i=1}^{n} x_i$. An $(1+1)$ EA has a 50/50 chance to converge to the local or global optimum and thus has an expected runtime in $\Theta(n^n)$[82,83]. In experiments, the $(1+1)$ FEA achieves polynomial runtime seemingly in $\mathcal{O}(n^2 \ln n)$ on TwoMax[220,221,231].

**URI** A *Uniform Resource Identifier* is an identifier for an abstract or physical resource in the internet[230]. It can be a URL, a name, or both. URIs are supersets of URLs. The connection strings of the PostgreSQL DBMS are examples of Uniform Resource Identifiers (URIs).

**URL** A *Uniform Resource Locator* identifies a resource in the World Wide Web (WWW) and a way to obtain it by describing a network access mechanism. The most notable example of URLs is the text you write into web browsers to visit websites[21]. URLs are subsets of URIs.

**VND** *Variable Neighborhood Descend*; see VNS. VND in combination with GRASP has been applied to the 2BP|O|F in [168].

# Glossary XV



VNS *Variable Neighborhood Search* is a metaheuristic framework combining a systematic change of neighborhood with LS[98,159]. A *neighborhood* structure in LS defines which solutions can be reached from a given solution within one search step. Usually, one uses a single unary operator and, hence, has a single neighborhood structure. However, for many problems, we can easily design many operators. For permutations, for example, one operator may swap two elements, another one may reverse a section of the permutation, and yet another one, while yet another one may rotate a subsequence one step to the left or right[212]. In the VNS framework, a multiple neighborhoods are investigated in a systematic fashion. Assume that we have a list of neighborhood structures. Variable Neighborhood Descend (VND) works in a deterministic fashion. It finds the best solution that can be reached in one step from the current solution under the current neighborhood. If this is better than the current solution, we accept it and continue with the first neighborhood. Otherwise, we continue with the next neighborhood in the list. In Basic VNS, instead of picking the best move from the current neighborhood, the first improving move is used. Also, at the beginning of each cycle, a random point from the current neighborhood around the current solution is chosen as starting point.

WWW World Wide Web[20,53]

$i!$ The factorial $a!$ of a natural number $a \in \mathbb{N}_1$ is the product of all positive natural numbers less than or equal to $a$, i.e., $a! = 1 * 2 * 3 * 4 * \cdots * (a-1) * a$[38,69,147]. In Python, the module `math` provides the function `factorial` for computing the factorial.

$i..j$ with $i, j \in \mathbb{Z}$ and $i \leq j$ is the set that contains all integer numbers in the inclusive range from $i$ to $j$. It always holds that $i..j \subset \mathbb{Z}$. For example, 5..9 is equivalent to $\{5, 6, 7, 8, 9\}$

$A$ the $n \times n$ distance matrix. $A[a, b]$ indicates the distance between locations $a$ and $b$ with $a, b \in 1..n$. A distance matrix can be symmetric, in which case $A[a, b] = A[b, a]\ \forall a, b \in 1..n$. Distance matrices represent the problem instance data in TSPs and part of the instance data of QAPs.

$B$ the $n \times n$ flow matrix. $B[a, b]$ is the flow of goods between locations $a$ and $b$ with $a, b \in 1..n$. Flow matrices are one half of the problem instance data of QAPs.

# Glossary XVI

$f(y)$ The *objective function* $f : \mathbb{Y} \mapsto \mathbb{R}$ can compute a cost or rating for each candidate solution $y$ in the solution space $\mathbb{Y}$. The smaller this value, the better the solution, i.e., in the context of this work, objective functions are subject to minimization. Objective functions are not necessarily continuous and it is also quite common that they have integer or natural numbers as results, e.g., we often have $f : \mathbb{Y} \mapsto \mathbb{N}_0$ instead of $f : \mathbb{Y} \mapsto \mathbb{R}$.

$\gamma(x)$ The decoding function $\gamma : \mathbb{X} \mapsto \mathbb{Y}$ translates point from the search space $\mathbb{X}$ to one point $y$ in the solution space $\mathbb{Y}$. It is a very common case that $\mathbb{X} = \mathbb{Y}$ and that the decoding function $\gamma$ is the identity mapping. Examples for encodings that translate from a simple search space $\mathbb{X}$ to a complex solution data structure $\mathbb{Y}$ are the IBL algorithm that translates (potentially signed) permutations to two-dimensional packings on the 2BP, the permutation-based encoding of Gantt charts for the JSSP from[23,24,86,185,213], or the permutation-based encoding of game plans for the Traveling Tournament Problem (TTP) from[226]. Encodings can also translate between types of search spaces. The Random Keys encoding, for example, translates from $[0, 1]^n$ to permutations of the first $n$ natural numbers..

$k$ the number of jobs in an instance of the JSSP.

$m$ the number of machines in an instance of the JSSP.

# Glossary XVII



$n$ We usually use the value $n \in \mathbb{N}_0$ to denote the *scale*, i.e., the size of the problem to solve. This notion is very general in computer science and not limited to optimization. For example, if we sort a list or search in a list, the scale $n$ is the length of the list, i.e., the number of elements in it. In a TSP, this may be the number of cities. In a discrete benchmark problem such as OneMax or Trap, this may be the number of bits over which the problem is defined. In a QAP, this may be the number of facilities and locations. In a MaxSAT problem, this may be the number of variables that are input to the Boolean formula that we want to become true. There are also problems where the scale is not defined straightforwardly like this. For example, in a JSSP, we have a number of machines and a number of jobs, which together define the size of the problem. But even then, the scale $n$ relates to the size of the solution space $\mathbb{Y}$, and, hence, also of the search space $\mathbb{X}$, which usually grow exponentially with $n$. And this is already the main issue: The larger $n$, the (exponentially) larger the number of possible solutions and thus, the larger the runtime of algorithms. This is known as the "curse of dimensionality"[18,40]. Scales $n$ are also often used in conjunction with the $\mathcal{O}$-notation to denote algorithm complexity. For example, comparison-based sorting a list of $n$ items takes $\Theta(n \log n)$ steps[123] (because they need $\lceil \log_2 n! \rceil$ comparisons in the worst case).

$\mathbb{N}_0$ the set of the natural numbers *including* 0, i.e., 0, 1, 2, 3, and so on. It holds that $\mathbb{N}_0 \subset \mathbb{Z}$ and that $\mathbb{N}_0 = \mathbb{N}_1 \cup \{0\}$.

$\mathbb{N}_1$ the set of the natural numbers *excluding* 0, i.e., 1, 2, 3, 4, and so on. It holds that $\mathbb{N}_1 \subset \mathbb{Z}$ and that $\mathbb{N}_1 = \mathbb{N}_0 \setminus \{0\}$.

$\mathcal{NP}$ is the class of computational problems that can be solved in polynomial time by a non-deterministic machine and can be verified in polynomial time by a deterministic machine (such as a normal computer)[94].

$\mathcal{NP}$-complete A decision problem is $\mathcal{NP}$-complete if it is in $\mathcal{NP}$ and all problems in $\mathcal{NP}$ are reducible to it in polynomial time[94,178]. A problem is $\mathcal{NP}$-complete if it is $\mathcal{NP}$-hard and if it is in $\mathcal{NP}$.

$\mathcal{NP}$-hard A problem is $\mathcal{NP}$-hard if all problems in $\mathcal{NP}$ are reducible to it in polynomial time[94]. Algorithms that guarantee to find the correct solutions of $\mathcal{NP}$-hard problems[39,45,129] have a worst-case runtime that grows exponentially with the problem scale.

$\Omega(g(x))$ If $f(x) = \Omega(g(x))$, then there exist positive numbers $x_0 \in \mathbb{R}^+$ and $c \in \mathbb{R}^+$ such that $f(x) \geq c * g(x) \geq 0$ for all $x \geq x_0$[50,121,122]. In other words, $\Omega(g(x))$ describes a lower bound for function growth.

$\mathcal{O}(g(x))$ If $f(x) = \mathcal{O}(g(x))$, then there exist positive numbers $x_0 \in \mathbb{R}^+$ and $c \in \mathbb{R}^+$ such that $0 \leq f(x) \leq c * g(x)$ for all $x \geq x_0$[6,50,121,122,127]. In other words, $\mathcal{O}(g(x))$ describes an upper bound for function growth.

# Glossary XVIII

$\Theta(g(x))$ If $f(x) = \Theta(g(x))$, then $f(x) = \mathcal{O}(g(x))$ and $f(x) = \Omega(g(x))$[121,122], meaning that there exist positive constants $c_1, c_2, x_0 \in \mathbb{R}^+$ such that $0 \leq c_1 g(x) \leq f(x) \leq c_2 g(x)$ for all $x \geq x_0$[50]. In other words, $\Theta(g(x))$ describes an exact order of function growth.

$o(g(x))$ If $f(x) = o(g(x))$, then there exist positive numbers $x_0 \in \mathbb{R}^+$ such that $0 \leq f(x) \leq c * g(x)$ for all $x \geq x_0$ *all* $c \in \mathbb{R}^+$[50]. Different from $\mathcal{O}(g(x))$, $o(g(x))$ holds for *all* constants $c > 0$, whereas $\mathcal{O}(g(x))$ implies the existence of *some* constant $c > 0$. This basically means that $f(x)$ becomes insignificant compared to $g(x)$ as $x_0$ grows toward infinity. It is often written that $\lim_{x_0 \to \infty} \frac{f(x)}{g(x)} = 0$. In other words, $o(g(x))$ describes an upper bound for function growth that is not asymptotically tight, i.e., an upper bound that grows faster than $f(x)$.

$\mathbb{R}$ the set of the real numbers.

$\mathbb{R}^+$ the set of the positive real numbers, i.e., $\mathbb{R}^+ = \{x \in \mathbb{R} : x > 0\}$.

$\omega(g(x))$ If $f(x) = \omega(g(x))$, then for every positive constant $c > 0$, there exists a number $x_0 \in \mathbb{R}^+$ such that $0 \leq c * g(x) < f(x)$ for all $x \geq x_0$[50]. In other words, $f(x) = \omega(g(x))$ if and only if $g(x) = o(f(x))$. In other words, $\omega$ describes a lower bound that is not asymptotically tight.

$\mathbb{X}$ The search space $\mathbb{X}$ is the space through which an optimization algorithm navigates in order to find good solutions. The search operators of metaheuristics are used to sample points $x \in \mathbb{X}$, for example. On many benchmark problems, the objective functions $f$ directly evaluate the elements $x \in \mathbb{X}$, i.e., are defined as $f : \mathbb{X} \mapsto \mathbb{R}$. However, very often these elements $x$ are not the same data structures that the user is interested in and/or that the objective functions evaluate: It is common that the search space $\mathbb{X}$ is a simple and plain data structure that can be tackled with standard operations or mathematical tools, such as real vectors $\mathbb{R}^n$, permutations, or bit strings, whereas the solution space $\mathbb{Y}$ can be something more complicated. Then, the user only cares about the solutions $y \in \mathbb{Y}$ and not about the search space $\mathbb{X}$. A decoding $\gamma : \mathbb{X} \mapsto \mathbb{Y}$ is employed to translate the $x \in \mathbb{X}$ to $y = \gamma(x)$ with $y \in \mathbb{Y}$.

$x$ a point in the search space $\mathbb{X}$.

# Glossary XIX

$\mathbb{Y}$ the set of candidate solutions of an optimization problem is called the *solution space* $\mathbb{Y}$. The solution space corresponds to the data structure holding the candidate solutions $y$. These are the elements that are passed to the user after the optimization process is completed. They are also the parameters of the objective function(s) $f : \mathbb{Y} \mapsto \mathbb{R}$.

$y$ a candidate solution $y \in \mathbb{Y}$ is one possible solution to an optimization problem. After the optimization process is completed, one or multiple such solutions are passed to the user.

$\mathbb{Z}$ the set of the integers numbers including positive and negative numbers and 0, i.e., . . . , -3, -2, -1, 0, 1, 2, 3, . . . , and so on. It holds that $\mathbb{Z} \subset \mathbb{R}$.